AMESIM

E. Cabrol

Création **17/05/2016**
Dernière MAJ **01/09/2022**

Ce document compile des notes personnelles prises au cours du temps. Il ne s'agit donc pas d'un user manual exhaustif (la doc est plutôt bien faite, et riche d'exemples), mais plutôt d'un aide-mémoire sur des points qui me semblent utiles.

Ce document est sous licence Creative Commons BY-NC-SA. Pour consulter une copie de la licence, visitez https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.fr

NB : de nombreuses captures d'écran ont été faites sur d'anciennes versions.

1 INTRODUCTION 3

2 PRESENTATION DE L'INTERFACE 6

3 QUELQUES ASTUCES 8

4 LES QUATRE MODES 11

5 COMPOSANTS 13

6 MODELES SIMPLES 14

7 SIMULATION 16

8 POST-PROCESSING 18

9 ANALYSE DE MODELE 20

10 UTILISATION D'UN MODELE SUR CIBLE TEMPS-REEL 26

11 DOE - OPTIMISATION 28

12 SOUS-MODELES (AMESET) 30

13 SUPERCOMPOSANTS / SUPERCOMPONENTS 33

14 AMESIM FOR SYSTEM ANALYST 41

15 CO-SIMULATION 42

16 SCRIPTING 64

17 VEHICLE DYNAMICS 68

18 VEHICLE MODELS FROM PWT LIBRARY 97

19 CONTROLE 99

20 LINK WITH CAD : CAD2AME 99

21 NEURAL NETWORKS 100

22 RELEASES – NOUVEAUTES 102

23 ANNEXES 103

24 PUBLICATIONS 107

25 REFERENCES 108

1 INTRODUCTION

Amesim est un logiciel de **simulation multiphysique**.

Créé par Michel Lebrun en 1986, édité initialement par Imagine SA, il a ensuite été racheté par LMS International en 2007, qui a été rachetée par Siemens fin 2011 (voir annexe §23.1)

Il s'inspire du formalisme Bond Graph : chaque lien véhicule des variables de type Effort – Flux (voir annexe §23.1.3)

1.1 LES DIFFERENTS PRODUITS

La suite logicielle est constituée de :

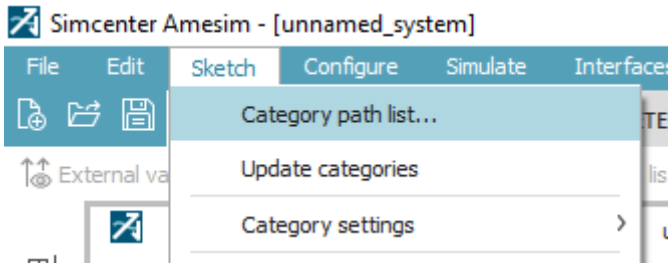
- *Amesim* :
  - Produit principal pour la modélisation et la simulation de systèmes dynamiques ;
  - Adaptatibilité, réutilisation, évolutivité, solveurs inégalés, fonctionnalités avancées.
- *AMECustom* :
  - Personnalisation de modèles et protection de la propriété intellectuelle ;
  - Adaptation et personnalisation de l'apparence des modèles, création et publication de catalogues prêts à l'emploi, facilite l'échange de modèles ainsi que leur chiffrement.
- *AMERun* :
  - Simulation de modèles déjà existants ;
  - Version run-only, adaptée pour les non-experts pour partager les modèles validés, solveurs standards et fonctionnalités avancées d'Amesim.
- *AMESet* :
  - Développement de nouveaux composants ;
  - Capitalisation de savoir-faire, plateforme ouverte, outil de développement intégré pour créer de nouveaux sous-modèles.

1.2 LES BIBLIOTHEQUES

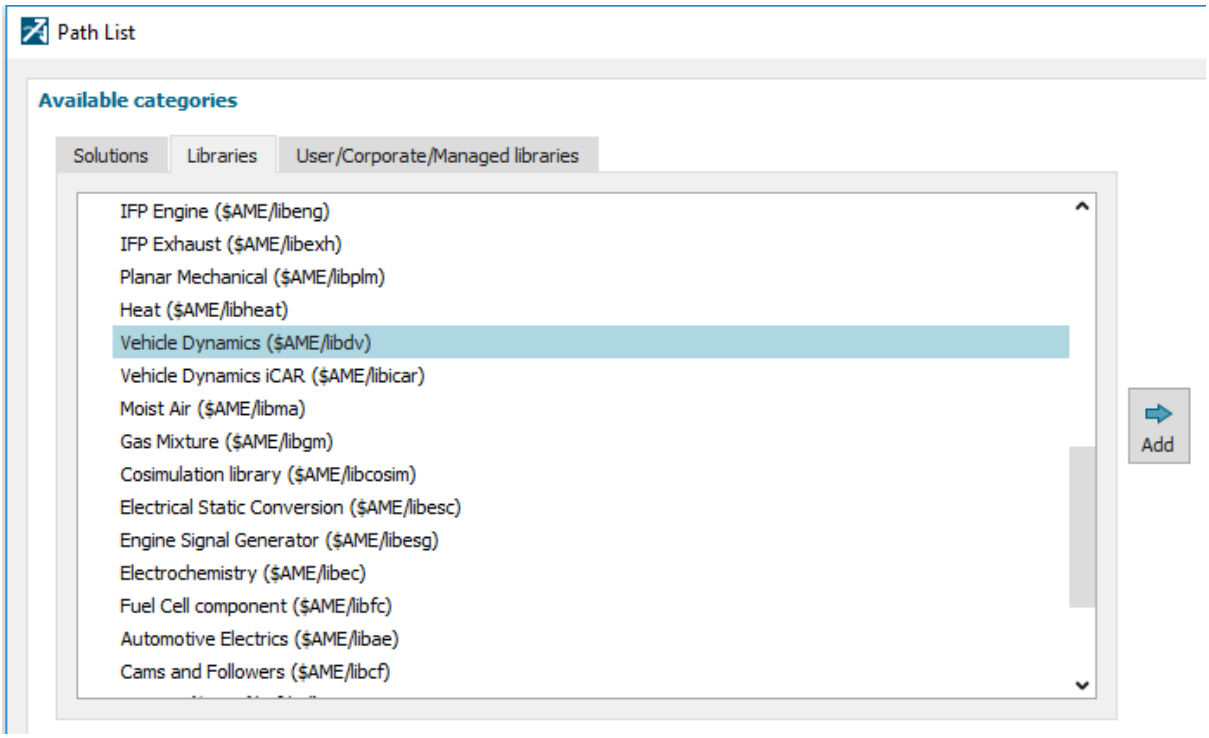
Amesim est fourni avec deux bibliothèques par défaut : mécanique et signal. Les autres bibliothèques (environ 30) doivent être achetées séparément. On en trouve pour les disciplines suivantes :

- contrôle-commande
- électrique
- fluide
- thermique
- hydraulique
- moteur à combustion interne …

Si les bibliothèques ne sont pas visibles (alors qu'on dispose de la licence …), voir dans Sketch / Category path list :



Sélectionner dans Libraries la bibliothèque de son choix, puis Add :



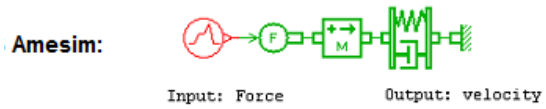
1.3 COMPARAISON AMESIM-SIMULINK

Amesim : modélisation « orientée technologique » (mécanique, hydraulique), plus économe en blocs que les diagrammes Simulink, plus simple à construire.

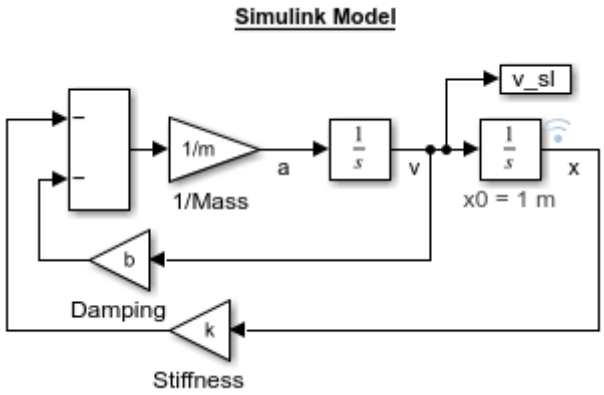
NB : comparable à SimMechanics

Sélection automatique des solveurs

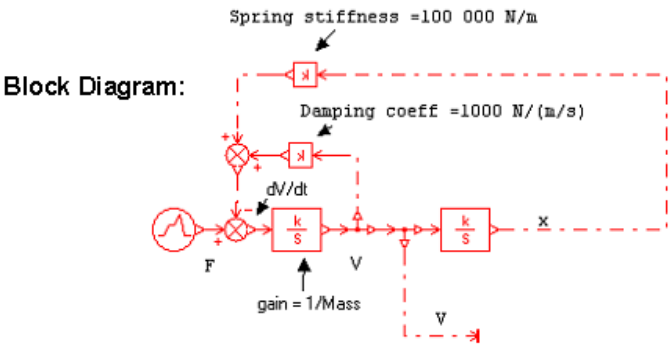
Exemple système masse/ressort/amortisseur



Le même en Simulink

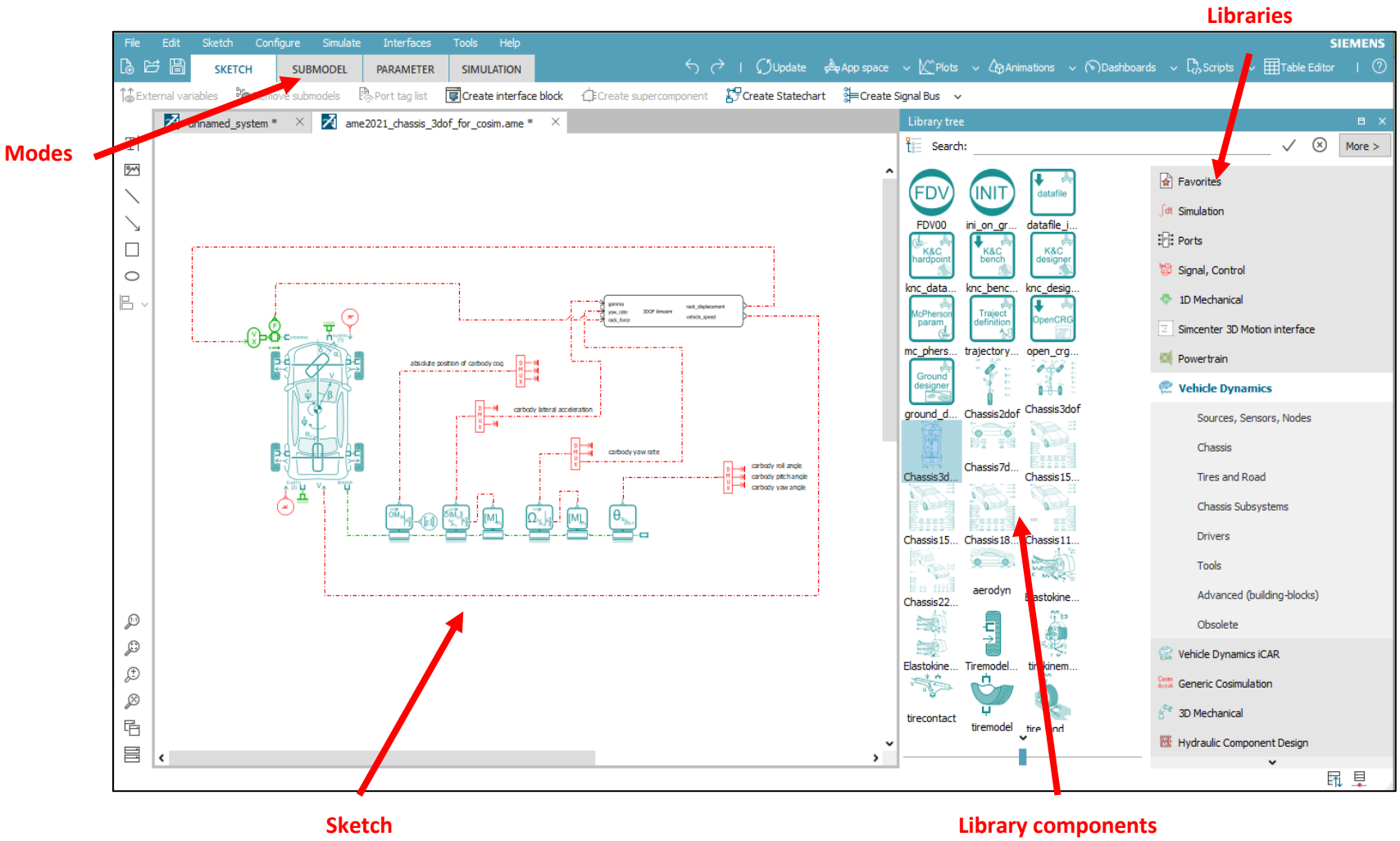


On peut aussi faire du « block diagram » avec Amesim :



… mais ça n'a d'intérêt que si les composants disponibles dans les librairies ne répondent pas au besoin.

2 PRESENTATION DE L'INTERFACE

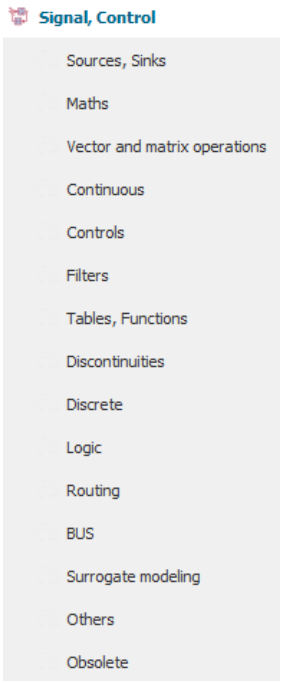


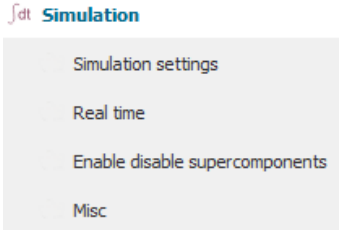
Confidential C

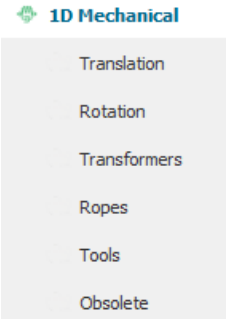
Les librairies contiennent des catégories, à leur tour divisées en sous-catégories, etc …

La librairie standard d'Amesim propose par défaut 3 catégories :

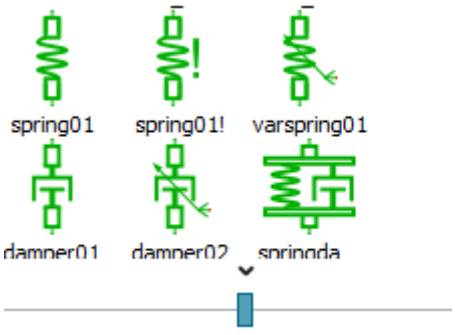
- Signal
- Simulation
- Mechanical







Un slider dans le library browser permet de zoomer :



Confidential C

3 QUELQUES ASTUCES

3.1 RACCOURCIS USUELS

Ctrl+T : forcer la recompilation

Ctrl+B : batch mode

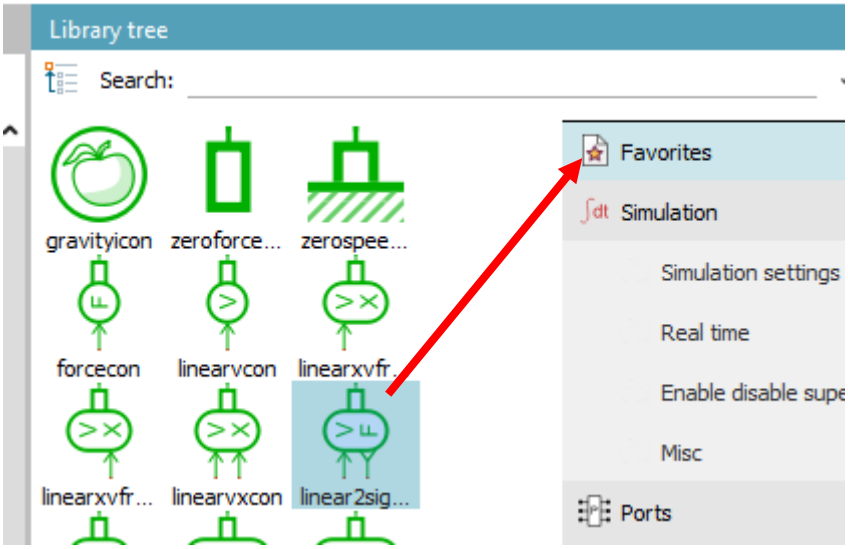
Ctrl+G : global parameters

Ctrl+R : faire tourner un composant

Space : dupliquer un composant

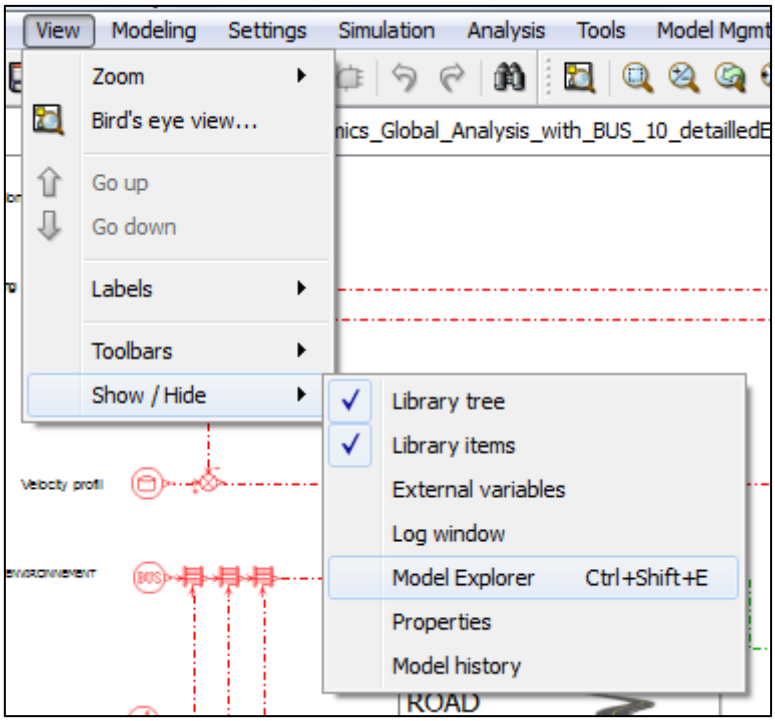
3.2 CREATION DE FAVORIS

Les faire glisser depuis la librairie :



3.3 MODEL EXPLORER

Accessible depuis le menu View ou par le raccourci Ctrl+Shift+E :



3.4 COMPARAISON DE MODELES

Configure / Compare Systems

Les deux modèles doivent être ouverts en parameter mode.

3.5 STOCKER DES DONNEES/TABLES AVEC LE MODELE

Créer un dossier ${circuitname}_.datastore, et placer les données dedans.

Comme tous les fichiers/dossiers ${circuitname}_.*, ça sera compressé à la fermeture du .ame, et réouvert ensuite.

Confidential C

Le nom de ce sous-dossier est (quasiment) libre, il faut juste éviter les quelques noms déjà réservés par Amesim (genre *data*). On peut même utiliser un global parameter pour le définir, puis pointer dessus avec par exemple ${data_directory}.

Confidential C

4 LES QUATRE MODES

4.1 SKETCH MODE

C'est le mode dans lequel on conçoit le modèle. On y ajoute et connecte les composants.

4.2 SUBMODEL MODE

Permet de sélectionner les configurations des sous-modèles correspondant aux composants placés dans le sketch. On peut utiliser le sous-modèle par défaut avec la fonctionnalité *Premier Submodel* :

To use the *Premier submodel* facility, do one of the following:

- Click on the *Premier submodel* button in the toolbar  .
- Use Ctrl + I.
- Select **Modeling > Premier submodel.**

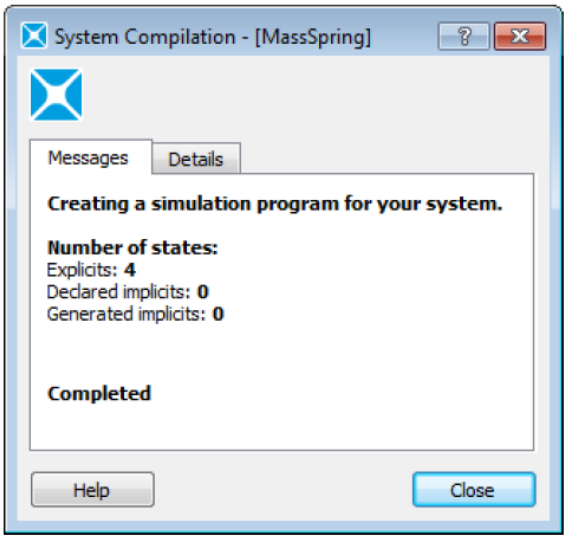
4.3 PARAMETER MODE

Click on the *Parameter* mode button in the horizontal toolbar  , or use the **F11** key.

Ce mode permet de « compiler le modèle », c'est-à-dire de générer un code exécutable. On peut forcer la recompilation via Ctrl+T.

La fenêtre de compilation apparaît et indique à la fois le nombre de :

- variables d'état
- variables implicites associées à des équations algébriques



Confidential C

On peut ensuite changer les valeurs des paramètres en cliquant sur chacun des composants du modèle. Il est également nécessaire de spécifier les valeurs initiales de toutes les variables d'état.

On peut définir des paramètres globaux (Ctrl+G, ou menu Settings)

NB : les unités de ces paramètres globaux sont données à titre indicatif !

4.4 SIMULATION MODE

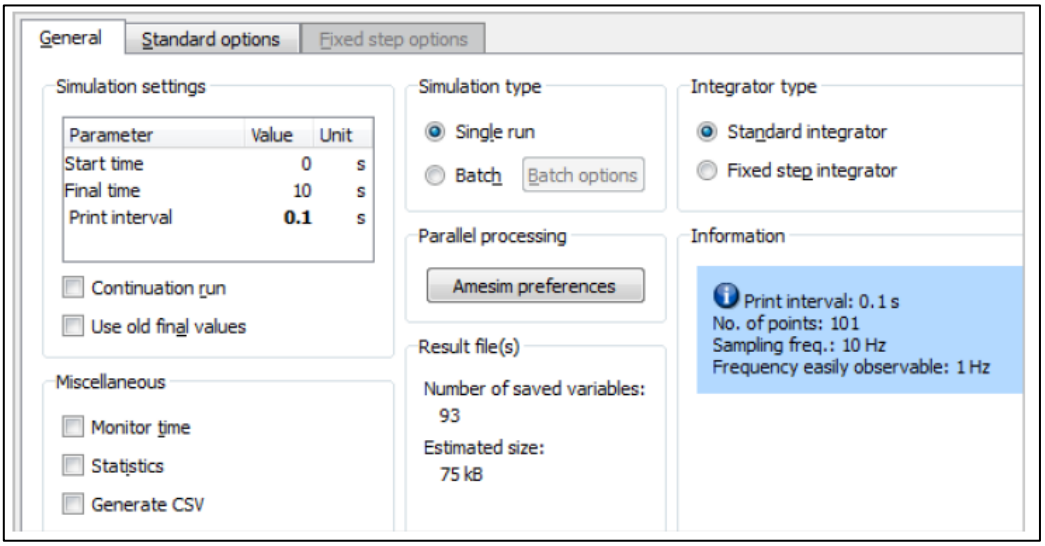
F12 ou 

pour passer en simulation

puis

Ctrl+U ou 

pour régler les paramètres du calcul et en particulier les instants de début et fin :

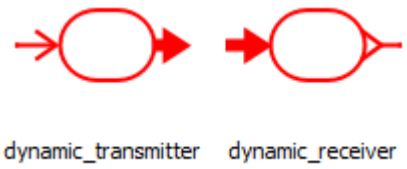


Confidential C

5 COMPOSANTS

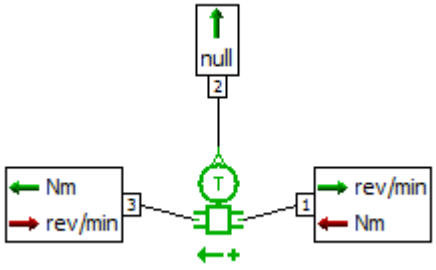
5.1 GOTO/FROM

Les équivalents des Goto/From de Simulink sont les dynamic transmitters et receivers (SIGTRANS0/SIGRECEI0) dans la librairie Signal/Routing

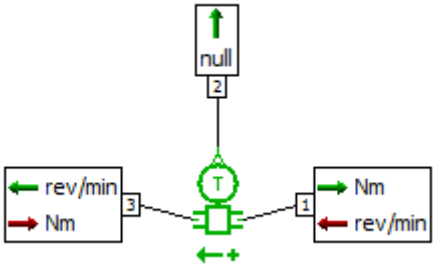


5.2 CAPTEURS DE COUPLE

« torque sensor » dans la librairie



MECTS0A

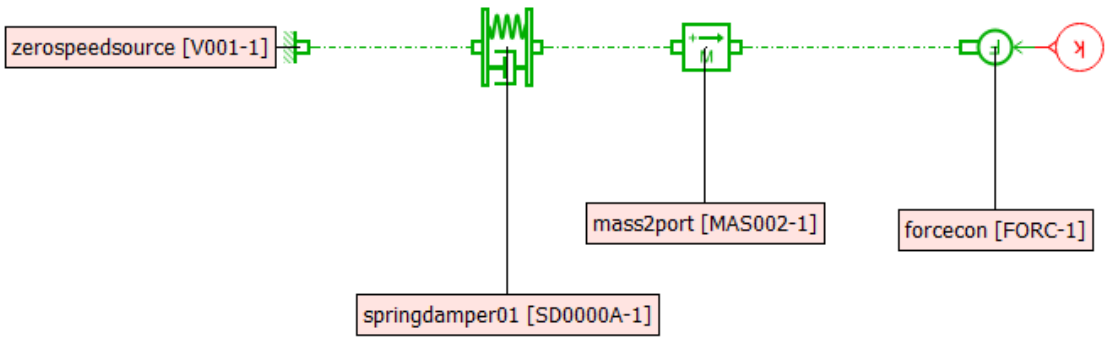


MECTS0B

Confidential C

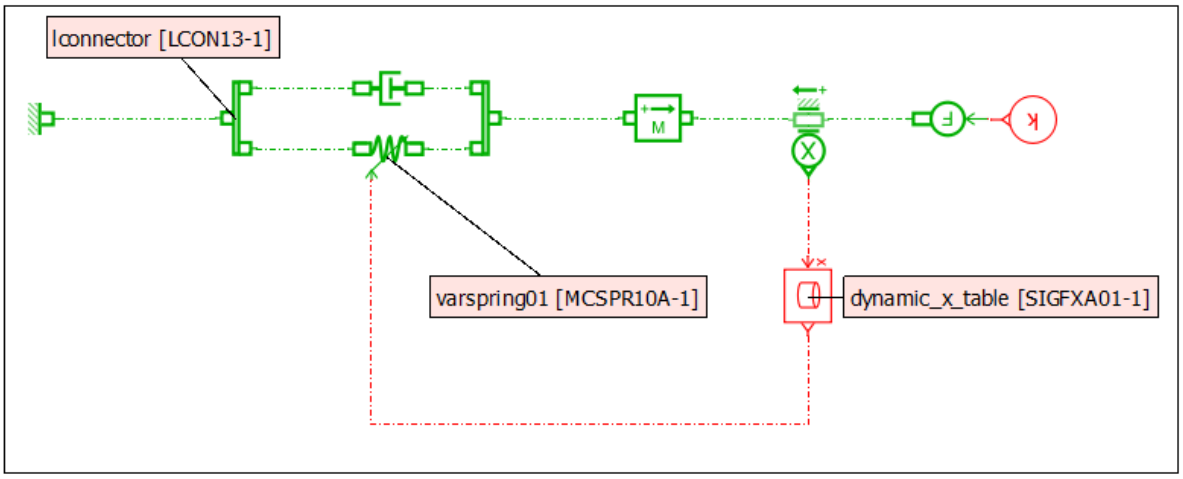
6 MODELES SIMPLES

6.1 MASSE-RESSORT SOUS EFFORT



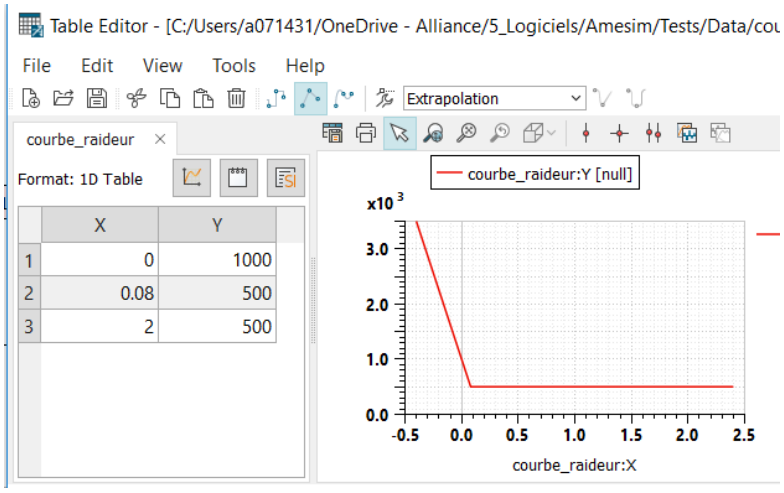
6.2 MASSE-RESSORT-AMORTISSEUR AVEC RAIDEUR VARIABLE

La raideur est passée au composant varspring par le biais d'une table prenant le déplacement de la masse en input



Attention aux signes ! Ici le déplacement de la masse est négatif, mais comme le displacement sensor a la convention de signe opposée, il faut définir une table avec des valeurs positives :

Confidential C



Confidential C

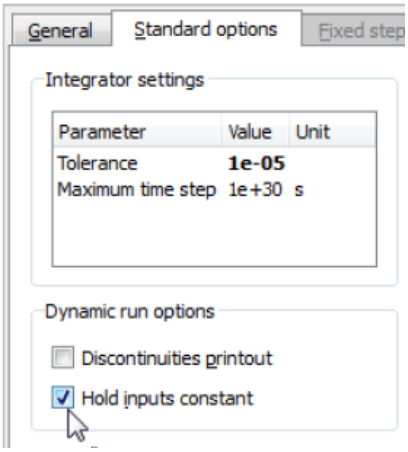
7 SIMULATION

7.1 OBTENIR UNE POSITION D'EQUILIBRE

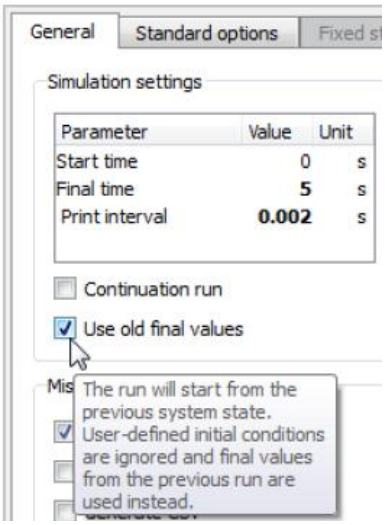
Il est fréquent qu'un modèle ne parte pas d'une situation d'équilibre. Il est alors préférable d'obtenir cet équilibre avant de soumettre le système aux inputs.

Le meilleur moyen de procéder est de décomposer le run en deux :

- une première phase en cochant *Hold inputs constant* dans les Standard Options



- une seconde en décochant cette case pour la remplacer par *Use old final values* (General)



L'inconvénient de cette approche est que la phase de stabilisation peut être longue.

Confidential C

Une alternative (moins robuste) est de recourir à un *Stabilizing run*.

	Hold inputs constant	Stabilizing run
Reliability	If the system is stable, the method is normally reliable.	The method does not always work and when it does work, it may give an equilibrium position which is not accessible from the starting value.
Uniqueness of solution	Normally if a solution is obtained, it does not vary significantly with parameters such as integrator tolerance.	If there are multiple equilibrium positions, it is difficult to tell in advance which one will be found.
CPU time	Can be significantly higher.	If the method works, CPU times are normally lower.
Parameters available to aid convergence	We can vary all the dynamic run parameters: tolerance, error type, maximum step size, standard/cautious, final time.	The following parameters will influence the success of the run and the CPU time: tolerance, error type, standard/cautious.

Do not think it is always easier to find a steady state solution directly than do a dynamic solution. Often the opposite is true.

7.2 SOLVER TYPES

These options are available in the *Run Parameters* dialog box in the *Standard options* tab. With numerical algorithms there is often a compromise to be made between speed and reliability. When the **Regular** solver type is selected, the Amesim solver is fairly aggressive trying to get a fast solution. With **Cautious** selected, slower and more reliable strategies are used which often lead to higher CPU times. These options apply to both stabilizing runs and dynamic runs but the effect is greater for stabilizing runs. However, with some specific systems the Regular option is more reliable than Cautious.

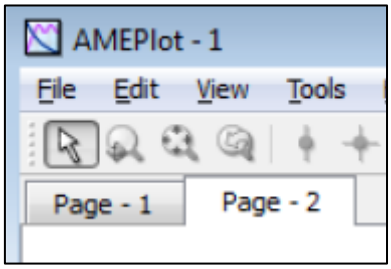
Confidential C

8 POST-PROCESSING

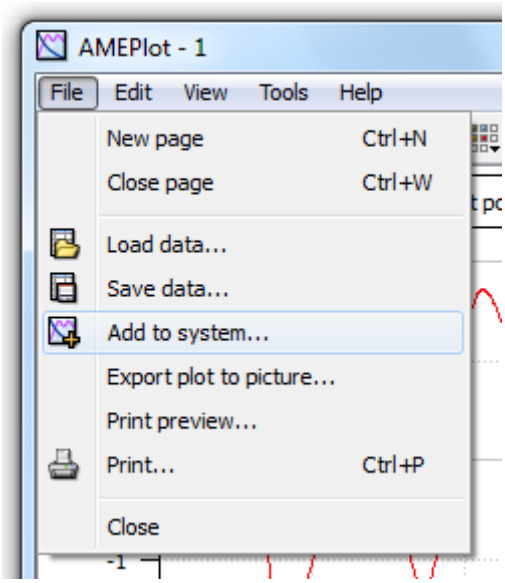
8.1 TRACE DE COURBES

Une fois la simulation terminée on peut sélectionner une variable associée à un composant et la faire glisser sur le sketch.

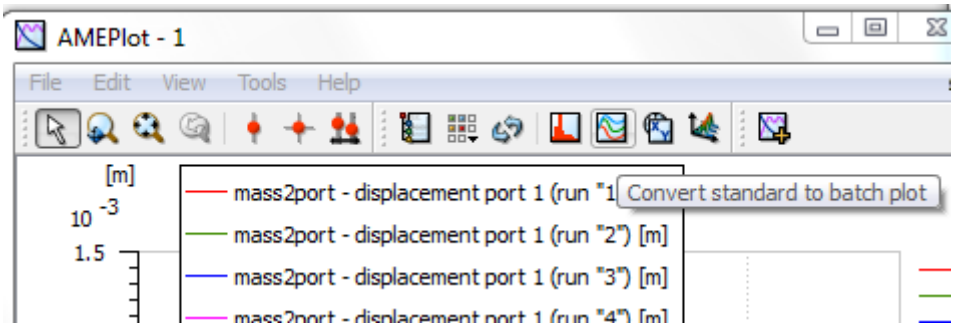
Une fenêtre plot peut être constituée de plusieurs pages (File / New page)



On peut sauver le template d'un graphique : `Add to system`



En mode batch, cliquer sur `Convert standard to batch plot` puis sur une courbe :



Confidential C

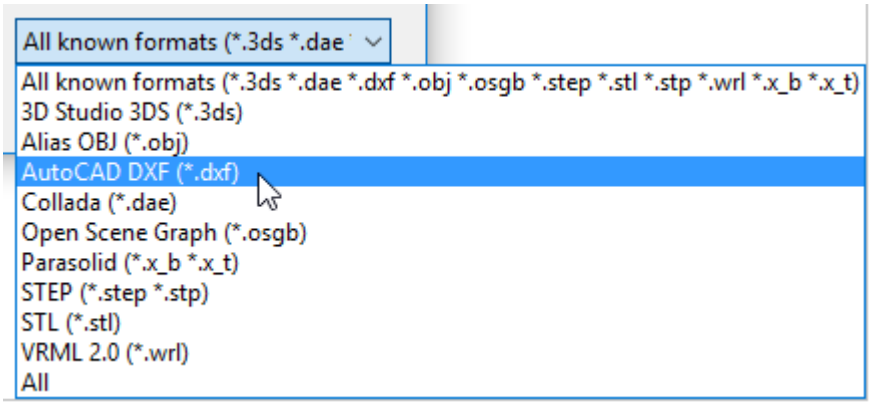
8.2 EXPERIMENT VIEW

You can use the *Experiment view* to save and load sets of parameters and associated results. In this way you can vary the parameters of a model, save the model in its differing states (as 'experiments'), and then quickly apply the different parameter sets and their corresponding results without needing to reconfigure your model and re-run simulations.

8.3 ANIMATION

8.3.1 MODÈLES 3D

Formats lus par Amesim



Confidential C

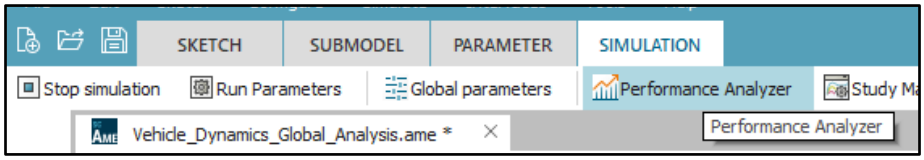
9 ANALYSE DE MODELE

9.1 ANALYSE DE PERFORMANCE

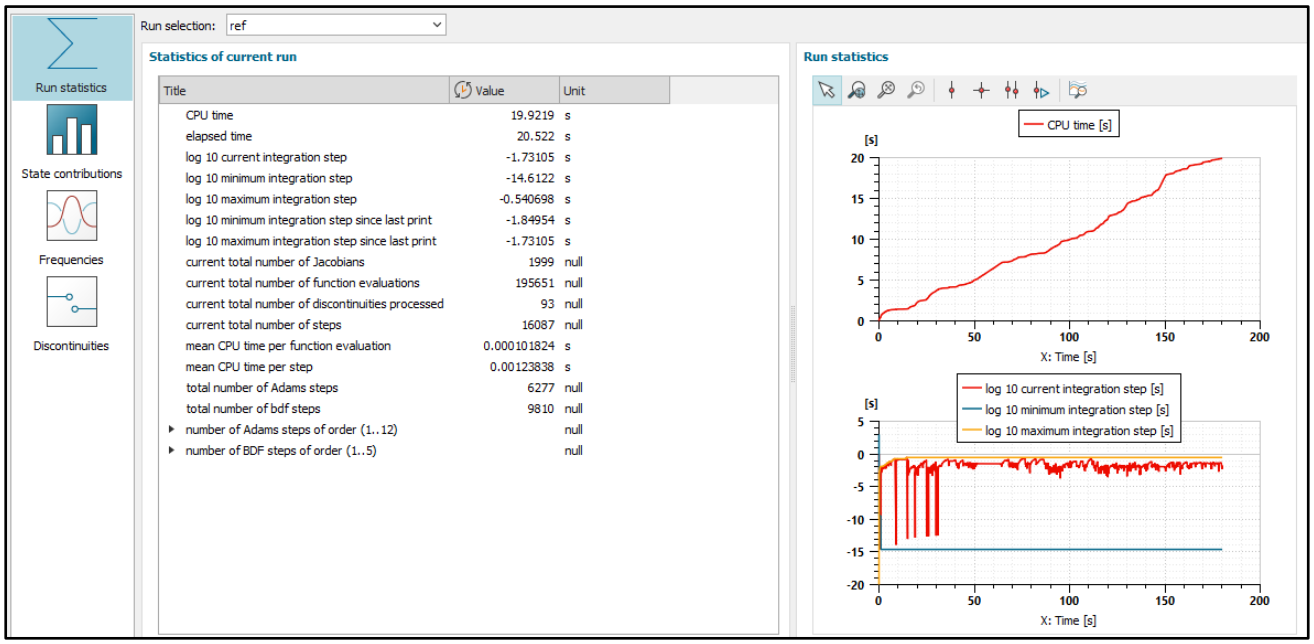
Le composant Run Stats doit être placé sur le sketch :



L'outil Performance Analyzer livre des informations utiles une fois la simulation terminée :



Dans l'onglet Run Statistics, on dispose d'informations sur l'évolution du pas de temps lors de la simulation (utile quand on est en pas variable, évidemment) :



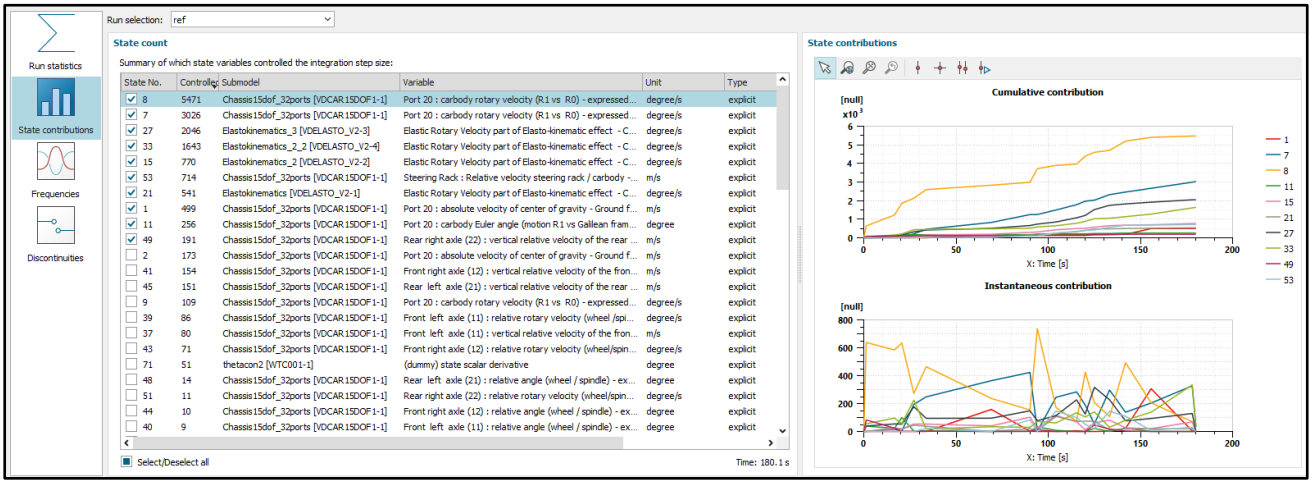
A noter cependant qu'un chute soudaine du pas de temps mini peut ne pas être capturée si la simu freeze peu après, car les infos présentes ici ne sont loguées qu'en fonction du *Print interval* sépcifié dans les *Run parameters*



Il pourra être utile de réduire ce pas de temps si besoin.

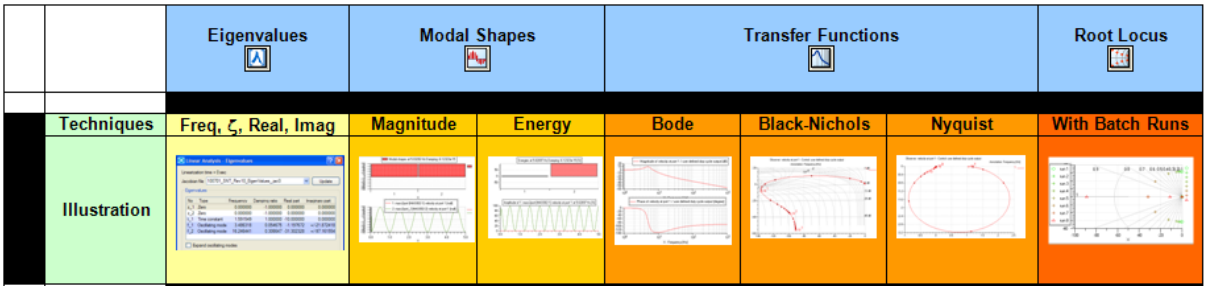
Confidential C

L'onglet *State contributions* est particulièrement instructif car il permet de déterminer quelles sont les variables d'état les plus pénalisantes pour la rapidité de la simulation :



Utile : un double-clic sur le submodel dans ce tableau permet de le mettre en évidence sur le sketch.

9.2 ANALYSE LINÉAIRE



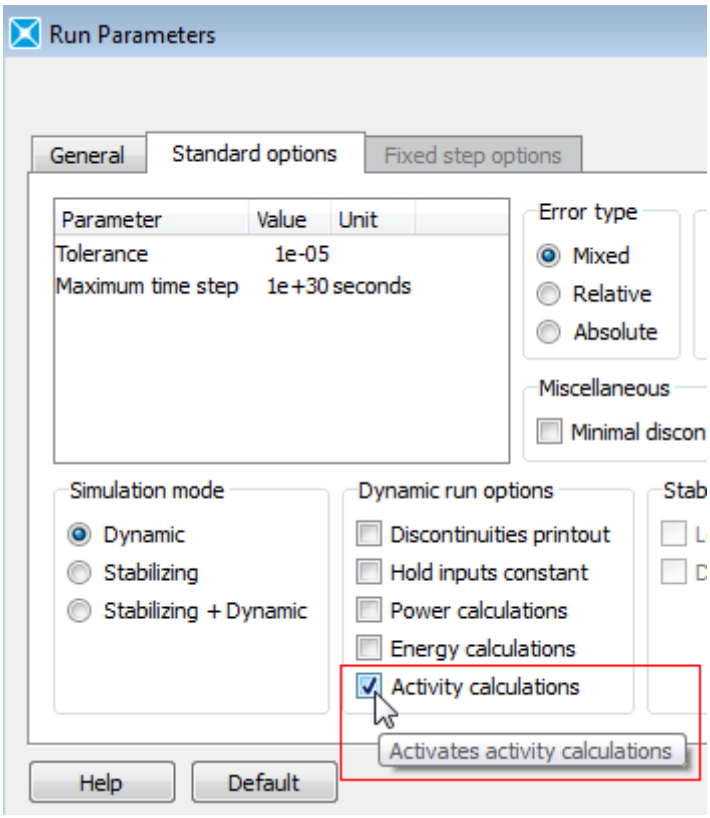
Les techniques d'analyse linéaire sont inutiles si la linéarisation n'est pas faite autour d'un point d'équilibre. Il convient donc d'effectuer un *Stabilizing Run* ou un *Dynamic Run* suffisamment long avant d'y recourir.

On peut cependant se contenter d'une analyse modale même sur un système qui n'est pas en équilibre, afin d'avoir une première idée des fréquences propres du système. Si on obtient des fréquences propres très élevées, a fortiori avec des amortissements faibles, on est à peu près certain que la simulation sera lente. Des solutions doivent être trouvées pour modifier le modèle de manière à éviter ces fréquences élevées.

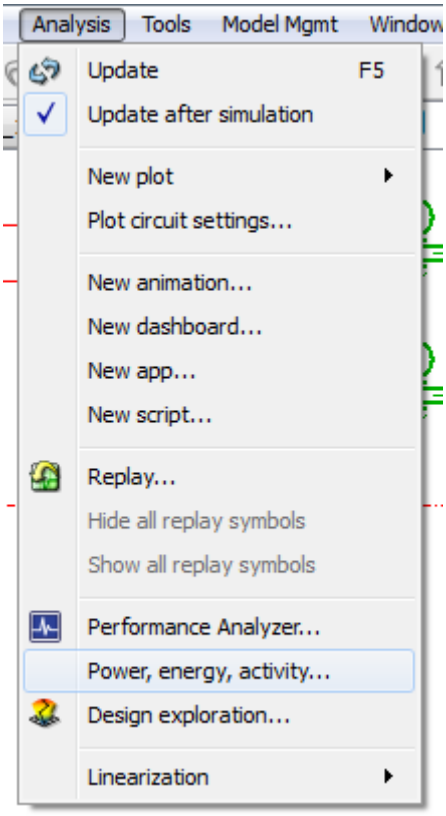
Confidential C

9.3 INDICE D'ACTIVITE (ACTIVITY INDEX)

Quantifie l'importance des paramètres du modèle selon un critère énergétique.

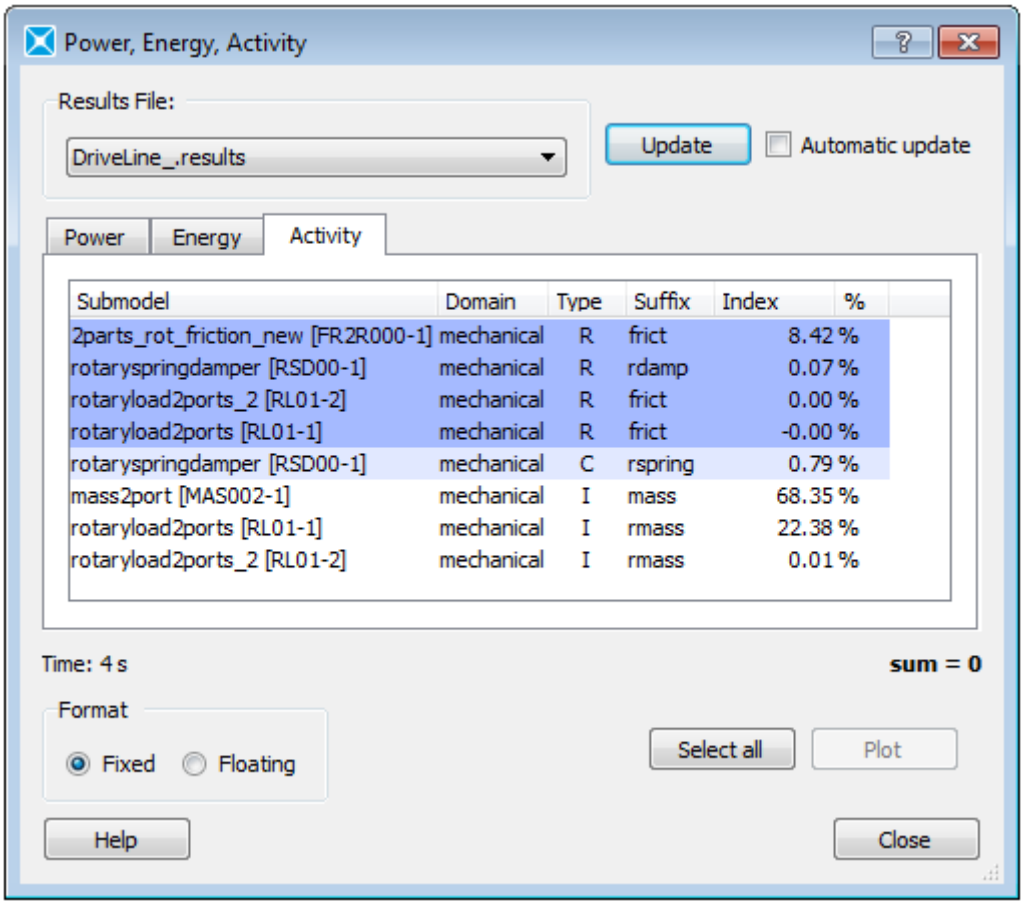


Une fois le run terminé, on peut exploiter ces résultats via le menu Analysis / Power, energy, activity



Confidential C

On y voit ce genre de résultats :



L'activité d'un élément est définie comme l'intégrale temporelle de la valeur absolue de la puissance :

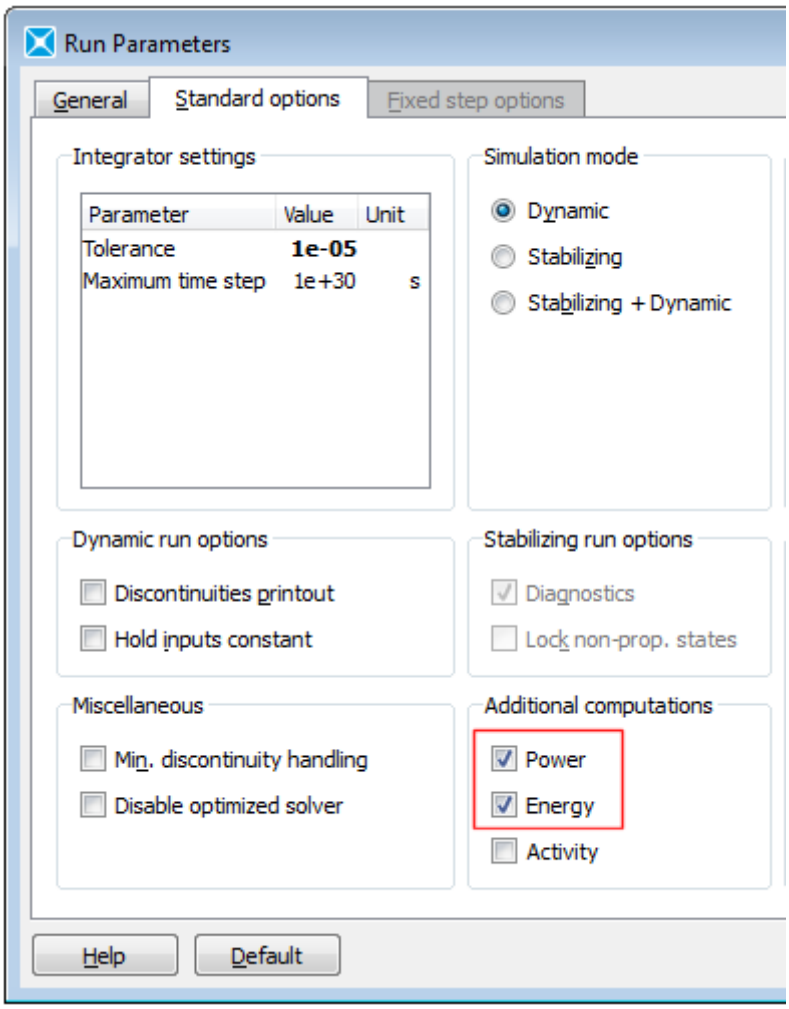
$$A_i(\tau) = \int_0^{\tau} |P(t)| \cdot dt$$

L'indice d'activité est défini comme étant le rapport de l'activité d'un élément sur l'activité totale du système :

$$AI_i = \frac{A_i}{\sum_{system} A_i}$$

Confidential C

9.4 ANALYSE ENERGETIQUE



NB : le fait de cocher une option peut modifier le nombre de variables d'état du système, et amener à une recompilation du modèle en mode Paramètre.

Le bilan d'équilibre des puissances (cf §23.1.3) s'écrit :

$$\sum_{\text{port } i} P_{IN\,i} + \sum_{\text{source elements}} P_S = \sum_{\text{capacitive elements}} P_C + \sum_{\text{inertial elements}} P_I + \sum_{\text{resistive elements}} P_R$$

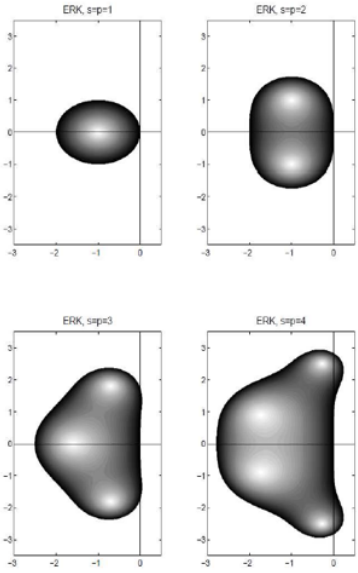
avec

- $P_{IN\,i}$ = power exchanged at port i (> 0 if it goes in the system and < 0 if it leaves it);
- P_S = source power variable inside the system;
- P_C = capacitive power of the system;
- P_I = inertial power of the system;
- P_R = resistive power of the system which is not recovered by the system (always positive).

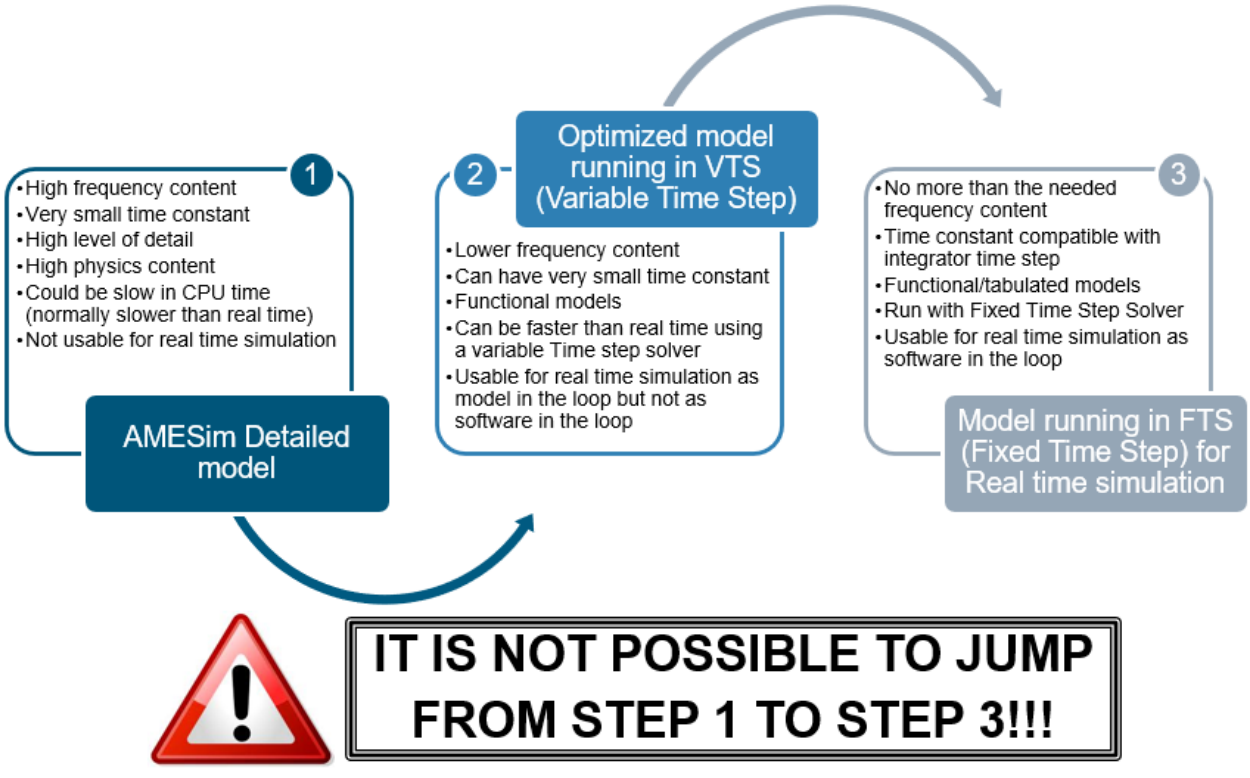
Confidential C

10 UTILISATION D'UN MODELE SUR CIBLE TEMPS-REEL

- For a **R**eal **T**ime (**RT**) simulation, the solver MUST be **EXPLICIT** and **FTS** and could be an Euler (Runge-Kutta 1st order) or a Runge-Kutta method of higher order.
- **Model dynamics should be in the stability range** of the solver. This stability zone is expressed in a complex plane:
  - the horizontal axis is the real axis
  - the vertical one is the imaginary axis.
- For **second order systems with natural modes not or little damped** we are close to the vertical axis and therefore the model could become **numerically unstable** even if stable from the natural modes point of view.
- The **user needs to pay attention** therefore at **natural modes that are little damped.**
- Then the **user has to pay attention to first order modes** that, **even if fully damped**, could be higher in frequency than the minimal frequency supported by the FTS integration method.



STABILITY REGION OF RUNGE-KUTTA SOLVER

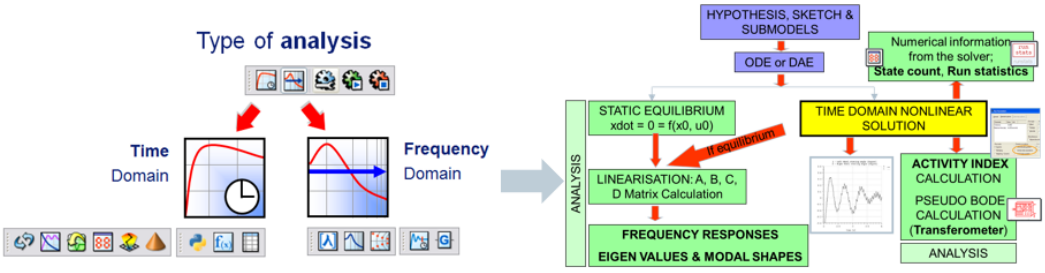


Tricks and Tips to reduce a model to RT

1. Avoid useless states variables
2. Avoid useless discontinuities
3. Maximum step size
4. Set correctly tolerance and Bad conditioning
5. Save only useful variables
6. Avoid generated implicit
7. LMS Amesim platform tools for model analysis and simplification

Optimized model running in VTS

Model running in FTS for RT simulation

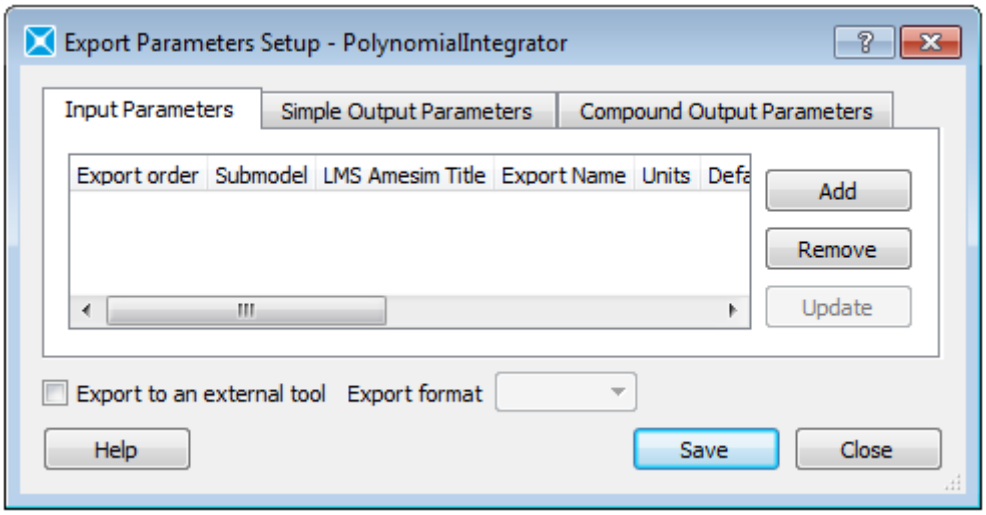


11 DOE - OPTIMISATION

11.1 EXPORTER LE MODELE

En mode *Parameter* : Settings / Export Setup

On peut ensuite faire glisser les paramètres sur la fenêtre qui s'ouvre :



On peut utiliser des expressions, en utilisant la syntaxe suivante :

Submodel	LMS Amesim Title	Export Name	Units	Default value
		a		1
		b		1
		c		1
fofx [FX00-1]	expression in terms of the input x	polynomial		${a}*x^2+${b}*x+${c}

Remarques :

- x est le temps
- ${a} est la valeur du paramètre a
- Le paramètre polynomial doit dans ce cas avoir le type *Formatted String*

On peut ensuite sauver (sans fermer), passer en mode *Simulation*, puis sélectionner de même les paramètres de sortie.

Après sauvegarde, on obtient trois fichiers avec les extensions :

- .in.tpl
- .out.tpl
- .xpt

Confidential C

11.2 LANCEMENT EN BATCH (LIGNE DE COMMANDE)

Il faut tout d'abord instancier avec les valeurs souhaitées le fichier template .in.tpl en ne conservant que l'extension .in. On peut alors utiliser en ligne de commande l'exécutable AMEPilot (s'assurer que son dossier – actuellement T:\logiciel\amesim\V_1210\soft\win32 - figure bien dans le PATH), en passant en argument le nom du modèle sans extension :

```
AMEPilot chemin/monModele
```

Remarques :

- le dossier spécifié doit contenir le modèle (.ame), le fichier.xpt créé précédemment, ainsi que le .in
- Le format des données numériques dans le fichier .in est libre pour les données numériques (ex : 12000 ou 1.2^{e}4, ou 12000.0)

11.3 EXPLOITATION DES RESULTATS

Le fichier .out.tpl n'est utilisé qu'avec certains add-ons, comme par exemple l'interface Excel.

LMS propose des interfaces avec Optimus et iSight.

11.4 DESIGN EXPLORATION

Confidential C

12 SOUS-MODELES (AMESET)

Voir le User Guide **AMESet (**abréviation de Advanced Modeling Environment - Submodel editing tool).

C'est une fonctionnalité qui permet d'intégrer de nouveaux sous-modèles (et de nouveaux icônes), en automatisant la création d'un template dans lequel on pourra intégrer son propre code (en C ou en F77).

12.1 VARIABLES ET PARAMETRES

Au cours d'une simulation certaines grandeurs changent, ce sont des **variables**. D'autres sont fixes, ce sont des **paramètres**.

Les variables peuvent être **externes** (elles sont échangées entre les composants par des ports) ou **internes**.

Les variables externes peuvent à leur tour se décomposer en **entrées** et **sorties**. La fonction première d'un sous-modèle est de calculer les sorties en fonctions des entrées.

AMESim tente de trier les sous-modèles de manière à ce que les entrées de chaque sous-modèle soient connues au moment où il est appelé. Dans le cas contraire, on parle de **boucle implicite** ou algébrique.

Le solveur AMESim résoud ce genre de problèmes en introduisant des variables implicites qui permettent de casser les boucles algébriques.

12.2 LES DIFFERENTS TYPES DE VARIABLES

Pour résumer (certains types sont détaillés par la suite), on trouve les types suivants :

- basic variables
- explicit state variables
- implicit state variables
- constraint variables
- duplicate variables (simple or reversed sign)
- fixed variables
- one-line macros
- multi-line macros

Les variables peuvent en outre être

- externes ou internes
- scalaires ou tableaux

Si elles sont internes il peut s'agir de variables d'activité. Si elles sont externes, ce sont soit :

- des inputs
- des outputs
- des inputs avec valeur par défaut
- des inputs non utilisés

Confidential C

12.2.1 VARIABLES D'ETAT

Les variables d'état ne sont pas calculées directement par le sous-modèle mais par l'intégrateur. Pour cela une valeur initiale doit être fournie.

La fonction du sous-modèle est donc de calculer une valeur de la dérivée de la variable d'état. Ce calcul peut être effectué de deux façons :

- une méthode directe, la plus commune. On parle alors de variable d'état explicite
- une méthode indirecte, assez rare, dans laquelle l'intégrateur essaie de minimiser une quantité appelée résidu, en jouant à la fois sur la variable d'état et sa dérivée. On parle alors de variable d'état implicite.

Toutes les variables d'état ont un « titre » et une unité.

Par exemple le plupart des blocs « masse » s'appuient sur une équation différentielle du type

```
dv/dt = force / masse
```

Même si on dit que la vitesse est une sortie du sous-modèle, en fait ce dernier calcule la dérivée de la vitesse, et c'est l'intégrateur qui se charge du calcul de la vitesse.

Pour illustrer la différence entre les approches implicite et explicite dans ce cas précis, la doc fournit un exemple simple mais limpide : on peut aussi bien imaginer calculer directement

```
v1dot = (f2-f1)/m
```

que tenter de minimiser le résidu

```
m*v1dot - (f2-f1)
```

Le conseil est de n'utiliser des états implicites que lorsqu'il y a une très bonne raison de le faire. Ici la 1e solution est meilleure, mais la 2nde a pour avantage de tolérer M=0 …

12.2.2 VARIABLES D'ACTIVITE

Elles sont utilisées dans les sous-modèles faisant intevenir des transferts d'énergie. L'activité est définie comme l'intégrale de la valeur absolue de la puissance :

$$A(\tau) = \int_0^{\tau} |P(t)| \cdot dt$$

Elle s'exprime en joules mais il s'agit d'une grandeur différente de l'énergie, du fait de la valeur absolue.

Confidential C

12.3 BLOC DYNAMIQUE

Le terme « dynamique » n'est pas utilisé ici en opposition à « algébrique » comme dans Simulink.

Un bloc dynamique (on pourrait aussi dire « évolutif ») est un composant dont certaines caractéristiques peuvent varier, par exemple :

- le nombre de ports
- le nombre de variables internes ou externes
- les dimensions de ces variables
- etc …

La modification d'une de ces caractéristiques implique une recompilation du modèle. Leur nombre ne peut donc pas varier en cours de simulation !

12.4 TEMPLATE DU CODE GENERE PAR AMESET

Il y a plusieurs sections créées par AMESet dans lesquelles on peut être amené à écrire du code :

- Private code section for own special purposes.
- Initialization function declaration statements.
- Initialization function checking statements.
- Initialization function executable statements.
- Calculation function declaration statements.
- Calculation function executable statements.

12.5 NOMMAGE DES VARIABLES

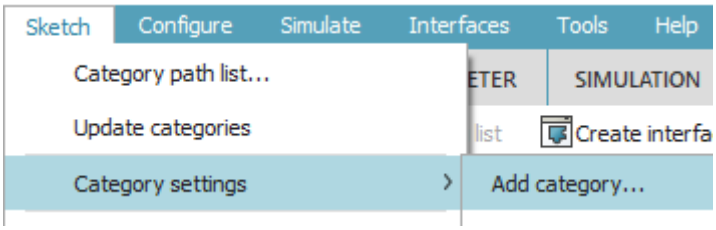
Attention : les noms de variables doivent commencer par une lettre, et les **tirets et underscores sont interdits** !

Confidential C

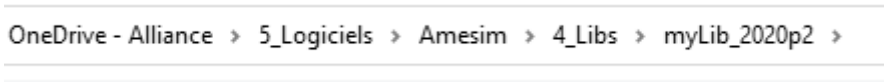
13 SUPERCOMPOSANTS / SUPERCOMPONENTS

13.1 CREATING CATEGORIES

Before creating supercomponents, it is better having an idea of where you will store them.

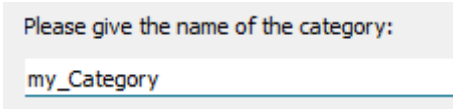


Add category, then select a directory. In my case :

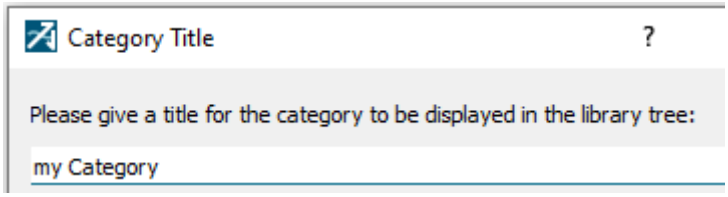


(be sure to select the folder itself)

Enter a name to identify it (no spaces allowed)



then a title which will appear in the Library browser (spaces allowed)



The Icon designer pops up, click save, it will still be possible to modify it afterwards.

The newly created category then appears in the Library, with an empty icon :

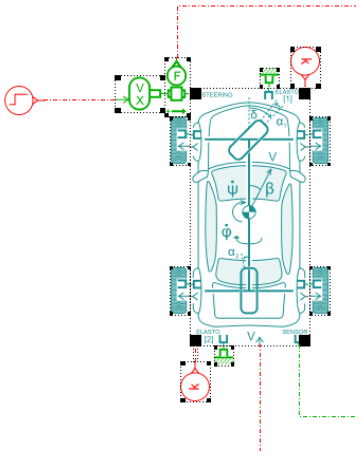


Confidential C

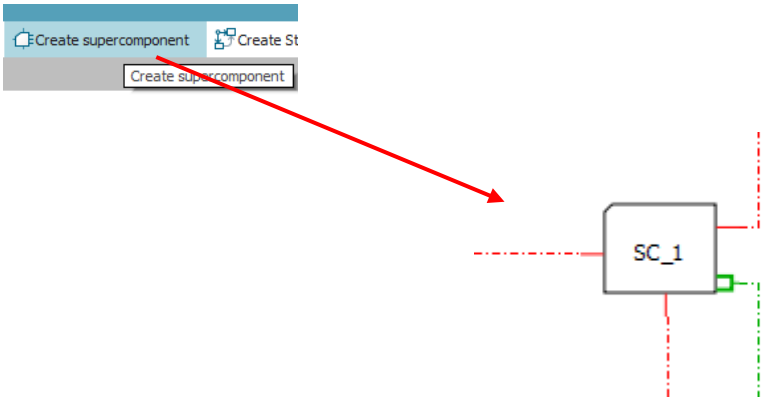
13.2 CREATING SUPERCOMPONENTS

13.2.1 MANDATORY STEPS

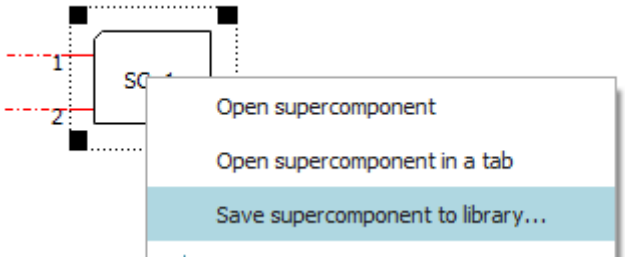
Select the elements (in sketch mode)

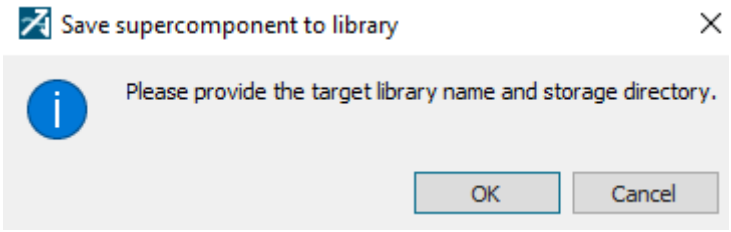


Create supercomponent

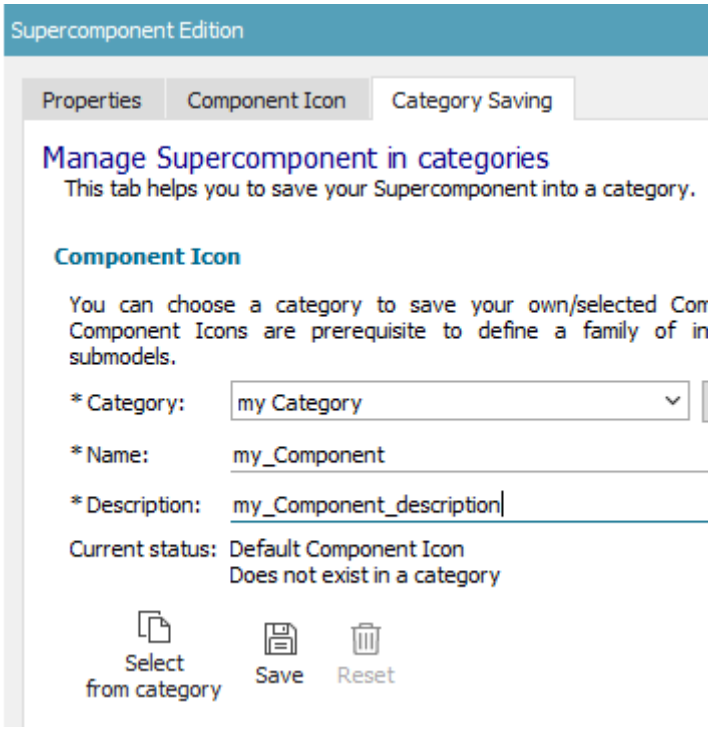


Right-click / *Save …*



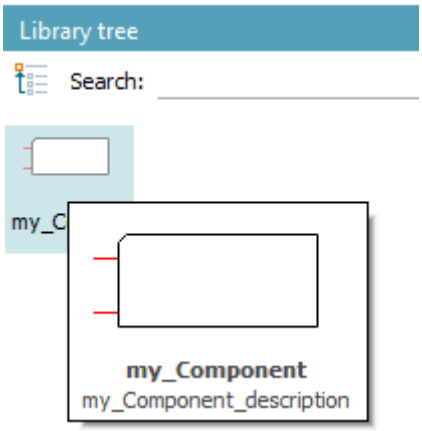


Select the category you created in 13.1, enter a name and a description for the component



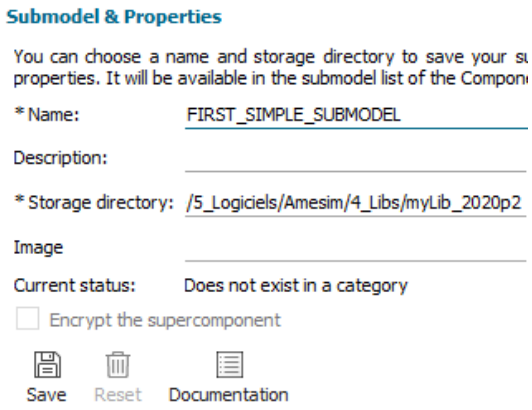
No spaces allowed here again, which is questionable for a description … (but this description seems rather useless, anyway)

Then *Save*, and you'll see the component and its description appear in the corresponding category of the Library



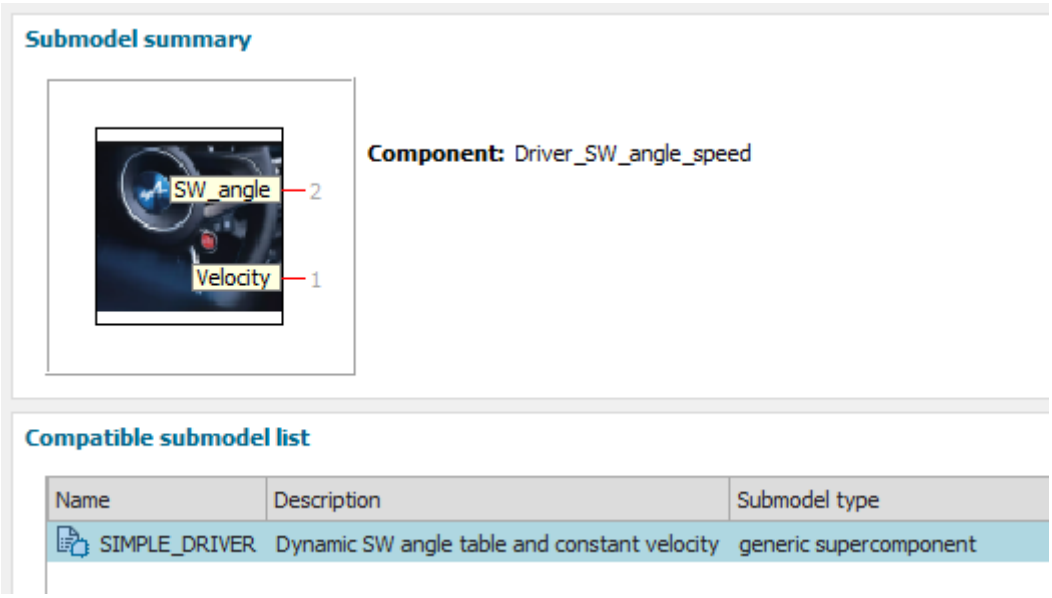
You can now enter the name of this submodel (reminder : a submodel is one of the available modeling choices for a component)

Enter a name (no spaces …), and *Save*



Note that for submodels the description is not mandatory, but I find it much more useful because it's the best way to provide the future user a short explanation of the submodel content.

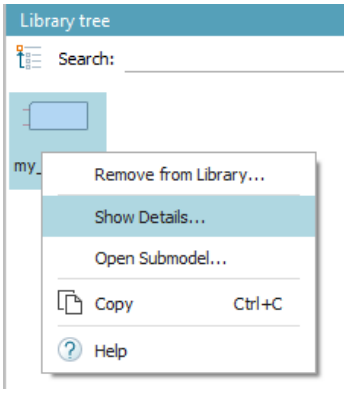
Example :



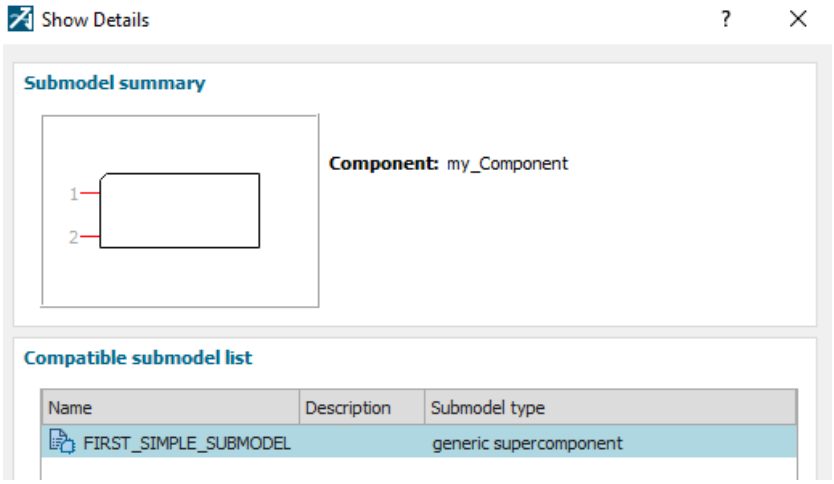
Confidential C

13.2.2 RESULTING .SPE SUBMODEL

Right-click on the component and …

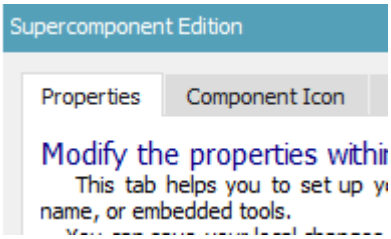


to see the available submodels :



Open Submodel will allow to open the .spe file in “standalone” mode

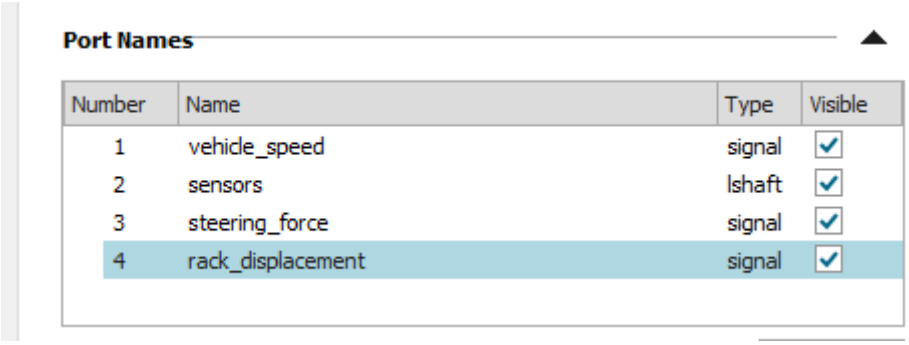
13.2.3 OPTIONAL STEPS



Confidential C

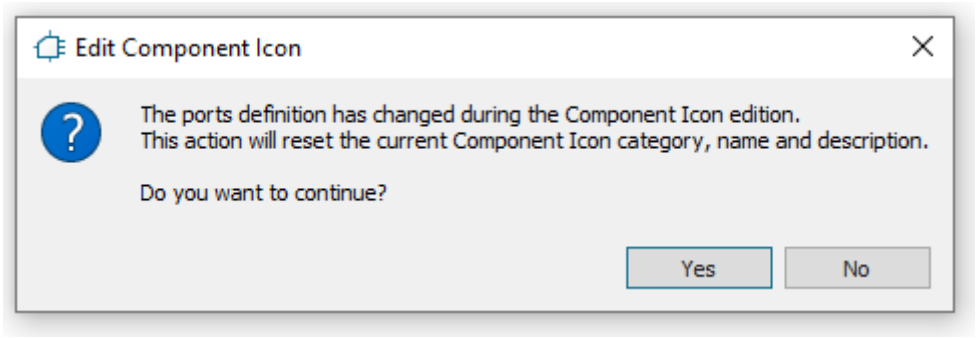
In the Properties tab, you can select an image to illustrate the component

In *Component Icon*, you can reopen the Icon designer if you feel like an artist today, and you can also set (which is probably more useful) the port names that will appear as inputs and outputs of the component :



Example from a SC created from a 3dof chassis from VD library

Note that if you modify the component icon even only from a design point of view, you will have to redefine most of the elements !



13.3 MODEL UPDATE AFTER CREATION

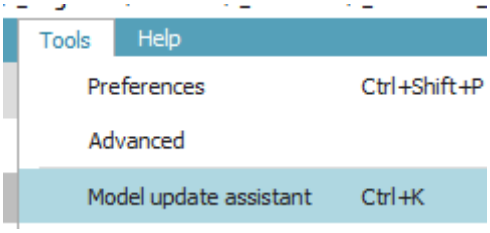
Sometimes (most of the time ?), the link between the SC and the library appears to be broken



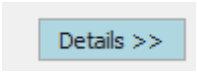
You can either right-click and *Synchronize* … but you need to do it SC by SC



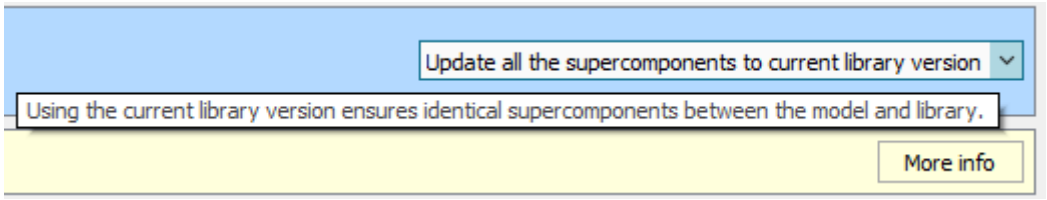
or perform a *Model update*



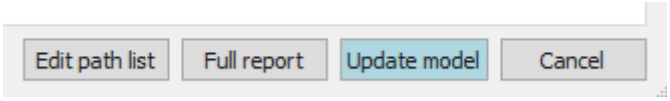
Even if the first message says “up to date”, click on *Details*



and select the following option



Then *Update model*



All SC should appear with a “fixed” (green) link

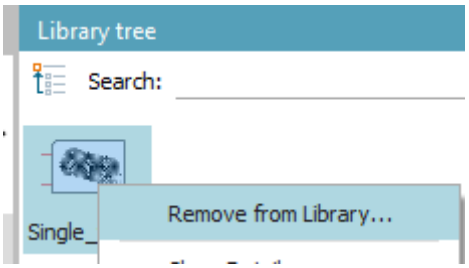


Confidential C

13.4 REMOVING …

13.4.1 REMOVING COMPONENTS

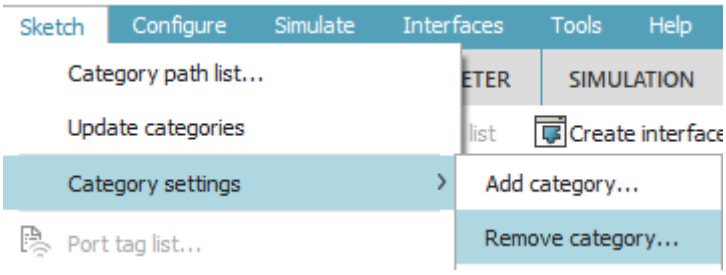
Right-click and …



You will be asked if you want to remove only a specific submodel, or the whole component

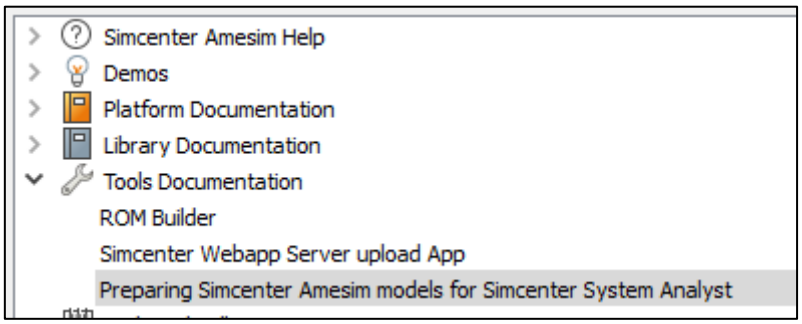
13.4.2 REMOVING CATEGORIES

Right-click on the category doesn't work here. The solution is in the sketch menu :



14 AMESIM FOR SYSTEM ANALYST

See Amesim doc first



Supercomponents must be saved in Amesim libraries. Save these libraries within the "libraries" folder of the Simulation Environment and into a subfolder dedicated to the Simcenter Amesim native tool. The administrator must set this path in the cz.xml using the `Tool_Directory` tag.

```
<ASSETS     value="./libraries">          <!-- simulation tool components -->
    <!-- Amesim tool components
    The libraries are identified recusively from the subfolder Amesim. The librar
    excluded. -->
    <Tool toolId="Amesim">
        <Tool_Directory value="./Amesim" recursive="True" exclude="*/private*"/>
    </Tool>
</ASSETS>

<MODELS     value="./models">             <!-- simulation tool models -->
    <!-- Amesim tool models
    The models are identified recusively from the subfolder Amesim. The libraries
    excluded. -->
    <Tool toolId="Amesim">
        <Tool_Directory value="./Amesim" recursive="True" exclude="*/private*"/>
    </Tool>

</MODELS>
```

Caution :

- **DO NOT HARCODE PATHS** in the submodels/supercomponents.

Confidential C

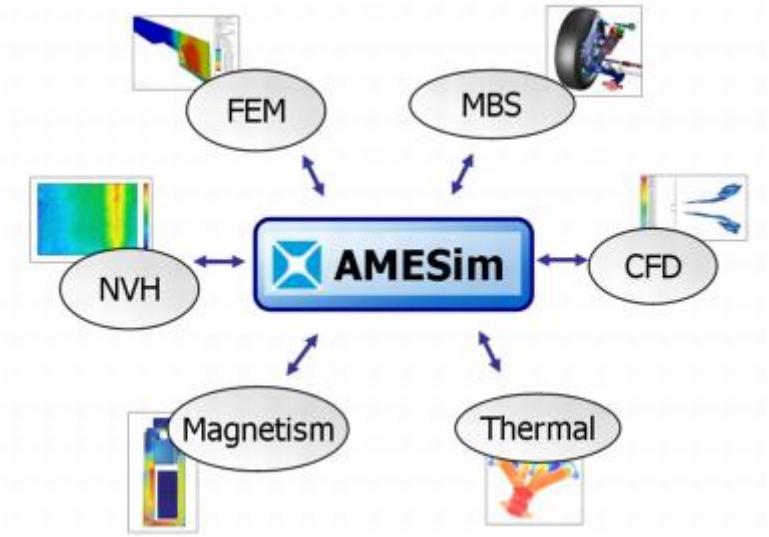
15 CO-SIMULATION

15.1 INTRODUCTION

La co-simulation permet de faire communiquer deux modèles (ou plus), qui sont simulés dans leur propre environnement, et communiquent à pas de temps fixe.

NB : ça ne signifie pas qu'il est obligatoire d'utiliser des solveurs à pas de temps fixe !

Il devient ainsi possible de coupler des « univers » différents :



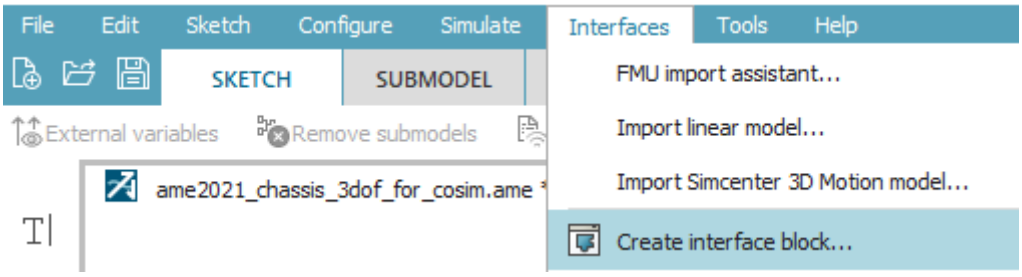
Des interfaces existent notamment avec :

- Virtual.Lab
- MSC Adams
- SimPack
- RecurDyn
- Star-CCM+

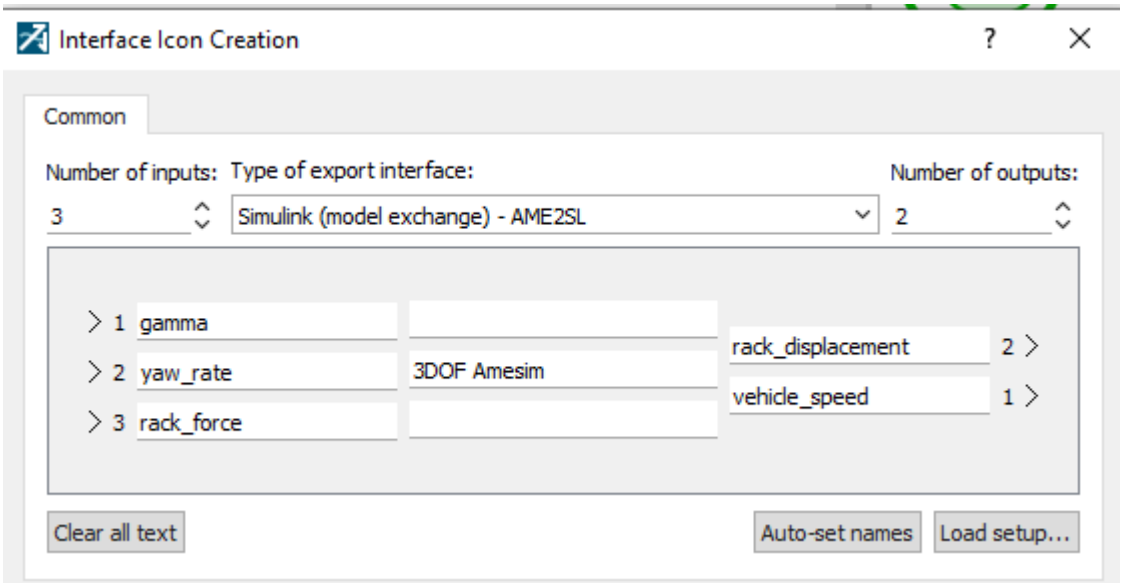
Software type	Examples
Proprietary / in-house codes	C code, Fortran code, Python, ...
Network simulation	Simulink, ...
System level simulation	**LMS Amesim** (**LMS Amesim-LMS Amesim** Co-simulation), other 1D system software, ...
Multi-Body Systems (MBS)	LMS Virtual.lab Motion, MSC Adams, Simpack Recurdyne, ...
Finite Element Methods (FEM)	LMS Virtual.Lab, ...
Computational Fluid Dynamics (CFD)	Fluent, StarCD, CFX, Eole, IFP C3D, EXA, ...

Confidential C

15.2 Y ACCÉDER EN DEUX CLICS



puis right click sur le bloc (en mode sketch) et Edit Interface icon :



Confidential C

15.3 INTERFACE AMESIM-SIMULINK

L'interface entre les deux outils est bi-directionnelle, puisqu'il est possible d'exporter :

- un modèle Simulink vers Amesim
- un modèle Amesim vers Simulink

15.3.1 LES DIFFERENTS MODES DE FONCTIONNEMENT POSSIBLES

15.3.1.1 SYNTHESE

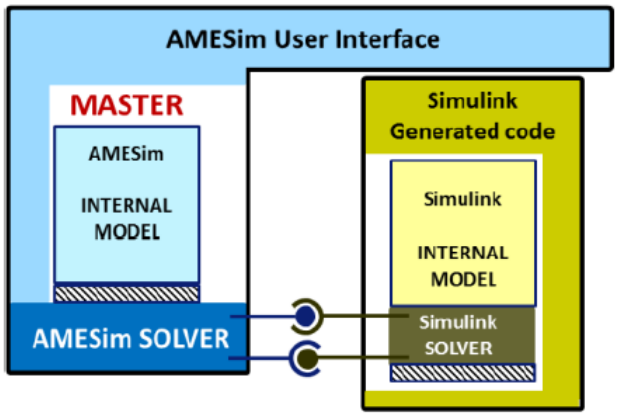
Envisaged interface mode	Supported
1.1 Co-simulation in **Simcenter Amesim**: **Simcenter Amesim** as Master (**Simulink** as Slave)	Yes [1.2.] (**SL2AMECosim**)
1.2 Model Exchange: Import of **Simulink** model into **Simcenter Amesim**	Yes [1.3.] (SL2AME)
2.1 Co-simulation in **Simulink**: **Simcenter Amesim** as Slave (**Simulink** as Master)	Yes [4.] (**SL2AMECosim**)
2.2 Model Exchange: Export of **Simcenter Amesim** model into **Simulink**	Yes [4.5.] (AME2SL)

1. For **SL2AME** and **SL2AMECosim**, please note that once the **Simulink** model has been imported into **Simcenter Amesim**, **Simulink** is not needed anymore for running the simulation (and no **Simulink** license either).
2. In **SL2AMECosim**, the embedded **Simulink** solver is always of type fixed-step. **Simulink** model referencing is supported for **MATLAB** version R2014b and greater.
3. It is advised to use SL2AMECosim (co-simulation) instead of SL2AME. SL2AME is provided for backward compatibility but is considered deprecated and does not support **MATLAB** versions higher than R2009b. For recent **MATLAB** versions, the use of SL2AMECosim is mandatory. Please refer to the **Simulink** Interface compatibility table for more information.
4. For model exchange or co-simulation in **Simulink** (AME2SL or AME2SLCosim), **Simcenter Amesim** models (with or without solver) are exported as S-functions.
5. This mode complies with the Black-Box option.

Important : le mode SL2AME n'est plus compatible avec les versions postérieures à Matlab 2009. Pour faire tourner un modèle Simulink "dans" Amesim, il ne reste plus que l'option de co-simulation SL2AMECosim.

Confidential C

15.3.1.2 CO-SIMULATION DANS AMESIM (SL2AMECOSIM)



The Simulink to Simcenter Amesim interface creates C code from a Simulink model using the Simulink Coder. The C code is generated in such a way that it can be used in Simcenter Amesim as a normal submodel.

Rules :

- One important rule (that may be eliminated in later versions) is that the input/output variables must be scalar and of type double.
- Only fixed step solvers can be used (the "auto mode" is not allowed: the solver itself and its step size need to be defined before running the SL2AMECosim script).

Lancer Amesim avec Matlab, en utilisant un .bat semblable à ceci

```
@echo off
setlocal
set AME=C:\Program Files\Simcenter\2021.2\Amesim
set MATLAB=C:\Program Files\MATLAB\R2018b
set
path=%AME%;%AME%\win32;%AME%\win64;%AME%\sys\mingw32\bin;%AME%\sys\mpich\mpd
\bin;%AME%\sys\python\win32
"C:\Program Files\Simcenter\2021.2\Amesim\win64\AMESim.exe"
endlocal
```

Démarrer Matlab depuis Amesim (menu Tools)

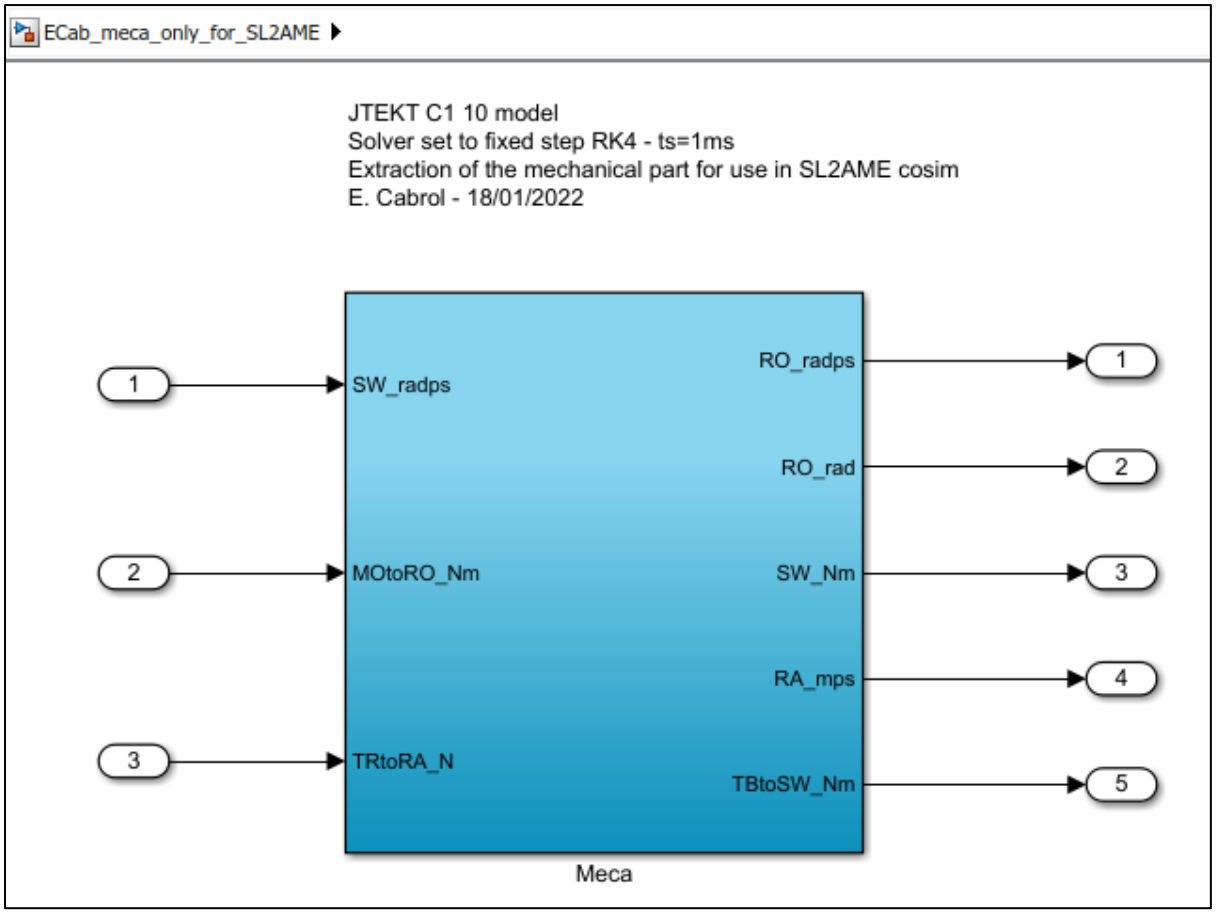
Ca permet de disposer de la librairie Amesim dans Matlab

```
Update Matlab path...
...adding  C:\Program Files\Simcenter\2021.2\Amesim\lib
...adding  C:\Program Files\Simcenter\2021.2\Amesim\interfaces\sl2ame
...adding  C:\Program Files\Simcenter\2021.2\Amesim\scripting\matlab\amesim
...adding  C:\Program Files\Simcenter\2021.2\Amesim\scripting\matlab\amesim\AMESimLib
...adding  C:\Program Files\Simcenter\2021.2\Amesim\scripting\matlab\amesim\AMESimLib\r2017a

========================================================================
 Simcenter Amesim library is available in Simulink library browser.
========================================================================
```

Préparer le modèle de façon à ce que le composant à générer soit au plus haut niveau avec des ports d'E/S :

Confidential C



exemple

Lancer sl2amecosim avec en arguments :

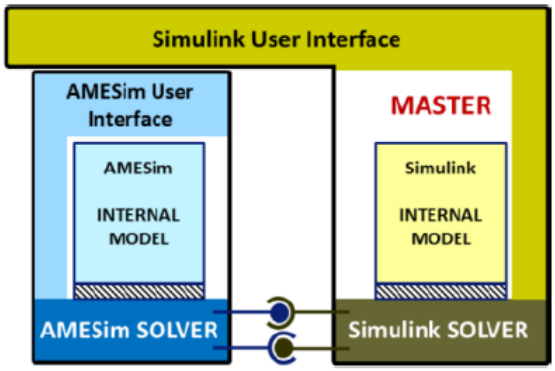
- le nom du .slx (sans extension)
- le dossier de destination où la librairie Amesim sera créée

```
sl2amecosim('ECab_meca_only_for_SL2AME','C:\myPath\libsl2ame','auto')
```

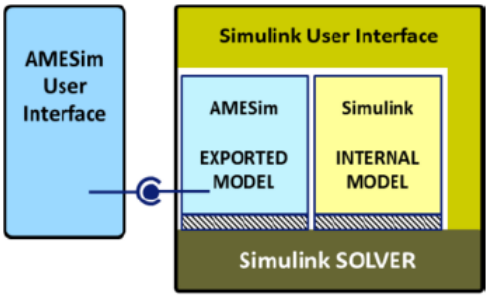
… et ça plante (01/2022)

Confidential C

15.3.1.3 CO-SIMULATION DANS SIMULINK (AME2SLCOSIM)



15.3.1.4 IMPORT DE MODELE AMESIM DANS SIMULINK (AME2SL)



15.3.2 UN EXEMPLE AVEC AME2SL

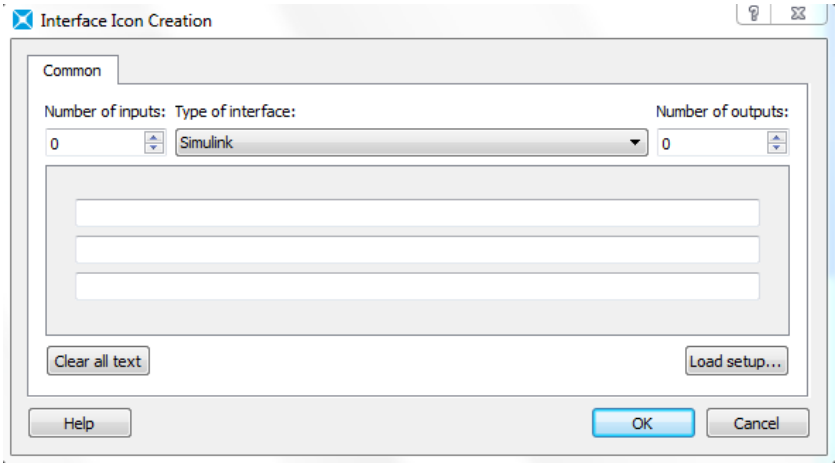
(§ à updater)

Prérequis :

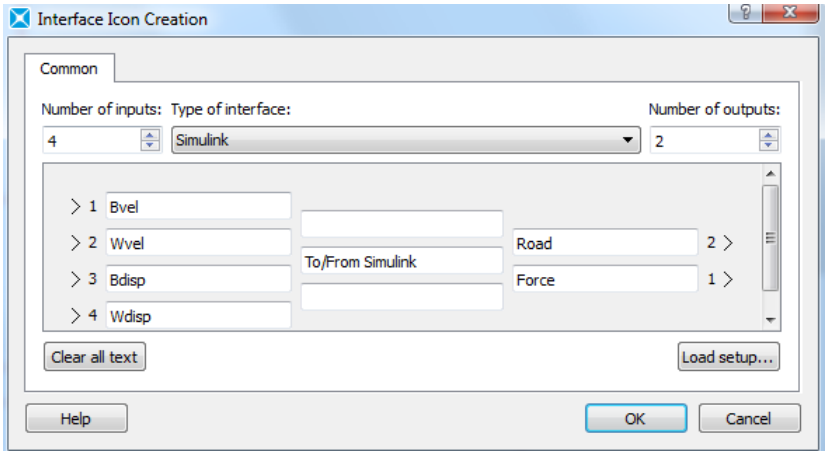
- Compilateur
- Variable d'environnement pointant vers Matlab
- (facultatif) Fichiers de configuration
    - simulink.conf, simulink_cosim.conf (dans $AME/interfaces/simulink)
    - simulink.make (dans $AME/interfaces/simulink/default ou win32)

Suivre le tutoriel décrit dans la doc simulink.pdf (§2)

Le bloc permettant d'interfacer Simulink ne se trouve pas dans les librairies, mais est accessible par le menu Modeling / Interface block / Interface Icon Creation :



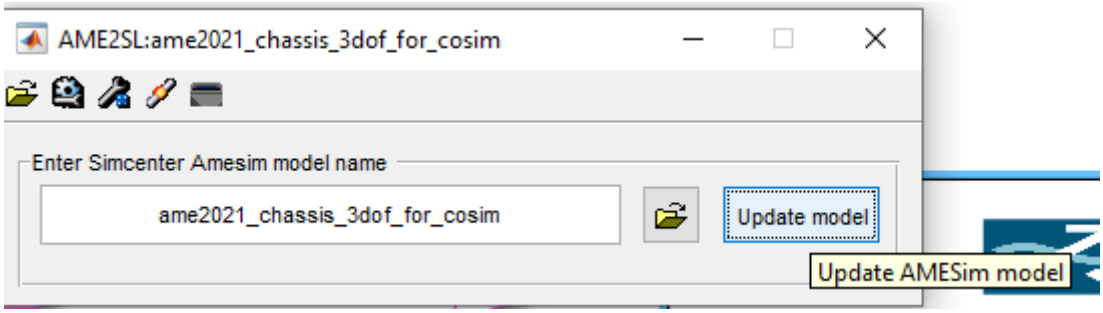
… dans lequel on spécifie entrées et sorties :



Tools / Options / Preferences, sélectionner Visual C++ 64 bits

Que faire si on modifie le modèle Amesim ?

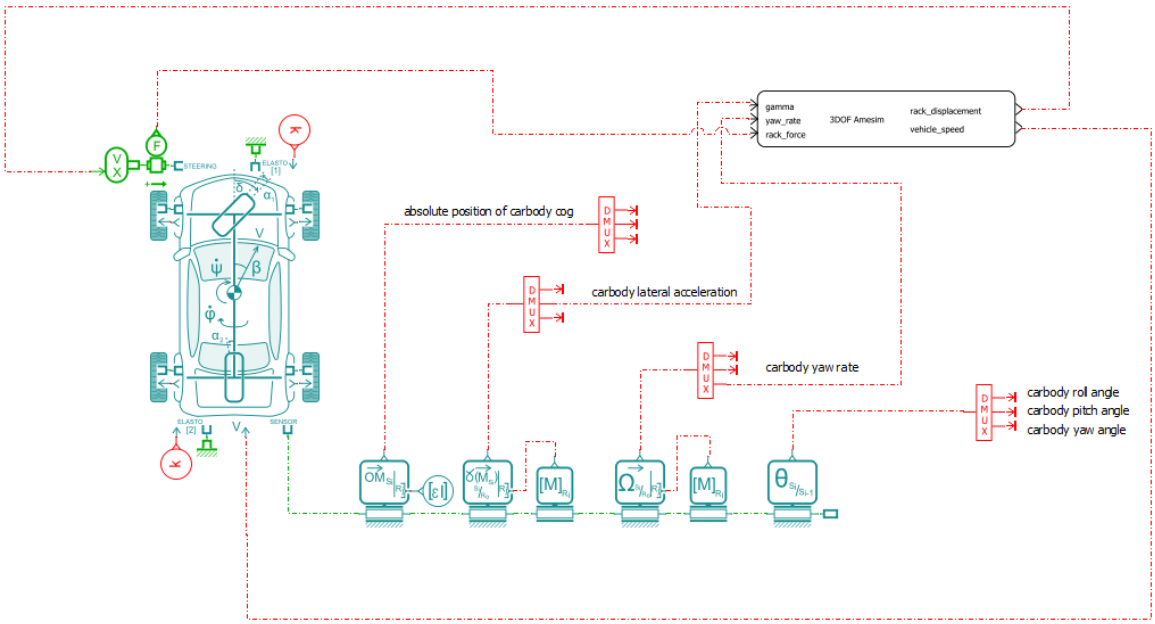
Il faut tout d'abord recompiler le modèle Amesim (Ctrl+T pour forcer la recompilation, puis repasser en mode simu), et une fois que le nouveau fichier _.mexw64 est généré, double-cliquer sur le bloc AME2SL dans Simulink et faire un "Update model" :



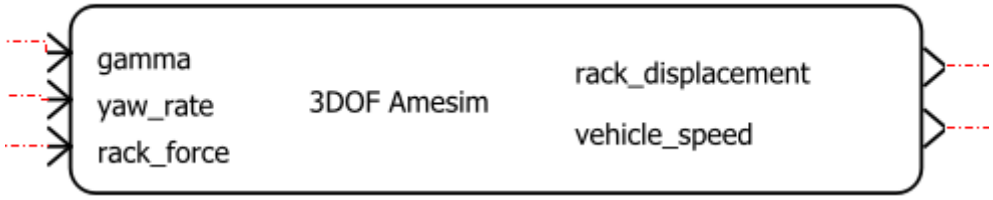
Confidential C

15.3.3 CHASSIS 3DOF + MODELE DE DIRECTION FOURNISSEUR

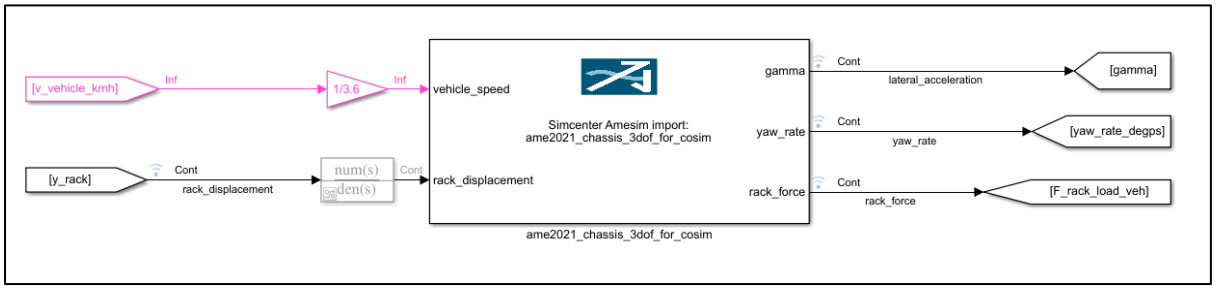
(updated oct. 2021)



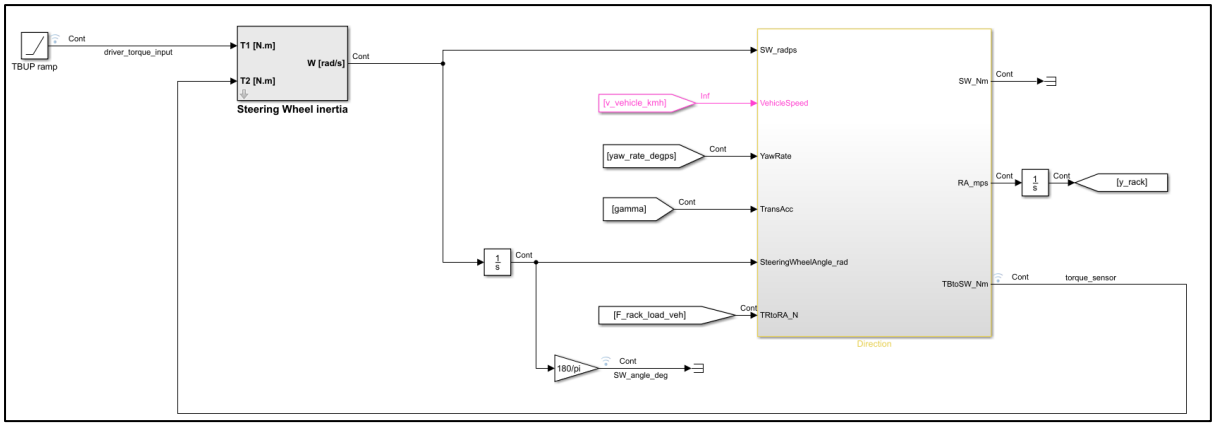
Amesim sketch



Bloc d'interface Amesim

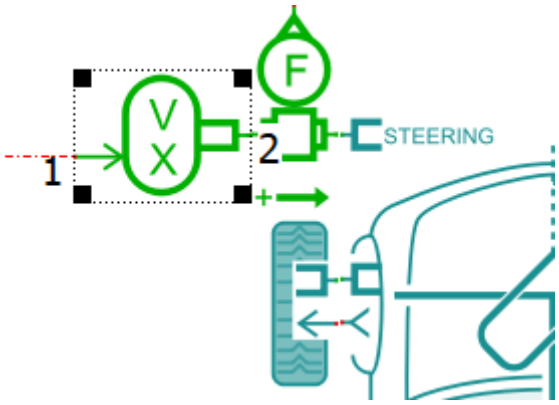


Interface côté Simulink



Reste du modèle Simulink (incl. bloc direction fournisseur)

Attention : côté Amesim, l'entrée déplacement crémaillère se fait via un bloc XVLC01



qui correspond à un submodel s'appuyant sur un retard du 1e ordre :

Name	Description	Submodel type
XVLC01	conversion of signal to a linear displacement and velocity using first order lag	generic submodel
XVLC3	conversion of signal to a linear displacement and velocity using implicit variable	generic submodel

Le paramétrage par défaut de la constante de temps de ce bloc est à 10^{-4} s.

Parameters of linearxvfromxcon [XVLC01-1]

Title	Value	Unit	Tags	Name
velocity initialization	zero			setVelocity
time constant for derivative of displacement	0.0001	s		tau

Confidential C

Il faut adapter la valeur au pas de temps utilisé côté Simulink, au risque de rencontrer des messages d'erreur du type "dérivée non finie" :

```
An error occurred while running the simulation and the simulation was terminated
Caused by:
  • Derivative of state '8' in block
    'ECab with chassis 3dof for cosim with jtekt model 2018b/ame2021 chassis 3dof for cosim/S-
    Function' at time 1.338 is not finite. The simulation will be stopped. There may be a singularity
    in the solution.  If not, try reducing the step size (either by reducing the fixed step size or by
    tightening the error tolerances)
```

Pour un modèle Simulink qui tourne à 0.001s, ça fonctionne en paramétrant également le modèle Amesim avec τ = 0.001

Parameters of linearxvfromxcon [XVLC01-1]

Title	Value	Unit	Tags	Name
velocity initialization	zero			setVelocity
time constant for derivative of displacement	**0.001**	s		tau

Confidential C

15.4 CO-SIMULATION GENERIQUE

Comme son nom l'indique cette possibilté est plus générique que l'interface Amesim-Simulink. Elle est accessible depuis une librairie dédiée :



Si on ne voit pas la library : cf §1.2

La co-simulation générique ne permet de coupler qu'un seul autre solveur à Amesim.

15.4.1 PROTOCOLES POSSIBLES

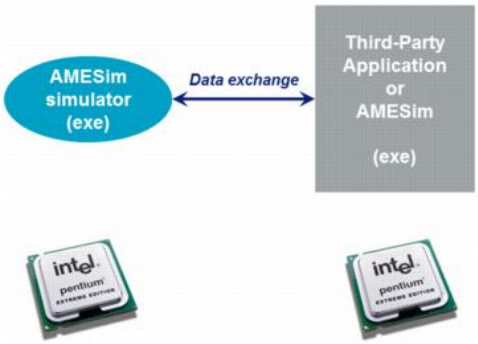
Three different protocols are currently supported by the Generic Co-simulation:

- Network Co-simulation: two computers, each one running LMS Amesim
- Shared memory Co-simulation: two LMS Amesim running on a multi-CPU computer
- Import Co-simulation: two LMS Amesim running on a single-CPU computer

15.4.2 MODES DE FONCTIONNEMENT

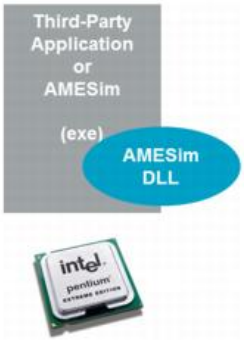
15.4.2.1 CO-SIMULATION STANDARD

There are as many executables as applications involved in the simulation. Each application uses its own solver and is launched separately. Applications simply exchange data during the simulation.



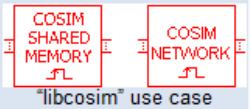
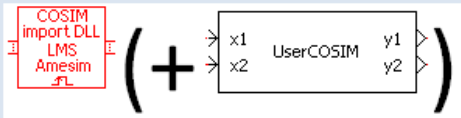
15.4.2.2 CO-SIMULATION HYBRIDE

There is only one executable. One application is chosen as master and pilots the other. Other applications are not used directly: they export their solvers which are used by the master. A typical Hybrid Co-simulation might consist in using, for example, a third-party application to pilot the dynamic library created with **LMS Amesim**



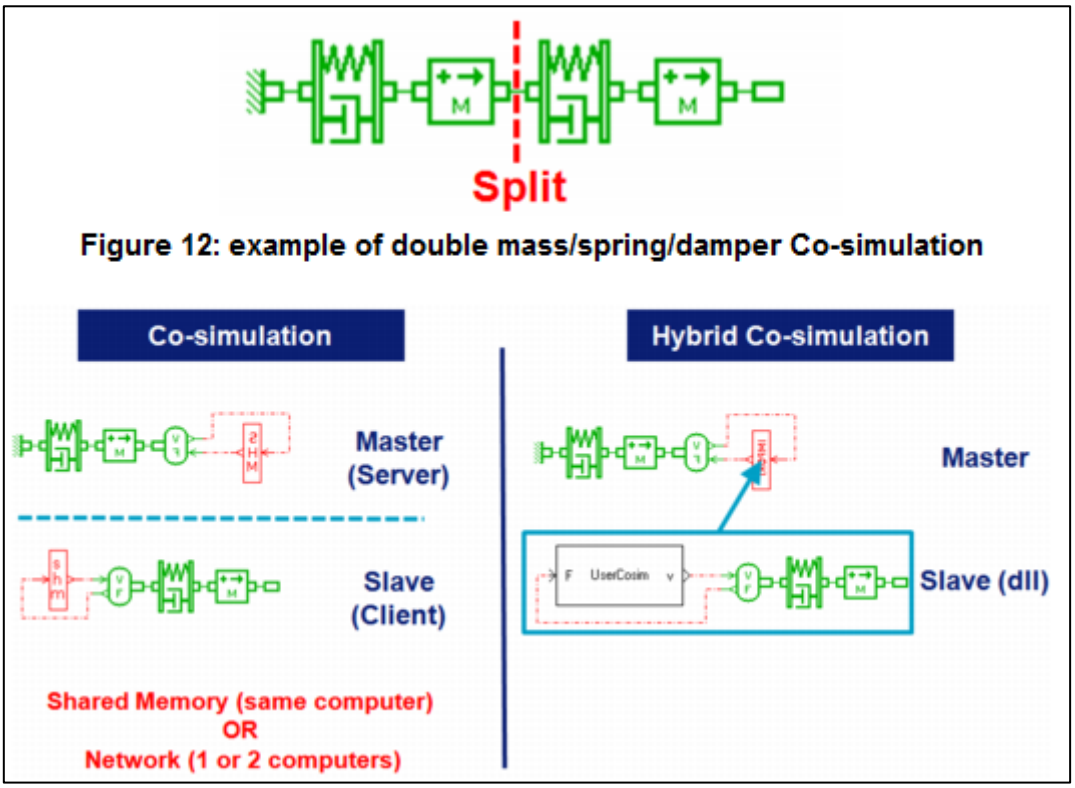
15.4.2.3 LIBCOSIM / USERCOSIM

- Co-simulation is performed at “submodel” level with a library named “**libcosim**”.
- Hybrid Co-simulation can be performed through a dedicated interface named the “**usercosim**” interface.

	 "libcosim" use case	 "usercosim" use case
Coupling type	Co-simulation	Hybrid Co-simulation
Number of executables	Number of applications involved in Co-simulation	1
LMS Amesim solver embedded in model	Yes	Yes
Solver available	Both **LMS Amesim** fixed and variable step	Both **LMS Amesim** fixed and variable step
Co-simulation step	Decided at runtime from master. **LMS Amesim** can be master or slave.	DLL function call **LMS Amesim** is slave.
Master starts slave too	No	Yes
Take advantage of Multi-core, Multi-CPU architectures	Yes	No
Network compatibility	Yes	No

Differences between libcosim and usercosim

Illustration des différences entre les deux approches :

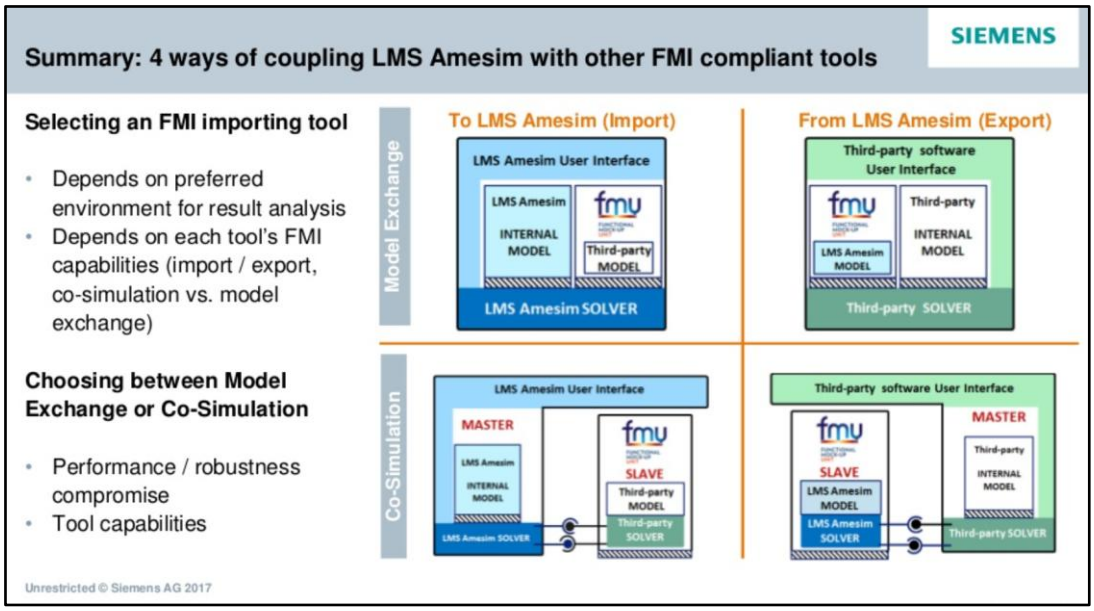


15.5 FMI

15.5.1 DIFFERENTES METHODES DE COUPLAGE

Model Exchange : le solveur n'est pas embarqué

Co-Simulation : le solveur vient avec le modèle

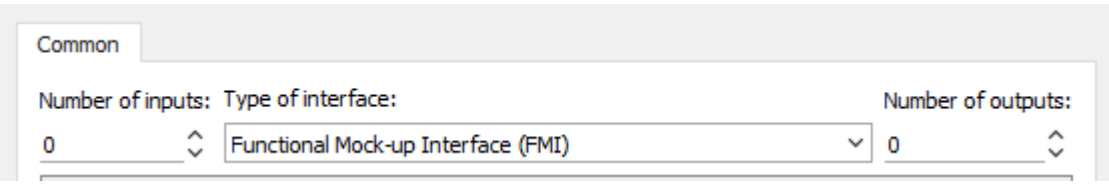


Model Exchange ne peut être utilisé si le modèle contient des variables d'état implicites (cf messages debug au moment de la compilation)

15.5.2 WORKFLOW SIMPLIFIE

- Construire le modèle comme d'habitude, et vérifier son bon fonctionnement
- Revenir dans le sketch et déposer un bloc interface par **Interfaces > Create interface block**

On peut définir un FMU "standalone" en ne déclarant aucun input/output :



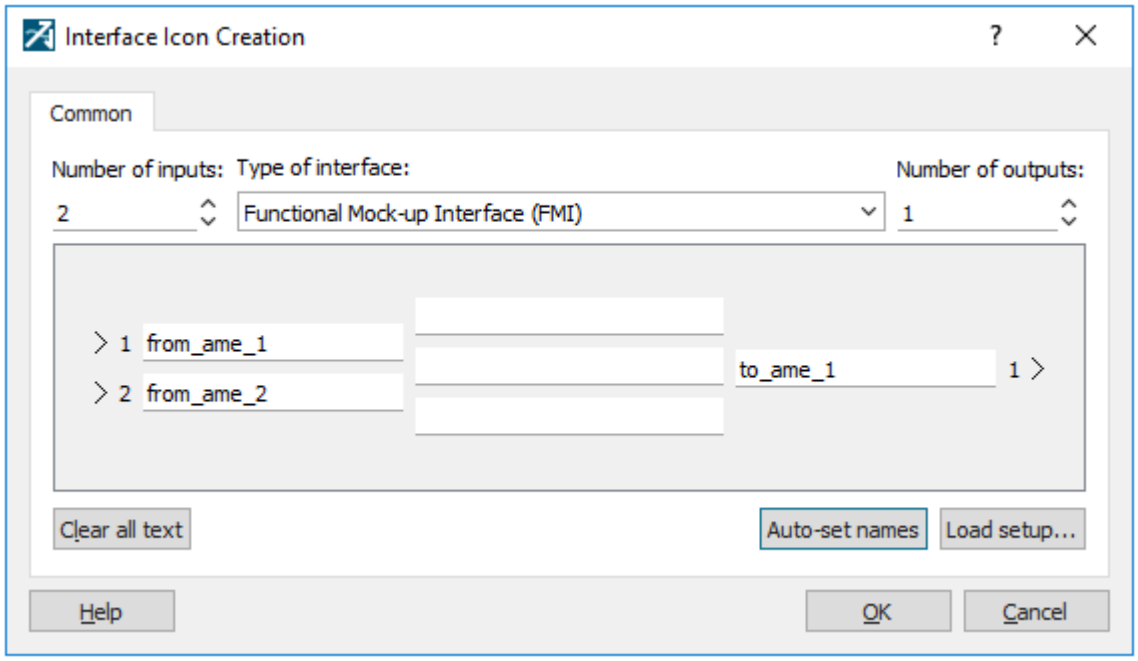
Cela permet par exemple de tester le modèle avec un outil comme le FMU Compliance checker

https://github.com/modelica-tools/FMUComplianceChecker/releases

- Repasser en mode Simu et lancer l'export **Interfaces > FMU export assistant**

Et voilà !

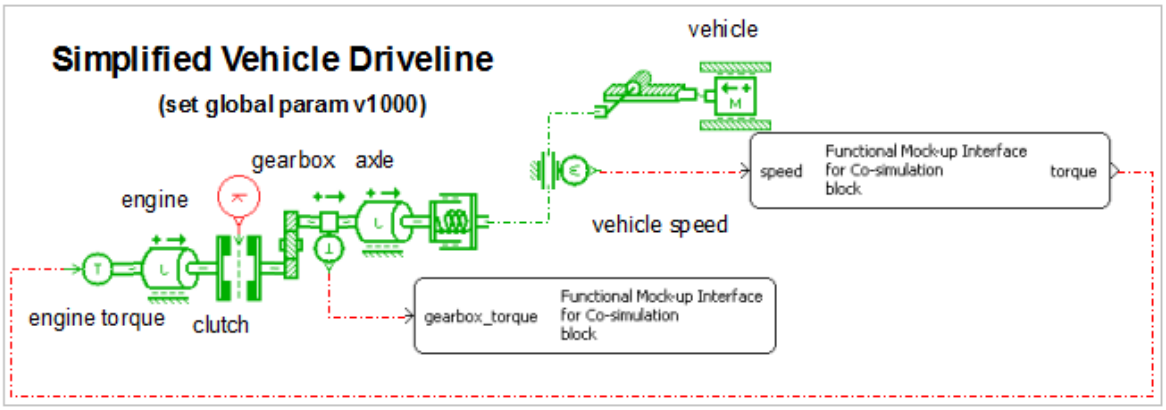
15.5.3 DEFINITION DES E/S



Auto-set names permet d'éviter de remplir les noms des E/S à la main, mais évidemment c'est plutôt déconseillé car peu lisible une fois le FMU généré.

Les trois "champs" centraux servent simplement à renseigner le nom qui apparait dans le sketch Amesim, mais ne sont pas pris en compte à la génération du FMU.

Connecter ensuite les ports du bloc aux signaux dans le sketch :

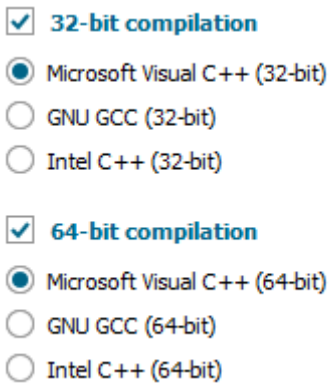


Confidential C

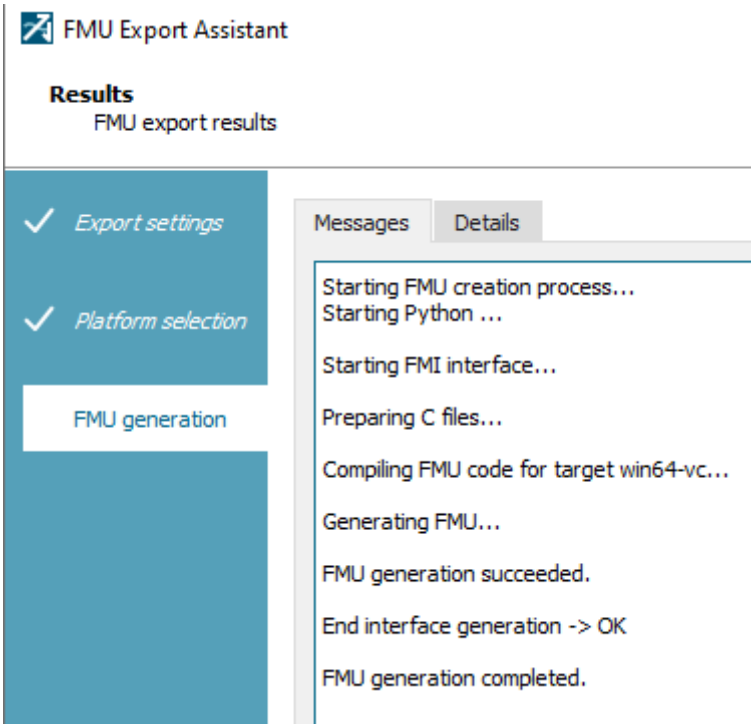
15.5.4 FMU EXPORT

On choisit dans cette phase la version du standard (1.0 ou 2.0) et le mode de fonctionnement du FMU (Model Exchange ou Cosim)

Sur cibles Win ou Linux on choisit également le compilateur. GCC est livré avec Amesim ce qui permet de garantir que l'on peut compiler le modèle, mais ce n'est pas forcément le plus performant. Aucun check n'est effectué quant à la disponibilité effective des autres compilateurs proposés :



Puis



Provide directional derivatives when applicable

Dispo seulement pour FMI 2.0

Confidential C

Aller lire la doc directement, rubrique "Directional Derivatives" :)

Embed tables

Si la checkbox est cochée :

- pour cibles Win/Linux, les données sous forme de tables sont embarquées dans la partie binaire du FMU
- pour cibles real-time … je n'ai pas bien compris comment ça fonctionne

The lower part of the dialog box contains a checkbox titled **Embed tables**. If the checkbox is selected, all the data files used as parameters by submodels are automatically included in the executable part of the FMU as C declarations. As a consequence, for an FMU which is targeting a native platform like **Windows** or **Linux**, the corresponding data are automatically embedded in the binary file of the FMU. For a source-code FMU targeting real-time platforms, the data are part of the source code that is to be compiled by the importing tool of the real-time environment. Thus, these data files do not have to be provided by the importing environment to execute the FMU. This applies to all data table file formats recognized by **Simcenter Amesim**, such as the 1D, 2D, 3D, 4D, 5D, 6D, 7D, 8D, Multi 1D, Multi M1D and XY tables (see ref. [7] for more details about table formats), provided that the data file is referenced by a text parameter of type **input file** of a submodel. If the data file is used directly by a submodel without referencing an input file text parameter, the data file is not processed by the **FMI export tool**, and is therefore not included in the FMU. This means in this case that the data file must be provided at runtime by the importing environment, otherwise the simulation of the FMU may fail because of a missing data file.

Si la machine cible ne dispose pas d'Amesim :

Note

A limited number of submodels may implicitly (i.e. by means of enumeration parameters) use tables - such as fluid or thermal properties - that are provided in the **Simcenter Amesim** distribution. FMUs need to access these tables as well. Therefore, if the target machine does not have **Simcenter Amesim** installed, these submodels need to be set - before the export - to their "user defined" modes in order to point to copies of these tables instead. These copies can then be embedded as described previously. Alternatively, a directory structure containing these tables (e.g. **$AME\libhydr\data**), similar to the "original" **Simcenter Amesim** directory structure, can be reproduced on the target machine, with an **AME** environment variable set accordingly.

15.5.5 PARAMETRAGE SOLVEUR POUR LA COSIMULATION

Tous les paramètres du solveur sont des variables du FMU qui peuvent être tunées par le système "hôte"

Pas fixe ou variable ?

Il est généralement conseillé, si l'on veut utiliser un pas fixe, de vérifier avant l'export que cela n'altère pas la précision du résultat, puis de l'exporter après avoir switché.

Le système hôte peut également procéder de la sorte, mais il n'y aura aucun moyen de vérifier la qualité du résultat.

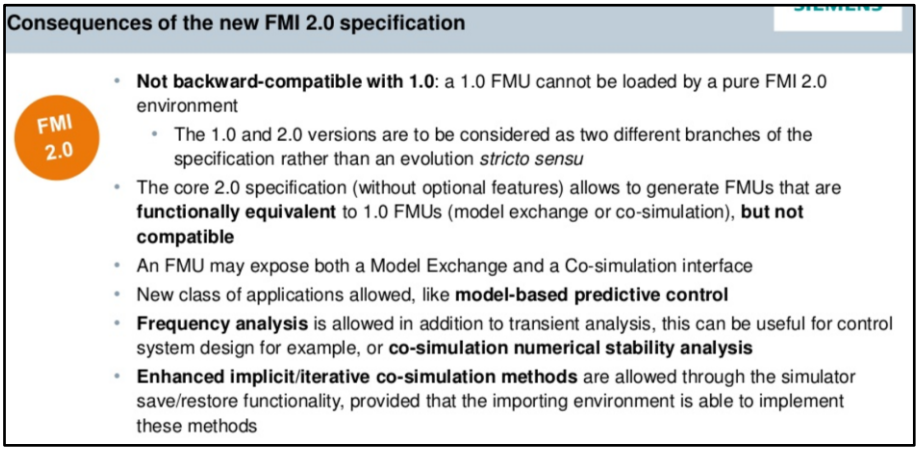
Confidential C

FMI type available for export	Simcenter Amesim numerical solver required for simulating the model		
	Variable step size; Explicit system	Fixed step size; Explicit system	Variable step size; Implicit system
Model Exchange 1.0 or 2.0 (native binary format)	✓	✓	✗
Co-simulation 1.0 or 2.0 (native binary format)	✓	✓	✓
Co-simulation 1.0 or 2.0 (real-time target)	✗	✓	✗

15.5.6 CIBLES TEMPS REEL

N'acceptent que des FMU en co-simulation utilisant un solveur à pas fixe.

15.5.7 FMI 2.0



15.5.8 LICENCE

Mail de K. Crenn (Siemens), 26/11/2020

> Il faut effectivement une licence runtime pour faire tourner un FMU. Cette licence est indépendante des librairies et composants utilisés, tant en terme d'origine des composants, qu'en nombre de composants.
>
> Il est également nécessaire d'avoir une installation minimale d'Amesim sur la machine cible (quelques Mo seulement). Un script de génération de cette installation minimale est disponible dans la documentation Amesim. Si Amesim est déjà installé sur la machine cible, il n'y rien de particulier à faire concernant l'installation.

Confidential C

Since the FMU contains a model that is obtained by connecting together submodels taken from various libraries, the runtime licenses of these libraries are required to simulate the FMU in the import environment. These are the licenses which are usually requested by **Simcenter Amesim** or **Simcenter Amesim Run** when simulating a model.

If the FMU is simulated on a computer on which **Simcenter Amesim**is installed, the license used by **Simcenter Amesim** or **Simcenter Amesim Run** is used. The FMU relies on the value of some environment variables (**AME** and **PATH** under Windows, **AME** and **LD_LIBRARY_PATH** under **Linux**) that are defined during the **Simcenter Amesim** installation process.

If **Simcenter Amesim** is not installed on the computer running the FMU, the simulation may however be performed, since the license provider does not need to stay on the computer that runs the simulation. For example, if a license server is available somewhere on the network the computer is connected to, the FMU can request the runtime license from this license server.

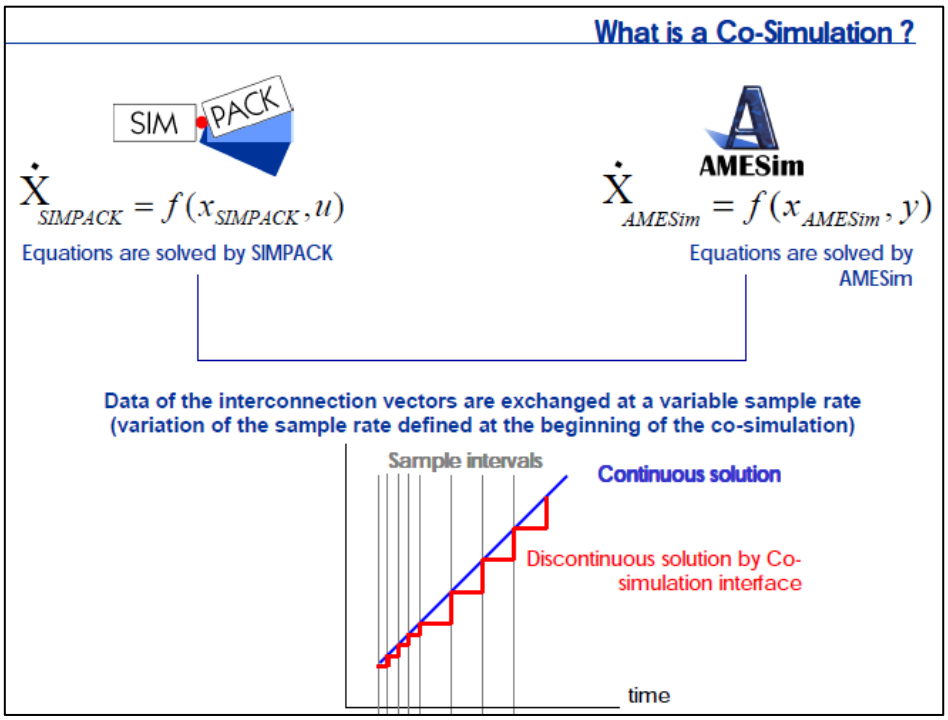
For accessing the license at runtime, the FMU needs two elements in the environment:

- The **AME** environment variable is defined and refers to a directory that contains the **licensing** subdirectory and the **config** subdirectory of the **Simcenter Amesim** distribution. This directory usually contains a license file that defines the information needed for accessing the license provider (local license or server).
- Under **Windows**, the **PATH** environment variable must refer to a directory containing the two dynamic link libraries called **AMELicXYZ.dll** and **LMSLicense_A.dll**. For **Windows** 32 bit, these files are located in the **win32** subdirectory of the **Simcenter Amesim** distribution. For Windows 64 bit, the *win64* subdirectory of the **Simcenter Amesim** must be present in the **PATH** environment variable.
- Under **Linux**, the **LD_LIBRARY_PATH** environment variable must refer to a directory containing a shared object called **libLMSLicenseDLL_A_linux_a.so**. This file is located in the *lnx* subdirectory of the **Simcenter Amesim** distribution for Linux 32 bit and in the **lnx_x64** subdirectory for Linux 64 bit. In the latter case the required file is called **libLMSLicenseDLL_A_linuxem64t_a.so**.
- The **LMS_LICENSE** environment variable must be set to the absolute path of a license file or to **@host**, with host the hostname of a license server.

Confidential C

15.6 AVEC SIMPACK

http://www.simpack.com/fileadmin/simpack/doc/usermeeting01/um01-imagine.pdf



Confidential C

16 SCRIPTING

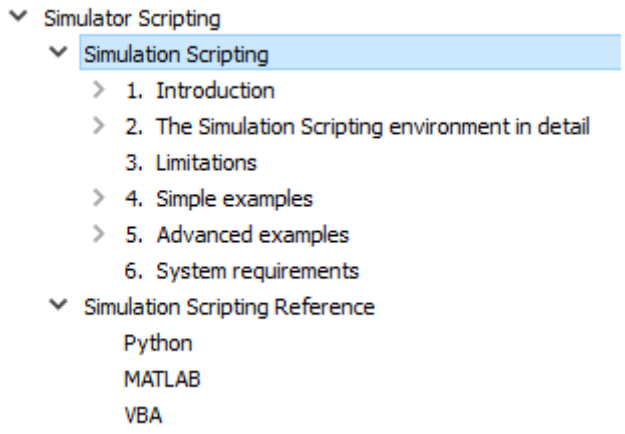
16.1 INTRODUCTION

There are mainly two different ways to "modify models" using scripting :

- simulation scripting
- Amesim API

The first one is mostly used for automation in the Parameter and Simulation mode. It is less powerful, but allows to work once the model is open.

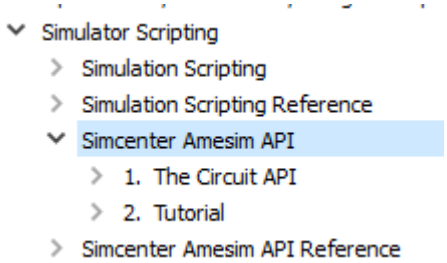
It is explained in this section of the documentation :



A more powerful solution is to use the Amesim API, but the model must be closed.

A model can be built from scratch using this API.

The associated documentation is located in the following section :



Confidential C

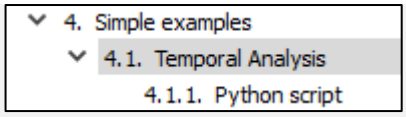
16.2 SIMULATION SCRIPTING

Limitations :

Please be aware of the following limitations when using the Simulation scripting:

- **Undo/Redo** is not supported for operations performed through scripting.
- To access a model and perform simulation, the model must be expanded.
- To run a simulation the model must be compiled
- Parameters changes are not saved in the model.
- For design experiment studies, control inputs and response outputs cannot be changed. They are defined from the **Simcenter Amesim** export setup only.
- Design of Experiment and Optimization are available only as a **Python** module.

Example (from the doc)




```
import os
import pylab
import scipy
import amesim

# Create a 1D map for SIGFXA01-1
os.chdir(r'C:\Users\a071431\Softs\Amesim\4_Models_tmp')
theta = [0, 10, 10.1, 180]
K = [100, 100, 1000, 1000]
K = list(scipy.pi/180.0*scipy.array(K))
amesim.fx2ame(theta,K,'varspringrate.data')
amesim.ameputp('scrdemo','SIGFXA01    instance    1    name    of    ASCII
file','varspringrate.data')

# Running a the simulation and getting results
sim_opt = amesim.amegetsimopt('scrdemo')
sim_opt.startTime = 0.0
sim_opt.finalTime = 1.0
sim_opt.printInterval = 0.01
amesim.amerunsingle('scrdemo', sim_opt)

# Post-processing the results
Results, Varnames = amesim.ameloadvarst('scrdemo', {'MECRL0A_1 shaft angle at
port 2 [degree]'})
print('Angle at time {:f} s is {:f} deg'.format(Results[0][-1], Results[1][-
1]))
pylab.plot(Results[0], Results[1],'r')
pylab.show()
```

A few notes :

- Before running the script, the user must "Write Files for Simulation" (File menu, or Ctrl+Shift+W) in Simulation mode
- the values of the parameters are in the ${circuitname}_.data file
- the names of the fieds to be modified are in the ${circuitname}_.param file
- fx2ame and ameputp are two useful functions here

Confidential C

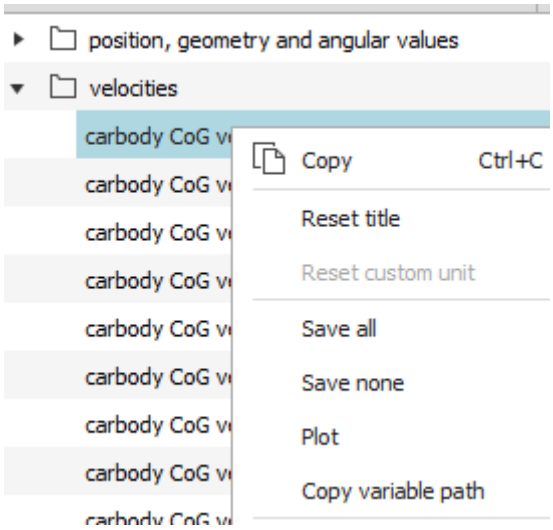
scrdemo_.data | scrdemo_.param

```
40  SIGFXA01 instance 1 boundary conditions
41  SIGFXA01 instance 1 cubic data out of r
42  SIGFXA01 instance 1 column to use for u
43  SIGFXA01 instance 1 output value Param_
44  SIGFXA01 instance 1 out of range input
45  SIGFXA01 instance 1 stairs data out of
46  SIGFXA01 instance 1 initialization meth
47  SIGFXA01 instance 1 name of ASCII file
```

16.3 APPLICATION AU POST-TRAITEMENT

Pour exploiter les résultats d'une simulation, depuis un modèle Amesim ouvert on peut lancer Matlab depuis le menu Tools, puis récupérer des variables à l'aide de la commande `ameloadvarstui`

L'exemple ci-dessous s'appuie sur le modèle Amesim_R.ame (nom typique d'un modèle créé par System Analyst), et l'id des variables (en plus du temps, qui est récupéré par défaut) s'obtient par un clic droit / Copy variable path

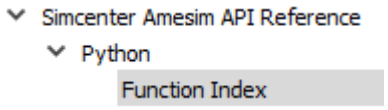



```
[Results, Varnames] = ameloadvarstui('Amesim_R',
{'sensorcha_VelGR0x@DAVIDChassisSensorIcon01'});
plot(Results(1,:), Results(2,:), 'r');
```

16.4 AMESIM API

16.4.1 LIST OF AVAILABLE FUNCTIONS

Confidential C



16.4.2 EXAMPLE

Some examples here :

C:\Users\a071431\OneDrive - Alliance\5_Logiciels\Amesim\6_Scripts

One example from the doc :

```
import os, sys
import ame_apy as apy

print(sys.argv[1])

pn = 'D:\\Workspace\\MSD_xiL'
fn = 'msd_full_fnl.ame'

os.chdir(pn)
apy.AMEInitAPI()
apy.AMEOpenAmeFile(fn)
apy.AMECheckCircuitSubmodels()
apy.AMEWriteAuxiliaryFiles()
apy.AMEGenerateCode()
apy.AMEGenerateRealtimeFiles('dSpace : ds1006 : o86')
apy.AMESaveCircuit()
apy.AMECloseCircuit()
apy.AMECloseAPI()

print("Complete!")
```

Once your script is written, run it from the Python command line with

```
exec(open(myScript.py').read())
```

or from the Windows command line with

```
python -m myScript.py
```

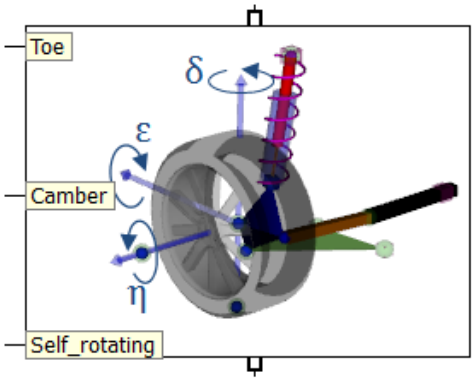
(provided python is well declared in your installation, which is not always the case)

Confidential C

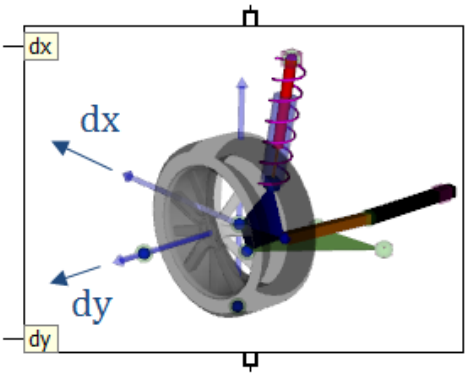
17 VEHICLE DYNAMICS

17.1 WHEEL DEGREES OF FREEDOM

- **δ** : Toe angle (total angle) [degree].
- **ε** : Camber angle (total angle) [degree].
- **η** : Self-rotating angle (total angle) [degree].



- **dX_1** : wheelbase variation [m].
- **dY_1** : half-track variation [m].



source *: Kinematics and Compliance test bench template*

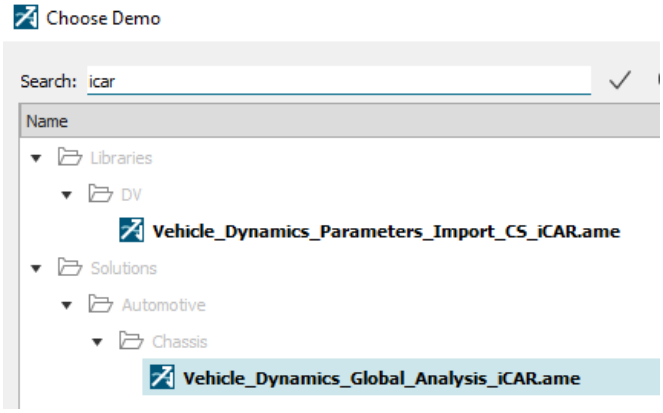
17.2 ICAR LIBRARY

Vehicle Dynamics iCAR

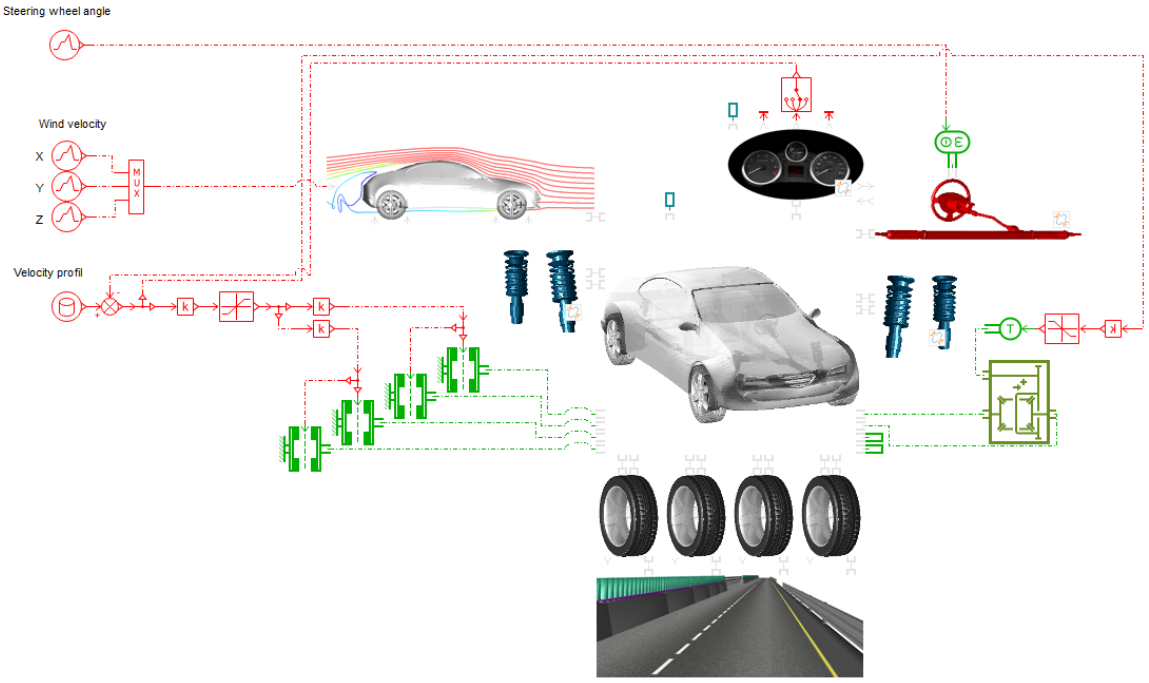
Meant to be used by non-experts, it was a first attempt to democratize vehicle dynamics in Amesim.

Have a look at one of the two demos, preferably the Global Analysis one

Confidential C

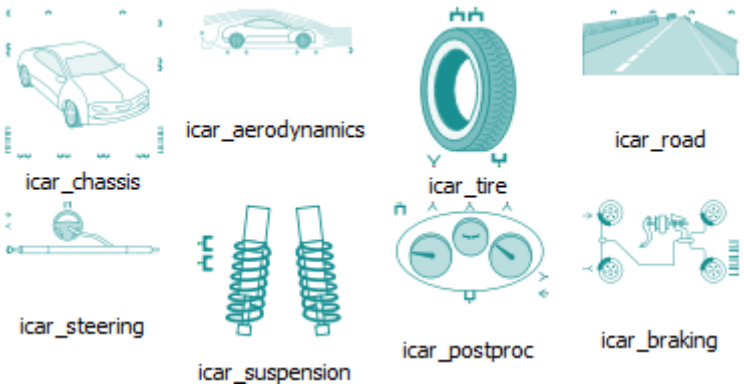


The components are much more integrated than in the standard library, as shown in this sketch :



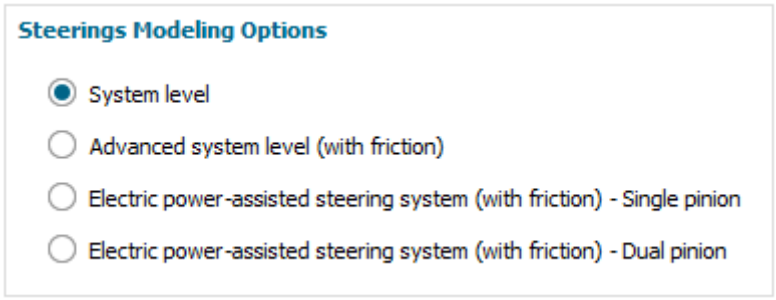
Confidential C

The list of available models is reduced to the minimum :

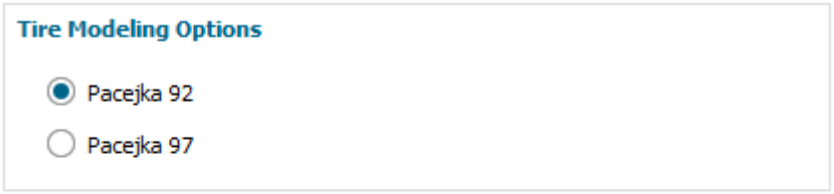


However the library is quite powerful and (imho) underrated.

Some systems offer different modeling options :



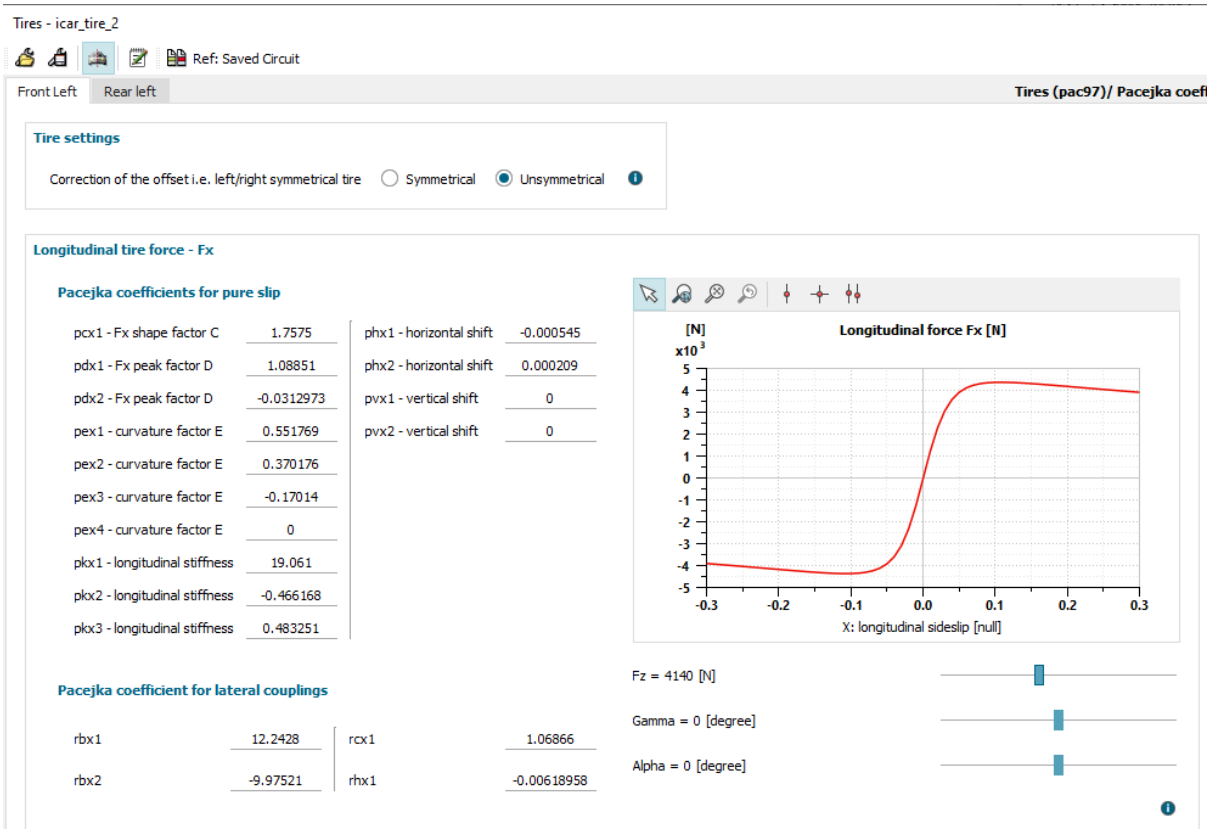
Regarding tires two models are available



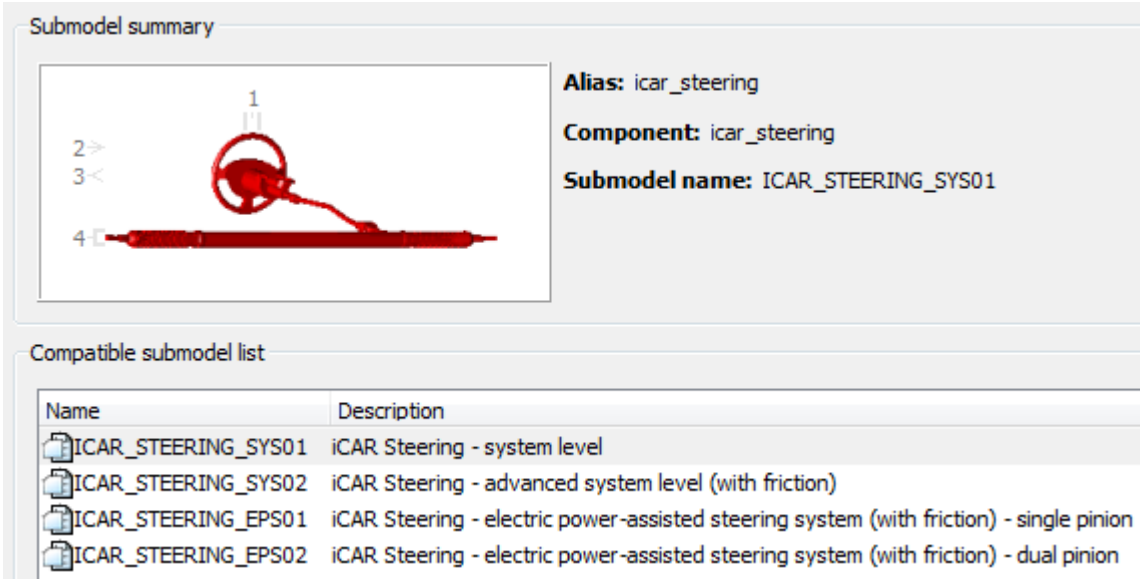


Confidential C

… and an interactive visualization tool is proposed :



iCar steering



Inputs :

Confidential C

- Steering wheel angle
- Vehicle speed (for assist torque map)
- Rack displacement

1. Port 1 - rotary shaft
    - input - Steering wheel angle [degree]
    - input - Steering wheel velocity [degree/s]
    - output - Torque on steering wheel [Nm]
2. Port 2 - input - Vehicle speed [m/s]
3. Port 3 - output - Steering wheel angle [degree/s]
4. Port 4 - linear shaft
    - input - Steering rack displacement [m]
    - input - Steering rack velocity [m/s]
    - output - Force on steering rack [N]

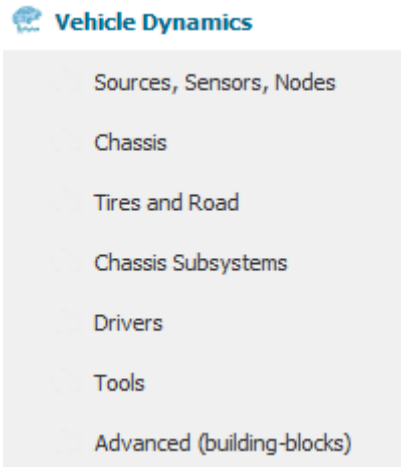
Outputs

- steering wheel torque
- steering wheel angle ??
- rack force

ATTENTION : la doc parle parfois d'une vitesse volant en tr/min, parfois en °/s

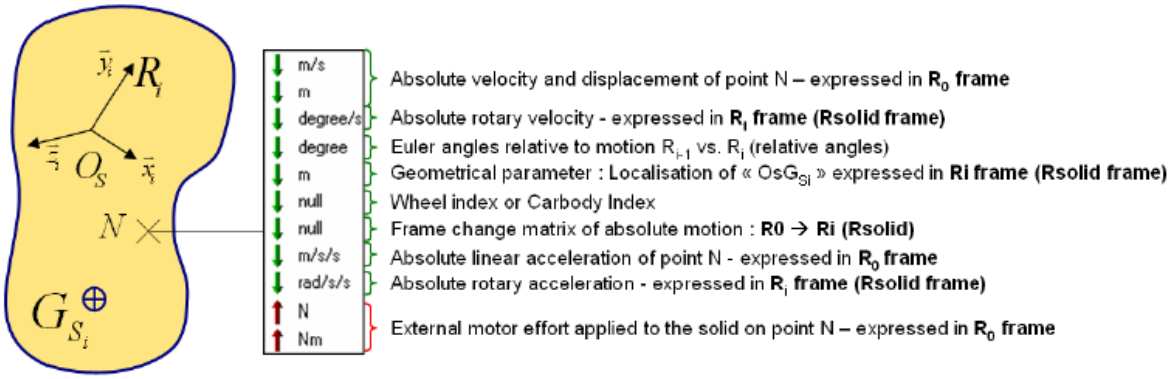
Confidential C

17.3 STANDARD LIBRARY



Attention : cette bibliothèque n'est en général pas « activée » par défaut. Il faut donc l'ajouter manuellement (pour rappel, voir dans Modeling / Category path list, cf §1.2).

Chaque corps (caisse, roue, etc …) dispose d'un port « 3D Mechanical » :



Les 4 roues sont indicées de la manière suivante :

- AVG : 11
- AVD : 12
- ARG : 21
- ARD : 22

Confidential C

17.4 CHASSIS

17.4.1 INTRO

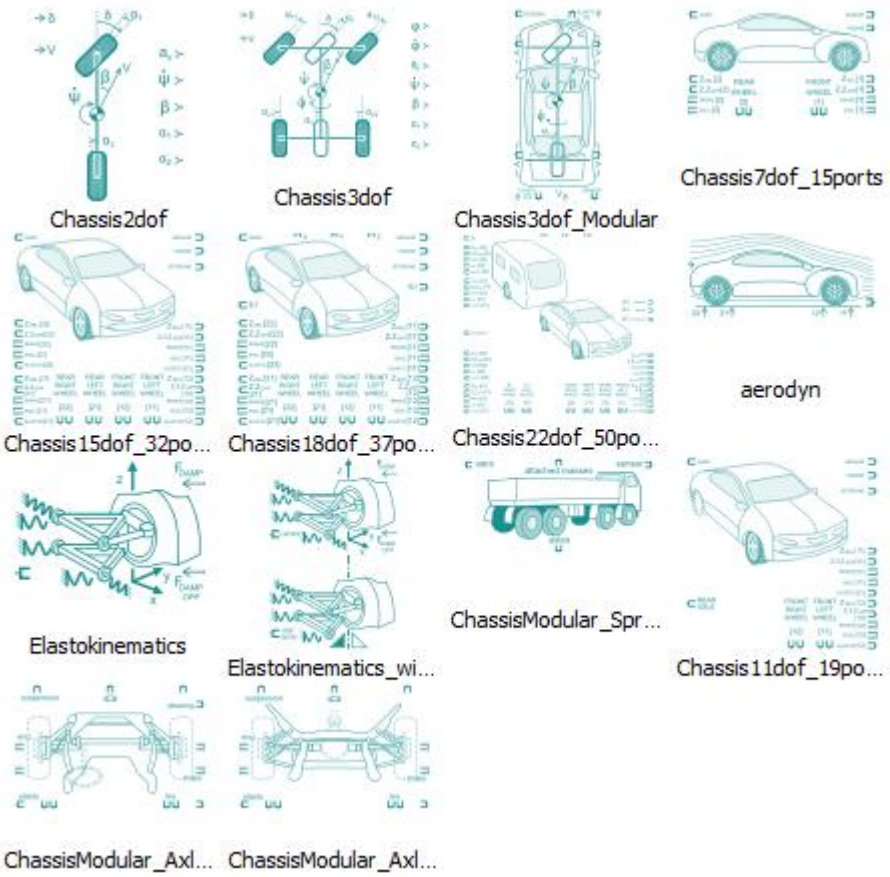
2 dof : single-track (« bicycle » model)

3 dof : interesting for rapid prototyping

7 dof : for longitudinal motion

15 dof : reference high-fidelity model

18 dof :



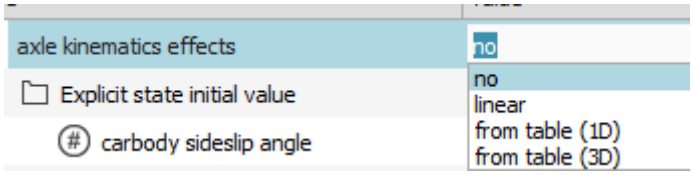
17.4.2 CHASSIS 3DOF

Pertinent pour une bonne partie des usages hors "prestation comportement".

Définition des inducteurs principaux (dimensions, masses et inerties, raideur anti-roulis, démul …)

▾ Masses and inertias			
mass of the vehicle	**1500**	kg	M
yaw inertia	800	kgm**2	Izz
roll inertia	600	kgm**2	Ixx
▾ Geometry			
roll center definition	definition above center of gravity		rollcenterdef
height of center of gravity (absolute)	0.5	m	hg
height of roll center (absolute)	0.035	m	h0
front wheelbase	1	m	a1
rear wheelbase	1.5	m	a2
front half track	0.7	m	b1
rear half track	0.7	m	b2
▾ Axle characteristics			
front anti-roll stiffness	52000	N*m/rad	Kr1
rear anti-roll stiffness	52000	N*m/rad	Kr2
roll damper rating	1000	N*m/rad/s	bphi
steer angle ratio (delta = ratio.Yn)	7	rad/m	Demul
front castor offset	0.03	m	ch

Cinématique modulable, avec plusieurs choix possibles :



Option linéaire : on définit des coefs induits par le roulis pour pince et carro

toe coefficient induced by roll - front axle	**0**	null	eps1
toe coefficient induced by roll - rear axle	**0**	null	eps2
camber coefficient induced by roll - front axle	**0**	null	lambda1
camber coefficient induced by roll - rear axle	**0**	null	lambda2

Confidential C

Avec "table 1D", on peut spécifier

- coefs de pince et carro induits par le roulis
- table de variation braquage roue = f(pos. crem)

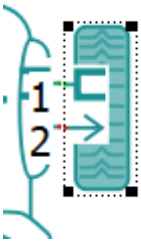
toe coefficient induced by roll - front axle	**0**	null
toe coefficient induced by roll - rear axle	**0**	null
camber coefficient induced by roll - front axle	**0**	null
camber coefficient induced by roll - rear axle	**0**	null
Front left axle (11) : filename for steer angle kinematic table DELTA 11 = f(Yn)	data/DELTA11.dat	
Front right axle (12) : filename for steer angle kinematic table DELTA 12 = f(Yn)	data/DELTA12.dat	

Avec "table 3D" :

- carro induit
- table de variation braquage roue = f(z, pcrem, zopp)

camber coefficient induced by roll - front axle	**0**	null
camber coefficient induced by roll - rear axle	**0**	null
Front left axle (11) : filename for steer angle kinematic table DELTA 11 = f(Z , Yn , Zopp)	data/DELTA11.dat	
Front right axle (12) : filename for steer angle kinematic table DELTA 12 = f(Z , Yn , Zopp)	data/DELTA12.dat	
Rear left axle (21) : filename for toe angle kinematic table DELTA 21 = f(Z , Zopp)	data/DELTA21.dat	
Rear right axle (22) : filename for toe angle kinematic table DELTA 22 = f(Z , Zopp)	data/DELTA22.dat	

Le châssis 3dof peut en outre être associé à un modèle de pneu spécifique :



Compatible submodel list

Name	Description
VDTIRELAT01	tire model - lateral force only (linear)
VDTIRELAT02	tire model - lateral force only (non linear)

Confidential C

Le submodel linéaire est caractérisé par des rigidités de dérive et de carrossage :

Parameters of Tiremodel_lateral [VDTIRELAT01-4]

Title	Value	Unit	Tags	Name
cornering stiffness	**65000**	N/rad		D
camber stiffness	1750	N/rad		Carr

Params pneu linéaire

Parameters of Tiremodel_lateral [VDTIRELAT02-1]

Title	Value	Unit
lateral force (implicit)	0	N
tire modeling	**non linear (load transfer effect)**	
lateral force calculation method (non linear case only)	...using implicit variable	
a3 - slope at the origin BCDo - maximum cornering stiffness	1399	N/degree
a4 - slope at the origin BCDo - vertical load for maximum c...	6.307	kN
camber stiffness	1750	N/rad

Params pneu non-linéaire

Dépendance au Fz :

$$F_{y\,tire} = -\left(a_3 \sin\left(2\arctan\left(\frac{F_z}{a_4} \right) \right) \right) . \alpha_{tire}$$

- F_{tire} : Tire Lateral force
- F_z : Vertical load on the
- α_{tire} : Tire slip
- a_3 : maximum cornering stiffness
- a_4 : vertical load for maximum cornering stiffness

Confidential C

Possibilité de prendre en compte le temps de réponse du pneu :

(#) lateral force (explicit)		0	N
tire modeling		**non linear (load transfer effect)**	
lateral force calculation method (non linear case only)		**...using first order lag**	
a3 - slope at the origin BCDo - maximum cornering stiffness		1399	N/degree
a4 - slope at the origin BCDo - vertical load for maximum c...		6.307	kN
camber stiffness		1750	N/rad
time constant		0.015	s

$$\frac{dF_{y\,tire}}{dt} = -\left(F_{y\,tire} - D.\alpha_{tire}\right)\frac{V_x}{L} = -\left(F_{y\,tire} - D.\alpha_{tire}\right)\frac{1}{\tau}$$

- F_{tire} : Tire Lateral force
- α_{tire} : Tire slip
- **D** : Cornering stiffness
- V_x : Velocity of the center of tire contact (usually named "B") on x axis - Tire frame
- **L** : Relaxation length

Confidential C

17.4.3 CHASSIS 15 DOF

The functional analysis of the whole 15 DOF chassis model is presented on the following figure :

- Inputs of chassis model :

« Wheel » efforts at center of wheel (A_2)

Efforts on spindle at center of wheel (A2)

Suspension efforts (Spring, Damper, end stop, antiroll bar)

Efforts of drive shaft on wheel and spindle (action) - Efforts of engine shaft on carbody (reaction)

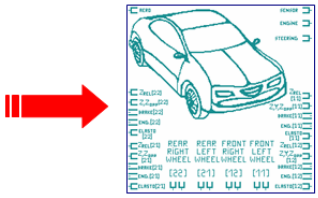
Brake efforts on caliper (wheel/spindle)

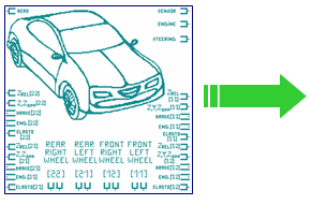
External efforts on carbody (Aero., Engine, Sources,...)

Efforts on steering rack.

Induced elastic displacement (Elastokinematics)

Tire efforts





- Outputs of chassis model :

Kinematic elements of carbody (Displacement, velocity, acceleration) - translation (cog) and rotation

Kinematic elements of spindle (Displacement, velocity, acceleration) - translation (cog) and rotation

Kinematic elements of wheel (Displacement, velocity, acceleration) - translation (cog) and rotation of the rim plane

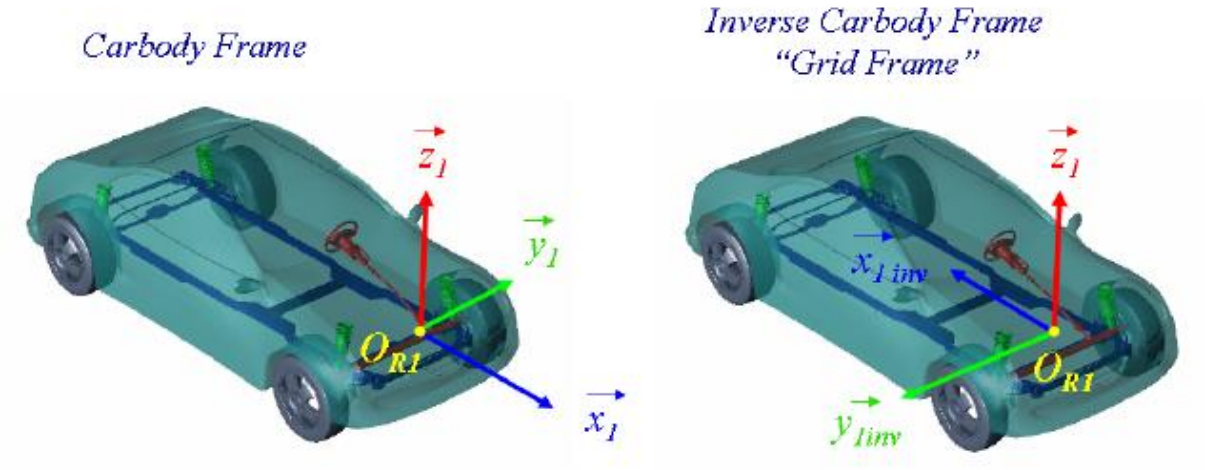
Displacement of steering rack

Generalized wheel lift (vertical lift)

Constraint efforts (Elastokinematics)

17.5 COORDINATE SYSTEMS

Two main systems are defined, the origin of which is usually not at the center of gravity :



17.5.1 ABSOLUTE FRAME (R0)

Absolute frame (R0): this frame is fixed to the ground and is used to locate the position of the carbody during the simulation. Therefore, this is the frame used to initialize the carbody position at the beginning of the simulation.

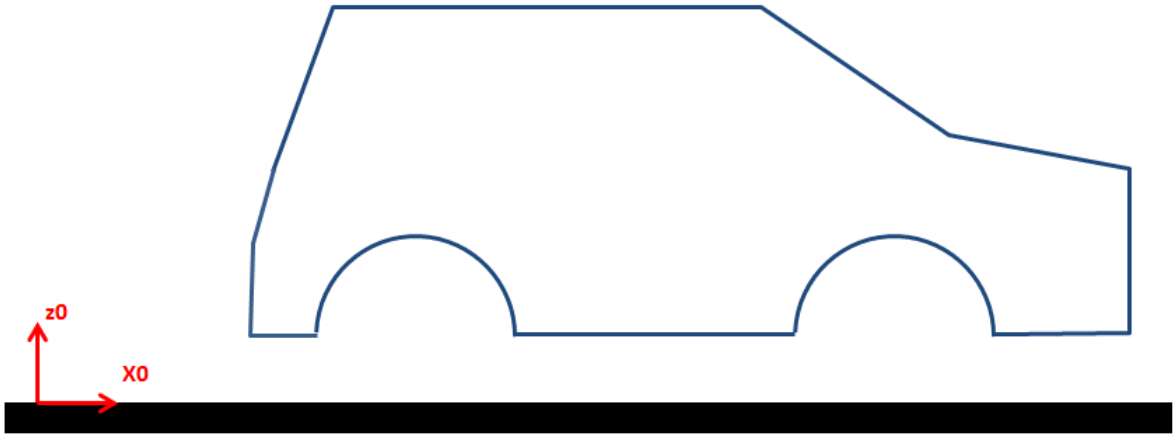


Figure 1: absolute frame R0

17.5.2 CARBODY GRID FRAME (R1)

This frame is fixed with respect to the carbody solid. Therefore, it is used to describe its geometry. It is defined the following way:

- Origin: center of the front wheel axle.
- X axis: oriented towards the rear of the vehicle
- Y axis: oriented towards the right of the vehicle
- Z axis: defined by the 2 previous axises.

Confidential C

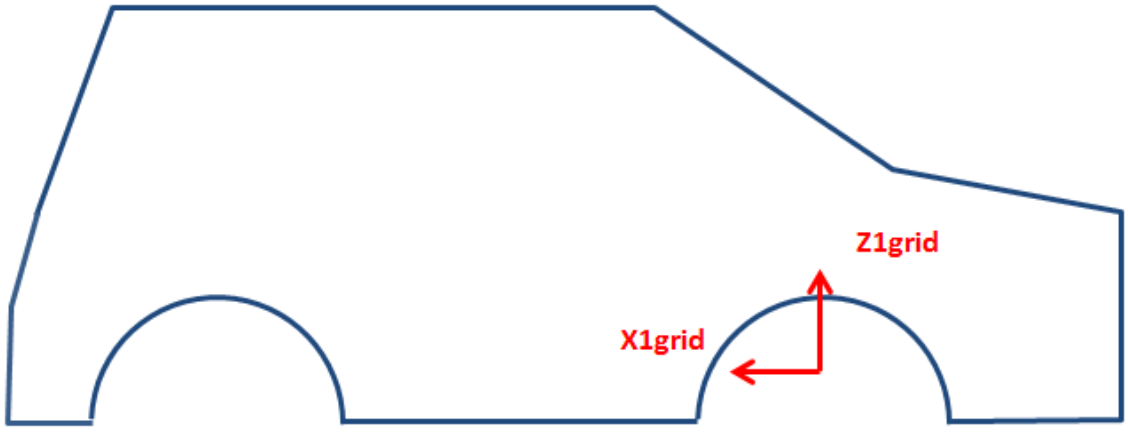
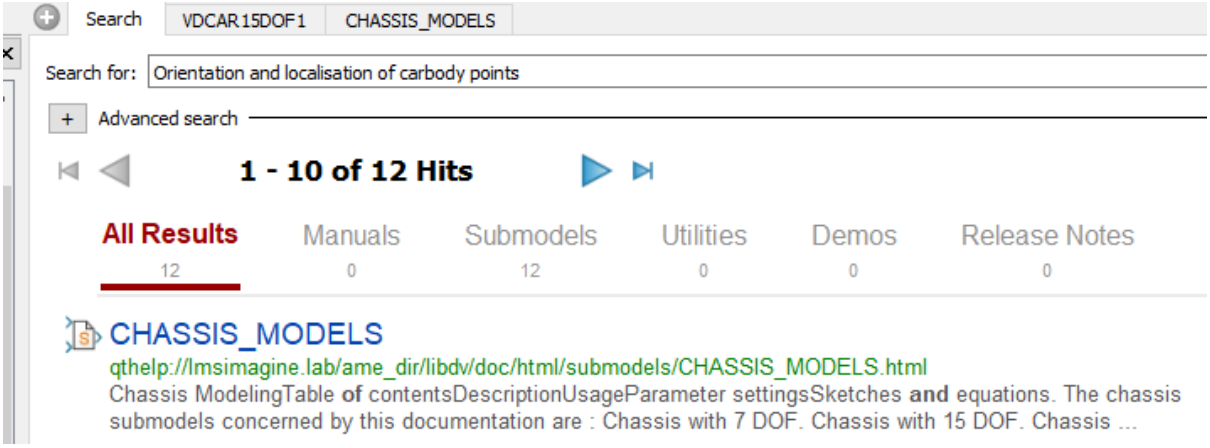


Figure 2: carbody grid frame R1grid

17.5.3 HELP DOCUMENTATION

For more information, see the Chassis Models help, looking for *Orientation and localisation of carbody points*



Then 3.1.4

3. Parameter settings
    1. Description
        1. Introduction
        2. Explicit state initial value (Initial conditions)
        3. Masses and inertias
        4. Orientation and localisation of carbody points

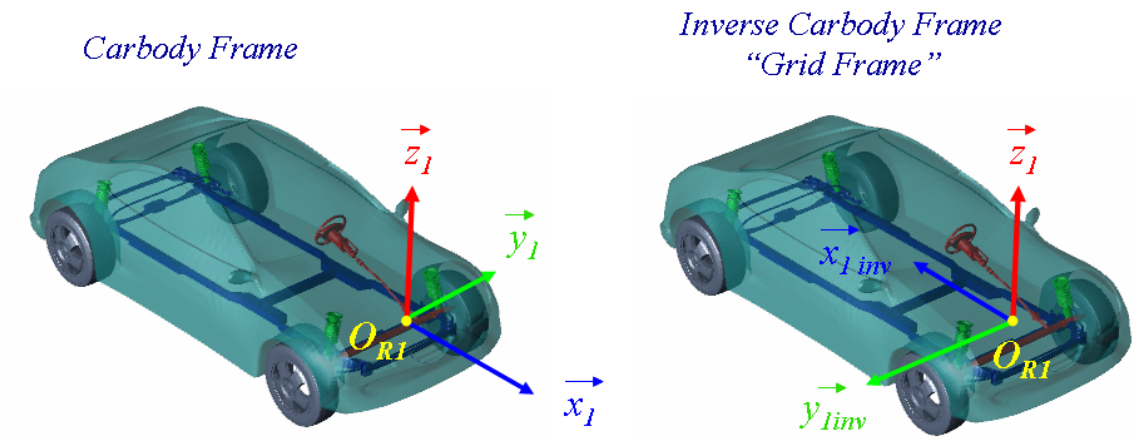
… leads to the following chapter

Orientation and localization of carbody points

The user can set the location of all characteristic points of sprung mass in carbody frame ($\mathbf{R_1}$) or carbody "grid" frame ($\mathbf{R_{1\ inv}}$). The carbody grid frame is constructed from carbody frame by a rotation of an angle of 180° around $\mathbf{Z_1}$ axis :

$$\begin{bmatrix} x_{grid} \\ y_{grid} \\ z_{grid} \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \end{bmatrix}$$

Moreover, the location setting can be done with two reference points : the center of gravity of carbody (**G**) or the center of carbody frame $\mathbf{O_{R1}}$, fixed in carbody frame and commonly locate at front axle system. The following figure shows both $\mathbf{R_1}$ and $\mathbf{R_{1\ inv}}$ frames, centered at point $\mathbf{O_{R1}}$ for example :



The frame selection is available with the following pull-down menu :

Expression frame for geometrical localisation of carbody points: Inv. Car Grid frame / Veh. Dyn. Car frame (for Ogrid = G)

- Inv. Car Grid frame / Veh. Dyn. Car frame (for Ogrid = G)
- Car Grid frame (origine Ogrid) , Xgrid rear - directedness

We point out the fact that this selection concerns only the location of carbody points and has no influences on kinematic tables frame selection nor calculation of the dynamics .

The center of carbody frame $\mathbf{O_{R1}}$ (also commonly noted " $\mathbf{O_{grid}}$ ") must be located vs. center of gravity **G** :

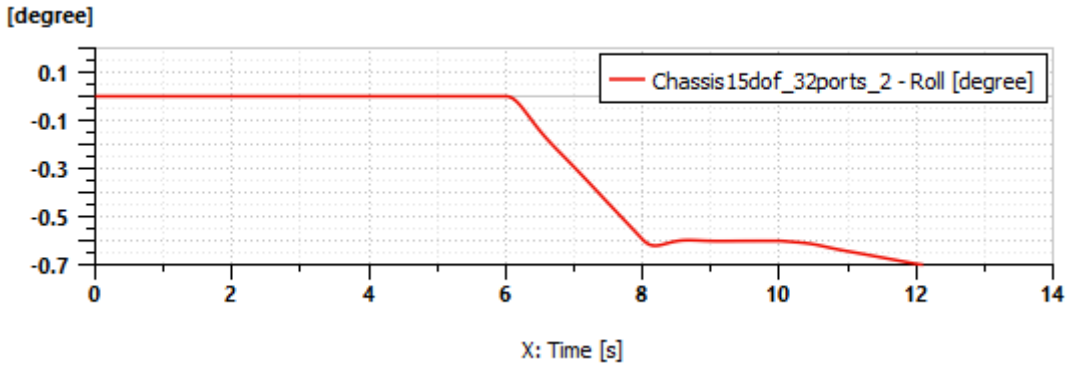
X - Position of center of gravity vs. center of carbody grid frame - [Ogrid G]
Y - Position of center of gravity vs. center of carbody grid frame - [Ogrid G]
Z - Position of center of gravity vs. center of carbody grid frame - [Ogrid G]

Confidential C

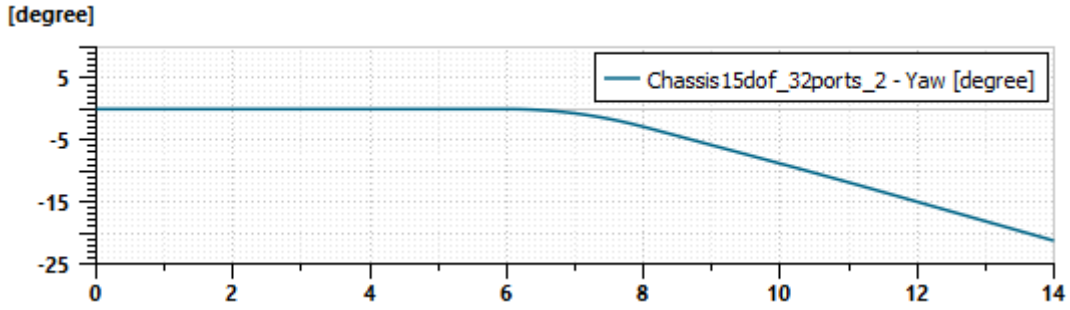
17.6 SIGN CONVENTIONS

17.6.1 EULER ANGLES

From the demos (see for example §**Erreur ! Source du renvoi introuvable.**) we can see that when the vehicle steers to the right, the roll is negative :



as well as the yaw :

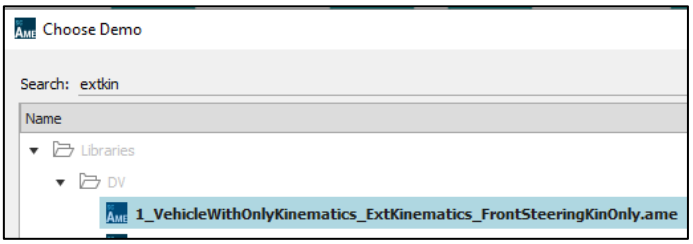


… as expected according to the ISO convention.

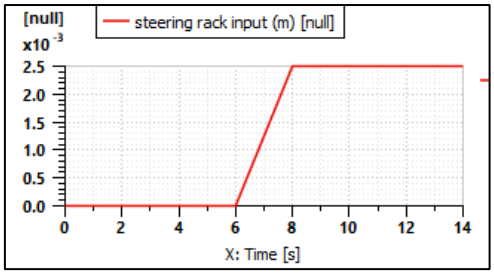
Confidential C

17.6.2 KINEMATICS

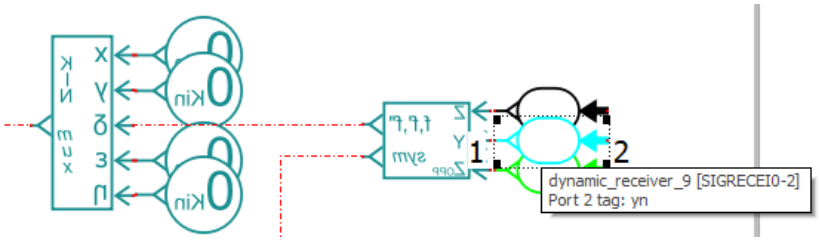
Open the following demo :



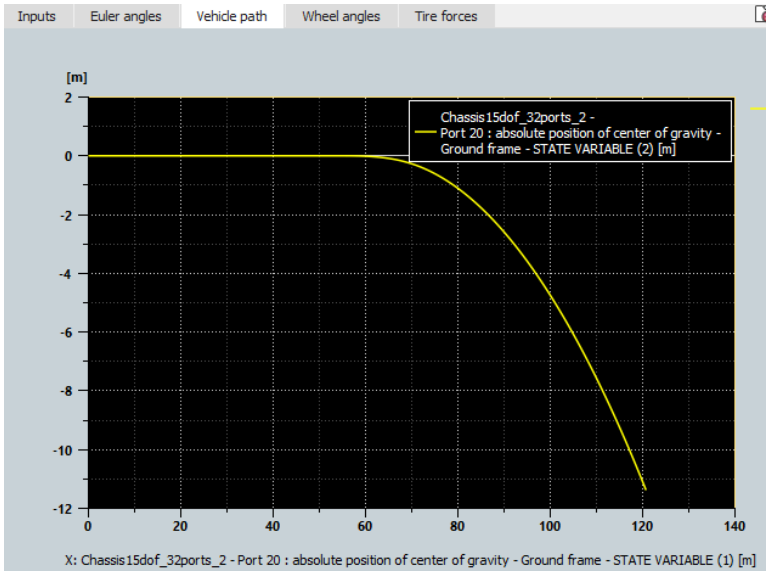
A vehicle is subject to a positive rack displacement input :



The rack displacement y_n has the same sign (and value of course !) as the rack input :

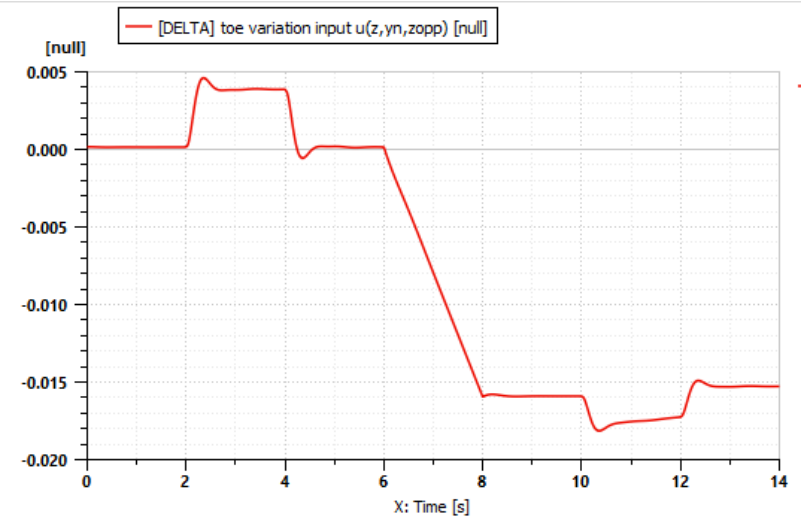


The vehicle turns right :

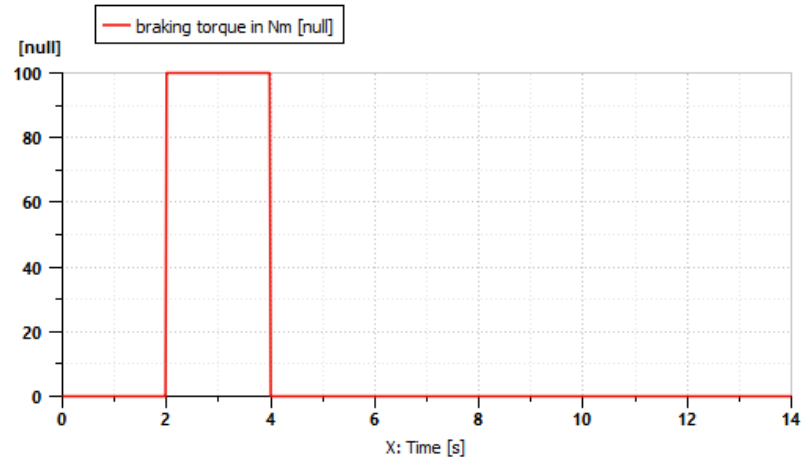


Confidential C

And the toe variation is negative once the steering begins :



NB : the "step" between t=2 and t=4s is only due to a braking input



So we have the convention that :

positive rack displacement => turn right => negative toe variations

This explains the negative sign in this expression in front of the rack displacement

$$toe = \frac{wheelbase}{\frac{wheelbase}{\tan\left(\frac{-rack_displacement}{steering_ratio * pinion_radius}\right)} - half_track} + 0.5 * wheel_stroke$$

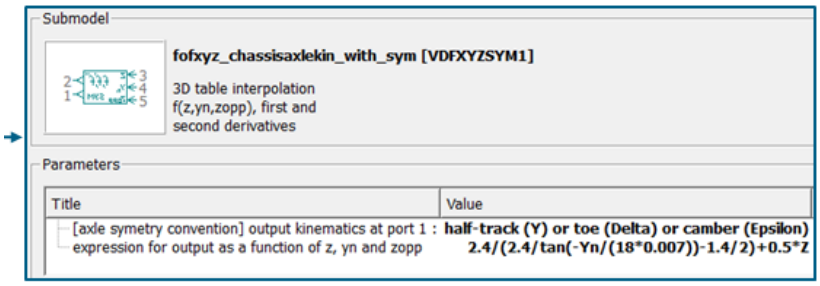
Confidential C

17.7 KINEMATICS

17.7.1 POLYNOMIAL OR TABULATED DEFINITION

Example, from the help :

CHASSIS_AXLE_KINEMATICS



i.e. mathematic expression of the pure Jeantaud function + linear toe dependency with stroke

17.7.2 EXPLANATION

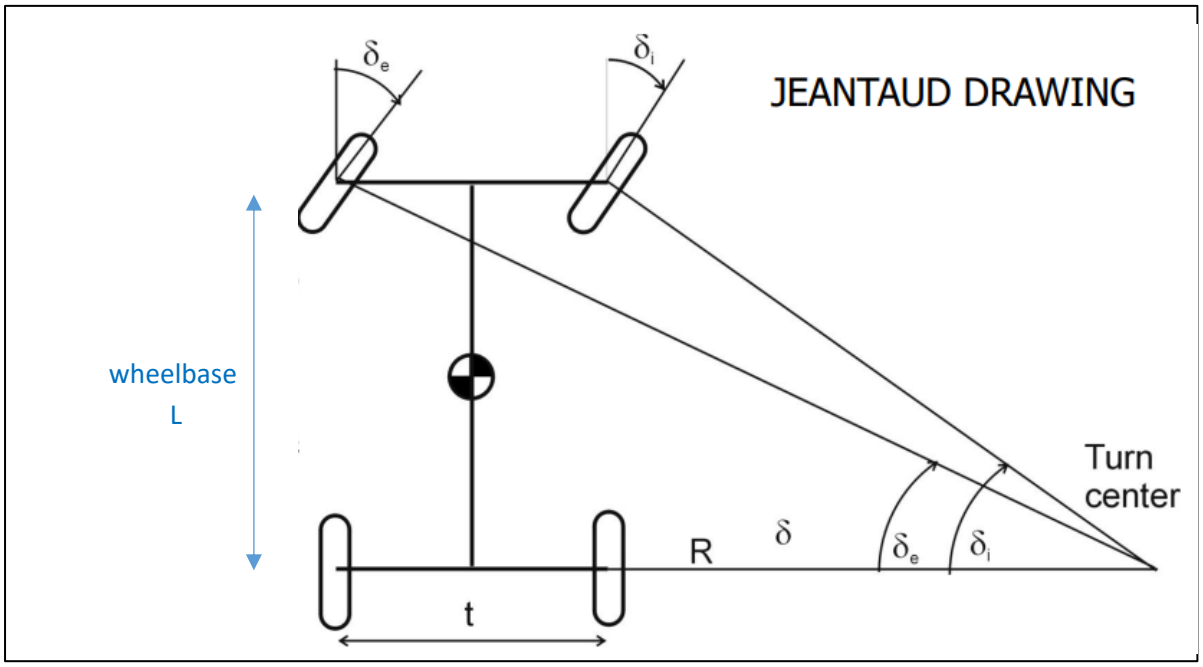
Let's rewrite the above expression without the numerical values :

$$toe = \frac{wheelbase}{\frac{wheelbase}{\tan\left(\frac{-rack_displacement}{steering_ratio * pinion_radius}\right)} - half_track} + 0.5 * wheel_stroke$$

Confidential C

Let's try to figure it out, starting from the purely kinematic Jeantaud-Ackermann description

Caution : in this source as well as in Amesim the steering angle is denoted δ, whereas it is α in Renault/MADA conventions



source

The Ackermann-Jeantaud condition can be written

$$\tan(\delta_i) = \frac{L}{R - \frac{t}{2}}$$

We can approximate the average front steering angle

$$\tan(\delta_{front}) = \tan\left(\frac{\delta_i + \delta_e}{2}\right) = \frac{L}{R}$$

So

$$R = \frac{L}{\tan(\delta_{front})}$$

$$R = \frac{L}{\tan\left(\frac{\delta_{SW}}{n}\right)}$$

where δ_{SW} is the steering wheel angle and n the steering ratio such as $n = \frac{\delta_{SW}}{\delta_f}$

Then

$$\tan(\delta_i) = \frac{L}{\frac{L}{\tan\left(\frac{\delta_{SW}}{n}\right)} - \frac{t}{2}}$$

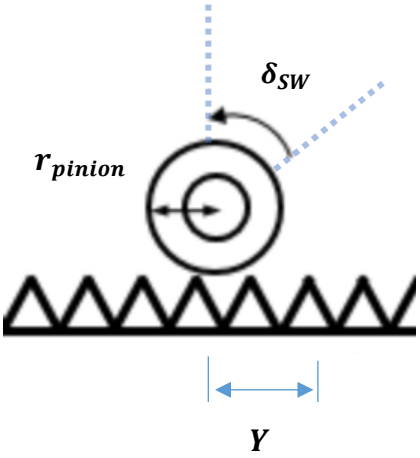
Confidential C

And since

$$\delta_{SW} = \frac{Y}{r_{pinion}}$$

where Y is the rack displacement and r_{pinion} the pinion radius





we can rewrite

$$\tan(\delta_i) = \frac{L}{\frac{L}{\tan\left(\frac{Y}{r_{pinion} * n}\right)} - \frac{t}{2}}$$

We recognize the above (litteral) expression

$$toe = \frac{wheelbase}{\frac{wheelbase}{\tan\left(\frac{-rack_displacement}{steering_ratio * pinion_radius}\right)} - half_track} + 0.5 * wheel_stroke$$

… if we put aside the wheel stroke dependency.

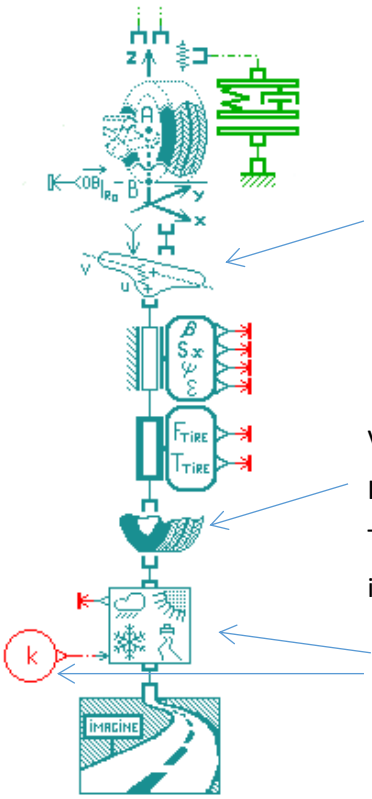
17.7.3 EFFETS BROUILHET

Jacking effects in english.

Can be post-processed

17.8 TIRES

17.8.1 PNEU / CEINTURE / CINEMATIQUE



VDSLIPxx computes sideslip and longitudinal slip from current speed and orientation. It is the crucial submodel for handling rolling / stopped transitions

VDTIRExx computes Fx and Fy from Fz and slip
It takes into account the relative adherence of the ground
Tires are often characterized on ground where max(Fx/Fz) is close to 1, but they can also be characterized on wet roads

VDADHERxx accepts a value and send it to VDTIRExx to be used as a correction for traction level. A value of 1 means no correction, a value of 0.5 means halving the level of traction available.

Confidential C

17.8.2 MODELS

Permettent de calculer les efforts et moments en fonction de la charge verticale et des angles caractéristiques (dérive, carrossage, glissement …)

A tire is modeled by the connection of three components :

- The tire kinematics computes all kinematics elements from the wheel center to the tire/ground contact point.
- Tire belt model with or without relaxation length.
- Tire model as Pacejka magic formula, or simplified approaches likes Dugoff or Rocard modelings

NB : le point de contact roue-sol est calculé à partir de la position du centre roue, en tenant compte du rayon sous charge.

17.8.3 INDEX (IDX) FILES

Used to import .tir files in Amesim

Make the correspondance between the keyword name in the .tir and its equivalent in Amesim

Extract :

```
<!--[LEFT_TIRE]-->
<!--[LONGITUDINAL_COEFFICIENTS]-->
<paramlist>
        <param name="tire_MF5X_pcx1@DAVIDTire4wIcon01.DAV_tire_road_interaction_22" type="scalar">
          <keyword>PCX1</keyword>
        </param>
        <param name="tire_MF5X_pdx1@DAVIDTire4wIcon01.DAV_tire_road_interaction_22" type="scalar">
          <keyword>PDX1</keyword>
        </param>
        <param name="tire_MF5X_pdx2@DAVIDTire4wIcon01.DAV_tire_road_interaction_22" type="scalar">
          <keyword>PDX2</keyword>
        </param>
```

Drawback : since the parameter name in Amesim contains the model name, it most of the time requires a “replace all” operation in a text editor to be adapted.

Confidential C

17.8.4 SIGN CONVENTIONS FOR PACEJKA

In the Magic Formula 5.2 implementation in Amesim the sign conventions are the same as the ones in the handbook “Tyre and Vehicle Dynamics” by Hans Pacejka.

In the 2002 edition, we find at §4.3.2 and in particular p.187 the following convention : C, D and μ must be positive both for longitudinal and lateral behaviour

$$C_x = p_{Cx1} \cdot \lambda_{Cx} \qquad (>0) \qquad (4.E11)$$

$$D_x = \mu_x \cdot F_z \cdot \zeta_1 \qquad (>0) \qquad (4.E12)$$

$$\mu_x = (p_{Dx1} + p_{Dx2} df_z) \cdot \lambda_{\mu x} / (1 + \lambda_{\mu V} V_s / V_o) \qquad (>0) \qquad (4.E13)$$

$$C_y = p_{Cy1} \cdot \lambda_{Cy} \qquad (>0) \qquad (4.E21)$$

$$D_y = \mu_y \cdot F_z \cdot \zeta_2 \qquad (4.E22)$$

$$\mu_y = (p_{Dy1} + p_{Dy2} df_z) \cdot (1 - p_{Dy3} \gamma^{*2}) \cdot \lambda_{\mu y} / (1 + \lambda_{\mu V} V_s / V_o) \qquad (>0) \qquad (4.E23)$$

NB : the sign of D_y is not explicit, but is a direct consequence of the sign of μ_y

As a result we should have in the .tir files :

- PCX1, PDX1 and PKX1 positive
- PCY1, PDY1 and PKY1 positive

e-mail J. Lagnier 05/05/2022

- *En Pacejka 5.0 (P97), implémenté dans le VDTIRE02A (non utilisé dans DAVID) et VDTIRE03A, le calcul accepte de travailler avec mu_x et mu_y négatifs, et on peut donc caractériser des pneus avec PCy1 < 0 et PDy1 < 0.*
- *En Pacejka 5.2 (P2002), dans VDTIRE03A, on implémente strictement les limites spécifiées dans les équations du livre de Pacejka (cf ref dans la doc du VDTIRE03A i.e. « Pacejka H.B., "Tire and Vehicle Dynamics", SAE Publication, Elsevier Science, ISBN 0 7680 1126 4, 2002, pp 184-196 »), et donc il y a dans ce cas (i.e. 5.2) une contrainte sur la caractérisation qui n’existaient pas en 5.0.*

17.9 ROAD

Plusieurs modèles de routes sont disponibles :

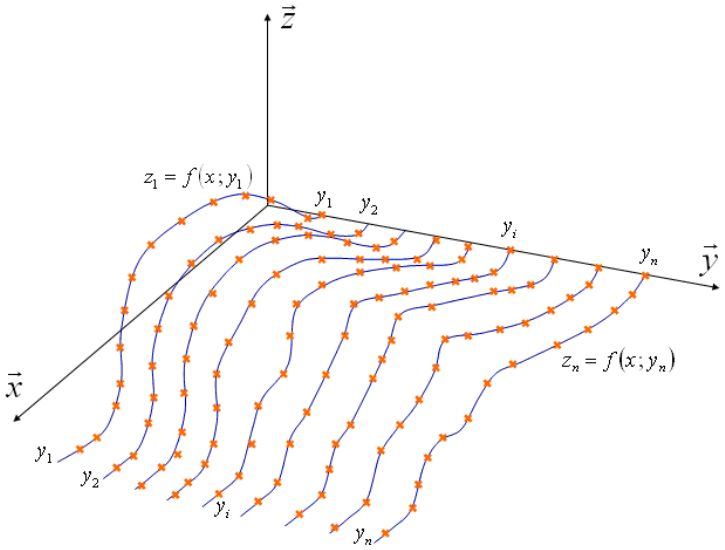
Road model Iconname: Road	
VDROAD00	flat road
VDROAD01	sinusoidal road (flat part before the sinusoidal part)
VDROAD02	road as a slope - flat part first and cylindrical part to ensure continuity with the slope
VDROAD03	uneven road - flat part at the beginning
VDROAD04	3D Road profil

Si l'on veut introduire du dévers, il faut utiliser le submodel VDROAD04, dont la doc est peu explicite au premier abord :

The user must specify the name of the data file. The data file must provide the data in the following order:

```
y1 N1
   x1,1 z1,1
   x1,2 z1,2
   ...
   x1,N1 z1,N1
y2 N2
   x2,1 z2,1
   x2,2 z2,2
   ...
   x2,N2 z2,N2
...
   ...
   ...
yM NM
   xM,1 zM,1
   xM,2 zM,2
   ...
   xM,NM zM,NM
```

Confidential C



A priori la déclaration d'une route dont le dévers est de 1° se résume aux lignes suivantes, ci-dessous sur une longueur de 100 mètres :

```
0.0    2
0.     0.017455
100.   0.
1.0    2
0.     0.017455
100.   0.
```

Y du 1e pt → `0.0`

Nb de lignes de données → `2`

`0. 0.017455` ← **sin(1°) … puisqu' on définit ici le dévers sur une largeur unitaire (cf Y du 2e pt)**

`100. 0.`

Y du 2e pt → `1.0`

`2`

`0. 0.017455`

`100. 0.`

Confidential C

17.10 STEERING

17.10.1 MODELE 3DOF

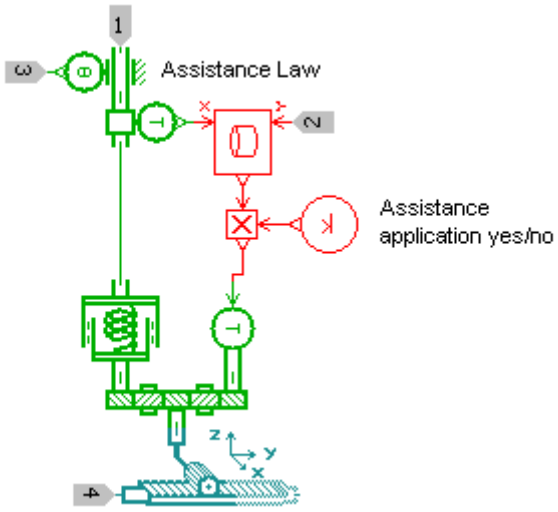
Dans le modèle 3DOF, l'info direction est alimentée par un ratio angle de braquage à la roue / déplacement crémaillère

La valeur par défaut est de 7rad/m

steer angle ratio (delta = ratio.Yn)	7 rad/m

En supposant par exemple une démul de 15, on a au volant 0.105 rad/mm
En inversant ça, on obtient 9.52 mm/rad soit 59.8 mm/tr, et on retrouve bien l'ordre de grandeur usuel d'un rack ratio.

17.10.2 REPRESENTATION FONCTIONNELLE SIMPLIFIEE



17.10.3 RAPPEL GRAVITAIRE

Regarder du côté du composant VDTFSUSP0

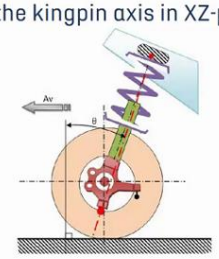
VDTFSUSP0 - Advanced variable transformer between (Z,Yn,Zopp) and (Spring, Damper, Upper and lower stop, Antiroll bar)



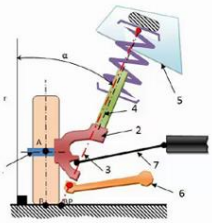
La prise en compte du rappel gravitaire est mentionnée dans la présentation commune PSA-Siemens de 2020

Rack force can change with spring compression

The caster angle of the kingpin axis in XZ-plan: about 5°



The inclination of the kingpin axis in YZ-plan: about 10°



Steering the road wheels increases the reacting force of the ground and then the carbody rises.
When releasing the steering wheel, the reacting force of the ground decreases and lets the weight of the carbody returns the knuckle in its starting position, this means the steering rack comes back on center position

Displace the rack about 20mm, the spring length differs about 2mm, and the spring force varies from a few daN

PSA GROUPE RD2 / DCTC / ICDV / CLDR / CFVL – Eric BOITTIAUX December 2020 3

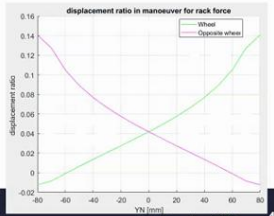
modeling in AMESIM : the work provided by the steering rack is equal to the work provided by the spring

- AMESIM uses « transformers » submodels to make « equal » the work of the rack and the suspension

Carbody  Suspension components : spring, damper, bumpers, anti-roll bar

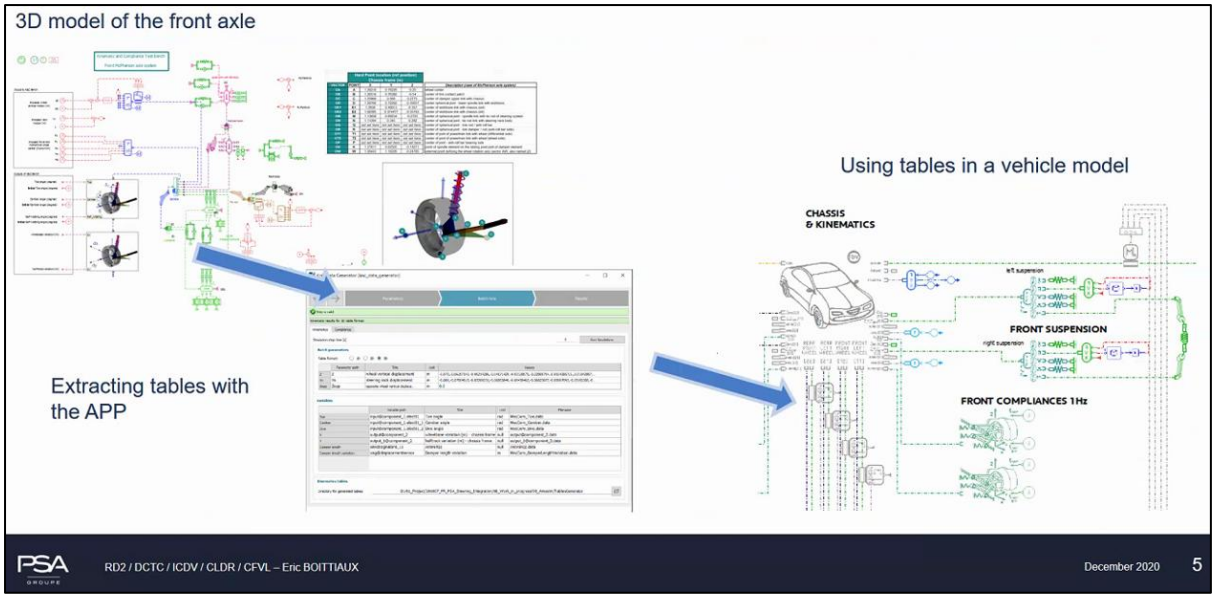
- The relation between suspension work and steering work is determined with a functional table :

Z spring = g(Zwheel, Yrack, Zopposite wheel) ➔ partial derivatives dZspring/dYrack



- This relation is reversible ⇔return ability of the steering system
- With a MBS tool or a 3D model in AMESIM, you can generate this tables

PSA GROUPE RD2 / DCTC / ICDV / CLDR / CFVL – Eric BOITTIAUX December 2020 4





17.11 DAMPERS

Voir la thèse de Master de (Reineh, 2012)

Confidential C

18 VEHICLE MODELS FROM PWT LIBRARY

Vehicle models are available in the powertrain library, mostly for longitudinal motions. But the 2D-3D models can also take roll motion into account.

18.1 MODELS "1D"

Compute acceleration from torque input

- rolling friction
- road slope
- aerodynamic losses
- No tire model
- No suspension



veh1D_2axles **veh1D_3axles** **veh1D_4axles**

18.2 MODELS "2D" : OBSOLETE

Replaced by 2D-3D model



veh2D_2wh *veh2D_2ax* *veh2D_3ax* *veh2D_4ax*

Confidential C

18.3 MODEL "2D-3D"

Can be modeled in a plane (3 degrees of freedom) or in space (6 degrees of freedom).



veh2D_3D

Suitable for drivability, comfort or vibration studies

With the 2D modeling, the field of study is limited to longitudinal, vertical and pitch motion. With the 3D option, one can also study the roll motion. This is particularly interesting for longitudinal engine layouts as the engine creates torque in the roll axis direction of the carbody.

It is also important that the 3D modeling level only intends to address comfort or vibration analysis on a straight road

Confidential C

19 CONTROLE

19.1 EPID (EXTENDED PID)

https://community.sw.siemens.com/s/article/How-to-easily-control-a-non-linear-system-using-the-extended-PID-in-Simcenter-Amesim

20 LINK WITH CAD : CAD2AME

L'utilitaire CAD2AME permet d'intégrer des composants géométriques, au format Parasolid ou STEP.

C'est un viewer assez élémentaire permettant à des utilisateurs sans connaissance spécifique en CAO de procéder à des opérations simples : combinaisons booléennes, découpage, mesure …

Confidential C

21 NEURAL NETWORKS

21.1 [TECH TIPS SIMCENTER AMESIM] REDUCING MODEL WITH NEURAL NETWORKS

https://www.youtube.com/watch?v=dPiVhf-hwMw

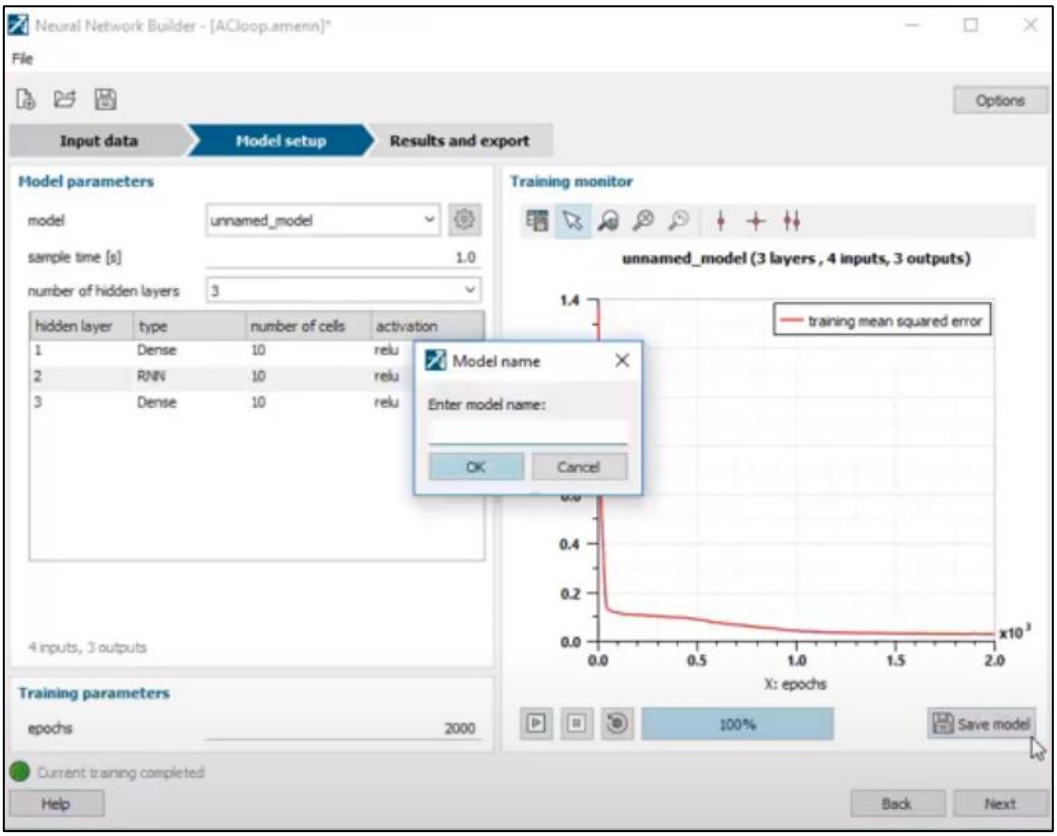
Add inputs and outputs to watch variables

Run batch simulations to train the model

Open NN Builder in the App space



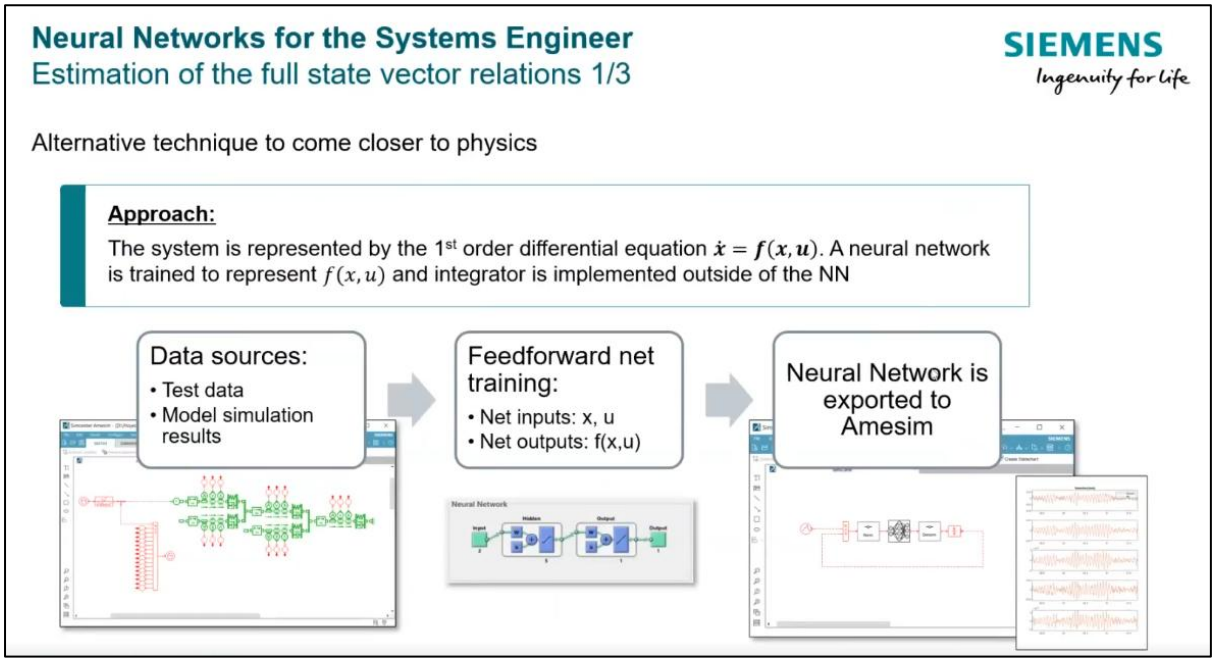




21.2 CHOIX DE L'ARCHITECTURE DE RESEAU

Lors du « Live Tech » organisé par Siemens pour Renault en 2020 (présentation de Peter Mas), il est dit qu'une meilleure stratégie est peut-être de ne plus chercher à identifier le modèle complet par un RNN, car cela contraint à passer en simulation à pas fixe.

A la place ce sont les équations d'état qui sont identifiées :



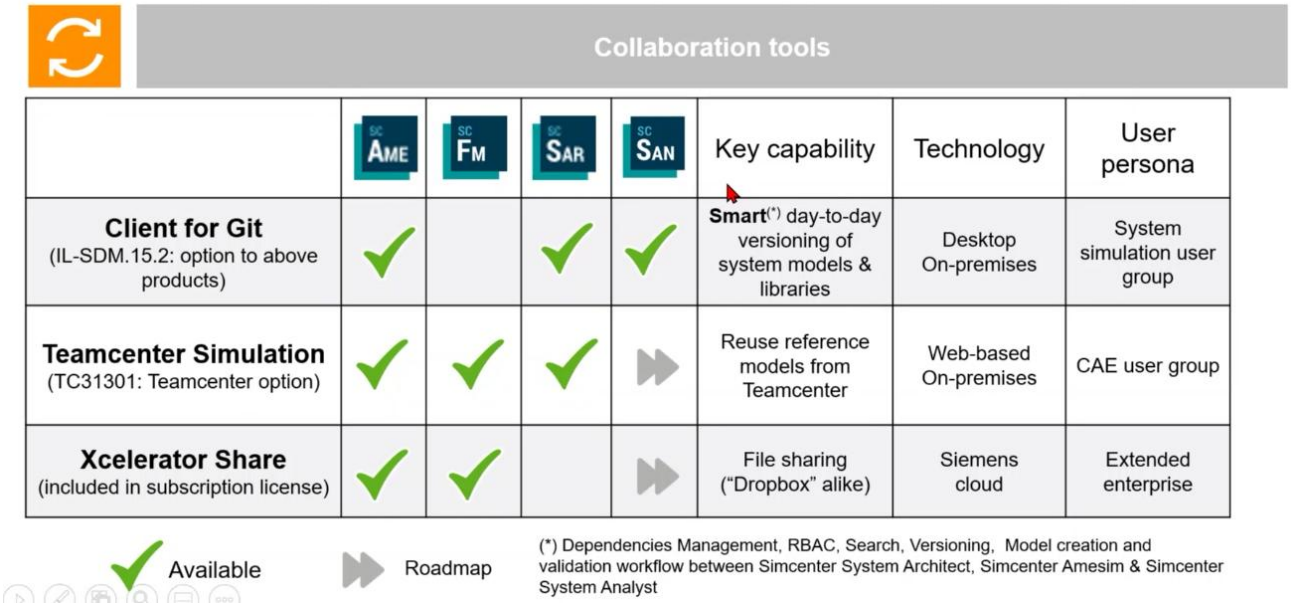
21.3 UTILISATION DE NN POUR LE CONTROLE

Lors du « Live Tech » organisé par Siemens pour Renault en 2020 (présentation de Yerlan Akhmetov), est mentionnée l'utilisation d'un NN comme modèle embarqué par un contrôleur MPC.

Confidential C

22 RELEASES – NOUVEAUTES

22.1 2021.2



Collaboration tools

	SC AME	SC FM	SC SAR	SC SAN	Key capability	Technology	User persona
Client for Git (IL-SDM.15.2: option to above products)	✓		✓	✓	**Smart**(*) day-to-day versioning of system models & libraries	Desktop On-premises	System simulation user group
Teamcenter Simulation (TC31301: Teamcenter option)	✓	✓	✓	»	Reuse reference models from Teamcenter	Web-based On-premises	CAE user group
Xcelerator Share (included in subscription license)	✓	✓		»	File sharing ("Dropbox" alike)	Siemens cloud	Extended enterprise

✓ Available » Roadmap

(*) Dependencies Management, RBAC, Search, Versioning, Model creation and validation workflow between Simcenter System Architect, Simcenter Amesim & Simcenter System Analyst

Confidential C

23 ANNEXES

23.1 HISTORIQUE SOCIÉTÉ

23.1.1 IMAGINE S'ADOSSE AU BELGE LMS INTERNATIONAL (02/2007)

http://www.lesechos.fr/13/02/2007/LesEchos/19856-097-ECH_ingenierie---imagine-s-adosse-au-belge-lms-international.htm

L'entreprise Imagine de Roanne (Loire), fournisseur de renommée mondiale de logiciels de simulation destinés aux systèmes de commande électromécanique et électronique pour l'automobile et l'aéronautique, va être achetée par son concurrent belge LMS. Ce dernier lancera en mars une offre publique d'achat (OPA), au prix de 33,40 euros par action, sur les 7 % de flottant de la société cotée au marché libre de la Bourse de Paris. Cette OPA valorise l'entreprise à 16,5 millions d'euros. Le reste du capital est aujourd'hui détenu par le président du conseil de surveillance, Michel Lebrun, des membres de son entourage, des cadres, mais aussi l'Institut Français du Pétrole (15 %).

Basé à Louvain, LMS affirme qu'il va ainsi devenir « le seul acteur du marché à disposer d'une plate-forme logicielle complète à même de modéliser des systèmes intelligents pour véhicules, avions et autres engins mécaniques complexes ». Parmi les principales applications d'AMESim, le logiciel phare d'Imagine, figurent la mise au point de systèmes de commande électronique pour l'injection de carburant, pour contrôler la sécurité des véhicules, ainsi que pour les trains d'atterrissage et les ailes des avions.

Imagine a été créé en 1986 par un économiste et un ingénieur français, ainsi qu'un « numéricien » britannique. Il s'est fortement développé pour atteindre un effectif de près de 120 personnes. En 2006, son chiffre d'affaires était d'un peu plus de 11 millions (réalisé pour plus des trois quarts à l'exportation), avec un résultat net de l'ordre de 15 % à 20 %. Pour 2007, la prévision de croissance de l'activité est au moins égale aux 35 % de l'an dernier.

Une fois achevée cette opération de croissance externe, LMS, qui possédait déjà un bureau à Bièvres (Essonne), disposera de 30 implantations dans le monde et d'un chiffre d'affaires global de 130 millions avec 800 salariés

23.1.2 SIEMENS ANNONCE L'ACQUISITION DE LMS INTERNATIONAL (11/2012)

http://w5.siemens.com/belux/web/fr/presse/presse/automation/pages/acquisition-lms-international.aspx

Avec plus de 1 200 salariés et un chiffre d'affaires avoisinant les 140 millions d'euros aux neuf premiers mois de 2012 (du 1er janvier au 30 septembre), soit une croissance de plus de 24 % par rapport à 2011, LMS est une société internationale qui travaille en partenariat avec près de 5 000 entreprises du secteur de l'automobile, de l'aéronautique et d'autres industries de fabrication de pointe. Cette entreprise privée fournit des logiciels de simulation mécatronique, des systèmes de test et des services d'ingénierie à tous les constructeurs automobiles et aéronautiques figurant dans classement des 500 premières entreprises américaines ainsi qu'à leurs équipementiers. LMS propose une plateforme logicielle complète permettant de modéliser, simuler et tester des véhicules, des avions et d'autres produits complexes afin d'en optimiser l'acoustique, les vibrations, les oscillations, la résistance à la fatigue et la dynamique.

Les activités de LMS seront intégrées dans le portefeuille de logiciels dédiés à la gestion du cycle de vie produit (PLM – product lifecycle management) de la division Industry Automation de Siemens.

Le prix d'achat de LMS se monte à près de 680 millions d'euros (hors excédent de trésorerie).

23.1.3 2020 : LES 25 ANS

https://blogs.sw.siemens.com/simcenter/simcenter-amesim-celebrating-25-years-of-innovation/

Michel Lebrun was a university teacher when he founded the company Imagine S.A. in 1986. Meanwhile, he was giving courses about the **bond graph theory** during summer university, gathering a wide community of experts to exchange about that engineering approach.

This practical approach enables to model and understand the energy transfer between the inner components of the system, as well as the energy storage, dissipation and modulation leading to the dynamics of the system, whatever the underlying physics.

That's by getting associated with Pr Claude W. Richards, in April 1994, who was programmer working on a solver at Bath University that the adventure started. Michel Lebrun brought the physics and bond graph knowledges while Claude W. Richards brought the solver.

This partnership led to the **first commercial release of AMEsim in 1995**. This first version was dedicated to fluid control systems and hydraulic topics, with a first and unique HCD (Hydraulic component design) library called AMEbel (basic element library).

Indeed, the first application of this AMEbel library was for Sandvik company (formerly Tamrock), for the design of hydraulic hammers. Customers such as Bosch or Peugeot-Citroën followed, some of whom still use Simcenter Amesim today.

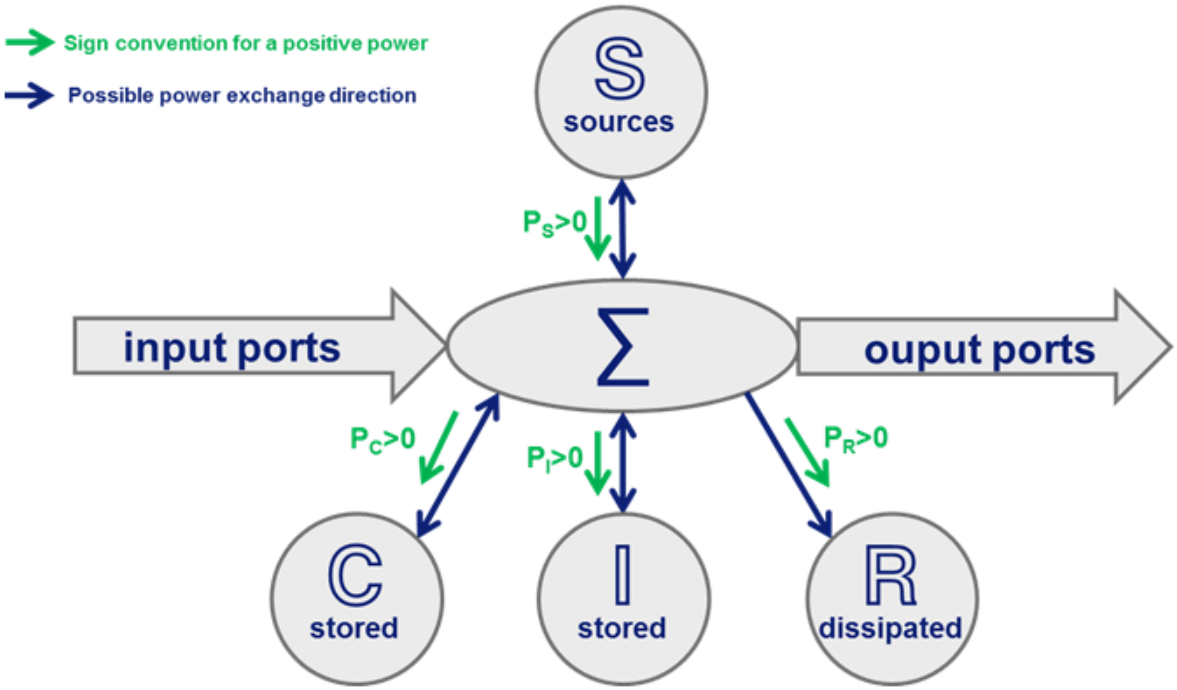
23.2 FORMALISME BOND-GRAPH DANS AMESIM

Voir le § sur le sujet dans le doc « ingénierie système »

A system Σ:

- receives or gives energy at its ports;
- receives energy from or gives energy to the Sources (whether hidden internally to the submodel or from an external source submodels);
- stores or restores energy in its Capacitive and Inertial elements;
- loses energy in its Resistive elements and gives it to the environment.

Confidential C



A noter qu'on peut distinguer deux types de composants résistifs :

- ceux dont la puissance dissipée P_R est perdue dans l'environnement
- ceux dont la puissance dissipée P_{RS} reste dans le système (après conversion dans un autre domaine de la physique, par ex mécanique => thermique)

P_{RS} n'apparait donc pas dans les bilans.

La doc Amesim ajoute ainsi :

I, C and R elements are the most common in Amesim. However, *S* sources have been implemented in components where energy is supplied in an implicit manner (i.e. without flowing explicitly through any of its physical ports). We can also have resistive *RS* sources to describe situations where energy is both:

- converted from another energy form into heat;
- used in the system by another component (e.g. transmitted to another component of the circuit through a thermal port) or by the component itself (e.g. kept in the component to increase its temperature), or both.

Resistive sources are mainly used in libraries that mix a bond graph formalism with thermal effects.

Confidential C

	I: Inertia	C: Capacity	R: Resistance
Translation			
Rotation			
Thermal	Not Appropriate		
Electrics			

23.3 VERSIONS – RELEASES

Comment enregistrer un modèle dans une version précédente ?

Ce n'est pas possible depuis l'interface, mais la solution proposée ici a fonctionné (au moins une fois …) :

https://community.plm.automation.siemens.com/t5/System-Simulation-Forum/How-to-save-the-amesim-simulation-file-into-an-earlier-version/td-p/493581

> Outside Amesim, open model.ame file with a zip editor like 7zip. Edit model_.cir inside and check the third line. Change AME_VERSION="14.2.0.x.x.z" into AME_VERSION="11". It's important to use a version older than the target, so use 11 not 12.1. Finally, 7zip should ask to update the content of the ame file, say yes.

Confidential C

24 PUBLICATIONS

Source http://oss.jishulink.com/caenet/forums/upload/2014/12/27/392/23680231207181.pdf

- Alirand M., **"Study and analysis of an active self leveling suspension"**, IEEE International Conference on Systems, Man and Cybernetics, Le Touquet, France, October 1993, pp 222-227
- Alirand M., Botelle E., Sau J., **"Modeling a force control actuator for semi active car dampers - Basics "**, 16th IAVSD Symposium on Dynamics of Vehicles on Roads and Tracks, Pretoria, South Africa , September 1999, pp 1-4
- Ney Y., **"Design methodology for automotive suspension systems - Fluid power software applications"**, SIA conference on Fluid Power and Transportation, Roanne, France, May 1999, pp 1-2
- Botellé E., Alirand M., Sau J., **"Modeling a force control actuator for semi active car damper: Flow valve analysis"**, 5th Int. Symposium on Advanced Vehicle Control, AVEC 2000, Detroit, MI, 2000, pp 1-8
- Alirand M., Urvoy E., Botelé E., **"Modeling a force control actuator for semi active suspension: Application"**, SIA Congress on Vehicle Dynamics, Lyon, France, June 2001, pp 1-6
- Alirand M., Botelle E., J. Sau, **"Modeling a force control actuator for semi-active car damper : Pressure controlled valve analysis"**, Scandinavian Int . Conference on Fluid Power, Linkoping, Sweden, June 2001, pp 1-6
- Lee C.T.., Moon B.Y., **"Study of the simulation model of a displacement sensitive shock absorber of a vehicle by considering the fluid force"**, Proc of the IMechE, Part D, Journal of Automobile Engineering, vol 219, 2005, pp 965-975
- Cimba D., Wagner J., Baviskar A. **"Investigation of active torsion bar actuator configuration to reduce vehicle body roll"**, Vehicle System Dynamics, vol 44, n° 9, September 2006, pp 719-736
- Gerth H., Resch R., Freimann R., **"Automated controller design for an anti-roll system"**, European AMESim User Conference, Strasbourg, France, March 2006, pp 1-9
- Lino P., Maione B., **"Near optimum control of a full car active suspension system"**, LMS Engineering Simulation Conf Europe 2008, Paris, France, October 2008,
- Falfari S., Brusiani F., Pelloni P., **"Coupling Between 1D-3D Simulation Results to Predict Cavitation in Motorcycle Forks"**, SAE paper n° 09FFL-0117, 2009, pp 1-13
- Falfari S., Brusiani F., Cazzoli G., **"Setup of a 1D model for simulating dynamic behaviour of motorcycle forks"**, SAE paper n° 2009-01-0226, 2009, pp 1-14

- Gubitosa M., Anthonis J., Albarello N., Desmet W., **"A Computer Aided Engineering Approach For the Optimal Design of An Active Suspension System"**, Proc ASME 2009 Int. Design Engineering Technical Conf., San Diego, CA, 2009, pp 1-11
- Lindvai-Soos D., **"Functional development process of the electric anti-roll stabilizer eARS"**, Vehicle Dynamics Expo, Stuttgart, Germany, 2010,
- De Bruyne S., Anthonis J., Gubitosa M., Van der Auweraer H., **"Model Based Actuator Management for a Hydraulic Active Suspension System Improving Comfort Performance by Advanced Control"**, Proc of the ASME 2011 Int Mechanical Engineering Congress & Exposition, Denver, CO , November 2011, pp 1-9
- Kim H.., Lee H., **"Study Model-based fault-tolerant control for an automotive air suspension control system"**, Proc of the IMechE, Part D, Journal of Automobile Engineering, vol 225, 2011, pp 1462-1480
- Moshchuk N., Li Y., Opiteck S. **"Air suspension system model and optimization"**, SAE Paper N° 2011-01-0067, 2011, pp 1-14
- De Bruyne S., Anthonis J., Gubitosa M., Van der Auweraer H, **"Modeling Model Based Design of a Hydraulic Active Suspension System"**, Int. Symposium on Advanced Vehicle Control, AVEC 2012, Seoul, Korea, 2012, pp 1-9
- Manlong P., Feng L., Wenkui F., Yunqing Z., **"Multi-domain modeling and robust design of hydraulic shock absorber"**, 2nd Int. Conf on Computer Application and System Modelling, paris, France, June 2012, pp 1128-1131
- Sadeghi Reineh M., Pelosi M., **"Physical Modeling and Simulation Analysis of an Advanced Automotive Racing Shock Absorber using the 1D Simulation Tool AMESim"**, SAE Paper n° 2013-01-0168, 2013, pp 1-11
- Pelosi M., Subramanya K., Lantz J., **"Investigation on the Dynamic Behavior of a Solenoid Hydraulic Valve for Automotive Semi-Active Suspensions Coupling 3D and 1D Modeling"**, 13th Scandinavian Int. Conference on Fluid Power, Linkoping, Sweden, June 2013, pp 1-10
- Barale S., Plisson A., Guillet J., Lagnier J., Alirand M., **"Improved Functional Modelling in Comfort Analyses for Hydraulic Suspension Testing"**, Chassis Tech Int. Conference, Munich, Germany, June 2013, pp 1-11

Confidential C

25 REFERENCES

Reineh, M. (2012). *Physical Modeling and Simulation Analysis of an Advanced Automotive Racing Shock Absorber using the 1D Simulation Tool AMESim.* Récupéré sur http://liu.diva-portal.org/smash/get/diva2:620275/FULLTEXT01.pdf